

DeepSeekMath: Pushing the Limits of Mathematical Reasoning in Open Language Models

Zhihong Shao^{1,2*†}, Peiyi Wang^{1,3*†}, Qihao Zhu^{1,3*†}, Runxin Xu¹, Junxiao Song¹
Xiao Bi¹, Haowei Zhang¹, Mingchuan Zhang¹, Y.K. Li¹, Y. Wu¹, Daya Guo^{1*}

¹DeepSeek-AI, ²Tsinghua University, ³Peking University

{zhihongshao, wangpeiyi, zhuqh, guoday}@deepseek.com
<https://github.com/deepseek-ai/DeepSeek-Math>

Abstract

Mathematical reasoning poses a significant challenge for language models due to its complex and structured nature. In this paper, we introduce DeepSeekMath 7B, which continues pre-training DeepSeek-Coder-Base-v1.5 7B with 120B math-related tokens sourced from Common Crawl, together with natural language and code data. DeepSeekMath 7B has achieved an impressive score of 51.7% on the competition-level MATH benchmark without relying on external toolkits and voting techniques, approaching the performance level of Gemini-Ultra and GPT-4. Self-consistency over 64 samples from DeepSeekMath 7B achieves 60.9% on MATH. The mathematical reasoning capability of DeepSeekMath is attributed to two key factors: First, we harness the significant potential of publicly available web data through a meticulously engineered data selection pipeline. Second, we introduce Group Relative Policy Optimization (GRPO), a variant of Proximal Policy Optimization (PPO), that enhances mathematical reasoning abilities while concurrently optimizing the memory usage of PPO.

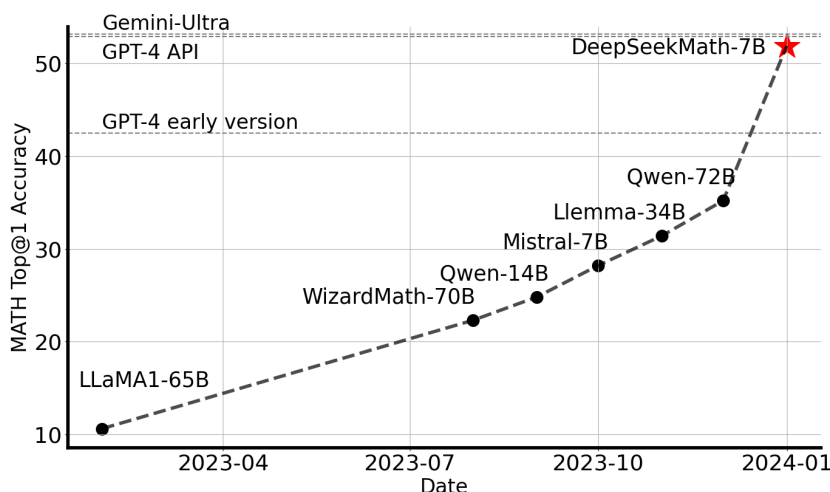


Figure 1 | Top1 accuracy of open-source models on the competition-level MATH benchmark (Hendrycks et al., 2021) without the use of external toolkits and voting techniques.

* Core contributors.

† Work done during internship at DeepSeek-AI.

1. Introduction

Large language models (LLM) have revolutionized the approach to mathematical reasoning in artificial intelligence, spurring significant advancements in both the **quantitative reasoning benchmark** (Hendrycks et al., 2021) and the **geometry reasoning benchmark** (Trinh et al., 2024). Moreover, these models have proven instrumental in assisting humans in **solving complex mathematical problems** (Tao, 2023). However, cutting-edge models such as GPT-4 (OpenAI, 2023) and Gemini-Ultra (Anil et al., 2023) are not publicly available, and the currently accessible open-source models considerably trail behind in performance.

In this study, we introduce DeepSeekMath, a **domain-specific language model** that significantly outperforms the **mathematical capabilities** of **open-source models** and approaches the performance level of GPT-4 on academic benchmarks. To achieve this, we create the DeepSeekMath Corpus, a large-scale high-quality pre-training corpus comprising **120B math tokens**. This dataset is extracted from the **Common Crawl (CC)** using a **fastText-based classifier** (Joulin et al., 2016). In the initial iteration, the classifier is trained using instances from OpenWebMath (Paster et al., 2023) as positive examples, while incorporating a **diverse selection of other web pages** to serve as **negative examples**. Subsequently, we employ the **classifier** to mine **additional positive instances** from the CC, which are further refined through **human annotation**. The classifier is then updated with this enhanced dataset to improve its performance. The evaluation results indicate that the **large-scale corpus** is of high quality, as our base model DeepSeekMath-Base 7B achieves **64.2% on GSM8K** (Cobbe et al., 2021) and **36.2% on the competition-level MATH dataset** (Hendrycks et al., 2021), outperforming **Minerva 540B** (Lewkowycz et al., 2022a). In addition, the DeepSeekMath Corpus is **multilingual**, so we notice an improvement in **Chinese mathematical benchmarks** (Wei et al., 2023; Zhong et al., 2023). We believe that our experience in **mathematical data processing** is a starting point for the research community, and there is significant room for improvement in the future.

math train (reason cap)

DeepSeekMath-Base is initialized with **DeepSeek-Coder-Base-v1.5 7B** (Guo et al., 2024), as we notice that starting from a **code training model** is a better choice compared to a general LLM. Furthermore, we observe the **math training** also improves **model capability** on MMLU (Hendrycks et al., 2020) and BBH benchmarks (Suzgun et al., 2022), indicating it does not only enhance the **model’s mathematical abilities** but also amplifies **general reasoning capabilities**.

After pre-training, we apply **mathematical instruction tuning** to DeepSeekMath-Base with **chain-of-thought** (Wei et al., 2022), **program-of-thought** (Chen et al., 2022; Gao et al., 2023), and **tool-integrated reasoning** (Gou et al., 2023) data. The resulting model DeepSeekMath-Instruct 7B beats all 7B counterparts and is comparable with **70B open-source instruction-tuned models**.

Furthermore, we introduce the Group Relative Policy Optimization (GRPO), a variant reinforcement learning (RL) algorithm of Proximal Policy Optimization (PPO) (Schulman et al., 2017). GRPO foregoes the critic model, instead **estimating the baseline from group scores**, significantly reducing training resources. By solely using a subset of **English instruction tuning data**, GRPO obtains a substantial improvement over the strong DeepSeekMath-Instruct, including both in-domain (GSM8K: 82.9% $\rightarrow$ 88.2%, MATH: 46.8% $\rightarrow$ 51.7%) and **out-of-domain mathematical tasks** (e.g., CMATH: 84.6% $\rightarrow$ 88.8%) during the **reinforcement learning phase**. We also provide a unified paradigm to understand different methods, such as Rejection Sampling Fine-Tuning (RFT) (Yuan et al., 2023a), Direct Preference Optimization (DPO) (Rafailov et al., 2023), PPO and GRPO. Based on such a unified paradigm, we find that all these methods are conceptualized as either **direct or simplified RL techniques**. We also conduct extensive experiments, e.g., online v.s. offline training, outcome v.s. process supervision, single-turn v.s. iterative RL and so on,

to deeply investigate the essential elements of this paradigm. At last, we explain why our RL boosts the performance of instruction-tuned models, and further summarize potential directions to achieve more effective RL based on this unified paradigm.

1.1. Contributions

Our contribution includes scalable math pre-training, along with the exploration and analysis of reinforcement learning.

Math Pre-Training at Scale

- Our research provides compelling evidence that the publicly accessible Common Crawl data contains valuable information for mathematical purposes. By implementing a meticulously designed data selection pipeline, we successfully construct the DeepSeekMath Corpus, a high-quality dataset of 120B tokens from web pages filtered for mathematical content, which is almost 7 times the size of the math web pages used by Minerva (Lewkowycz et al., 2022a) and 9 times the size of the recently released OpenWebMath (Paster et al., 2023).
- Our pre-trained base model DeepSeekMath-Base 7B achieves comparable performance with Minerva 540B (Lewkowycz et al., 2022a), indicating the number of parameters is not the only key factor in mathematical reasoning capability. A smaller model pre-trained on high-quality data could achieve strong performance as well.
- We share our findings from math training experiments. Code training prior to math training improves models' ability to solve mathematical problems both with and without tool use. This offers a partial answer to the long-standing question: *does code training improve reasoning abilities?* We believe it does, at least for mathematical reasoning.
- Although training on arXiv papers is common, especially in many math-related papers, it brings no notable improvements on all mathematical benchmarks adopted in this paper.

Exploration and Analysis of Reinforcement Learning

- We introduce Group Relative Policy Optimization (GRPO), an efficient and effective reinforcement learning algorithm. GRPO foregoes the critic model, instead estimating the baseline from group scores, significantly reducing training resources compared to Proximal Policy Optimization (PPO).
- We demonstrate that GRPO significantly enhances the performance of our instruction-tuned model DeepSeekMath-Instruct, by solely using the instruction-tuning data. Furthermore, we observe enhancements in the out-of-domain performance during the reinforcement learning process.
- We provide a unified paradigm to understand different methods, such as RFT, DPO, PPO, and GRPO. We also conduct extensive experiments, e.g., online v.s. offline training, outcome v.s. process supervision, single-turn v.s. iterative reinforcement learning, and so on to deeply investigate the essential elements of this paradigm.
- Based on our unified paradigm, we explore the reasons behind the effectiveness of reinforcement learning, and summarize several potential directions to achieve more effective reinforcement learning of LLMs.

1.2. Summary of Evaluations and Metrics

- **English and Chinese Mathematical Reasoning:** We conduct comprehensive assessments of our models on English and Chinese benchmarks, covering mathematical problems

from grade-school level to college level. English benchmarks include GSM8K (Cobbe et al., 2021), MATH (Hendrycks et al., 2021), SAT (Azerbayev et al., 2023), OCW Courses (Lewkowycz et al., 2022a), MMLU-STEM (Hendrycks et al., 2020). Chinese benchmarks include MGSM-zh (Shi et al., 2023), CMATH (Wei et al., 2023), Gaokao-MathCloze (Zhong et al., 2023), and Gaokao-MathQA (Zhong et al., 2023). We evaluate models’ ability to generate self-contained text solutions without tool use, and also the ability to solve problems using Python.

On English benchmarks, DeepSeekMath-Base is competitive with the closed-source Minerva 540B (Lewkowycz et al., 2022a), and surpasses all open-source base models (e.g., Mistral 7B (Jiang et al., 2023) and Llemma-34B (Azerbayev et al., 2023)), regardless of whether they’ve undergone math pre-training or not, often by a significant margin. Notably, DeepSeekMath-Base is superior on Chinese benchmarks, likely because we don’t follow previous works (Azerbayev et al., 2023; Lewkowycz et al., 2022a) to collect English-only math pre-training data, and also include high-quality non-English ones. With mathematical instruction tuning and reinforcement learning, the resulting DeepSeekMath-Instruct and DeepSeekMath-RL demonstrate strong performance, obtaining an accuracy of over 50% on the competition-level MATH dataset for the first time within the open-source community.

- **Formal Mathematics:** We evaluate DeepSeekMath-Base using the informal-to-formal theorem proving task from (Jiang et al., 2022) on miniF2F (Zheng et al., 2021) with Isabelle (Wenzel et al., 2008) chosen to be the proof assistant. DeepSeekMath-Base demonstrates strong few-shot autoformalization performance.
- **Natural Language Understanding, Reasoning, and Code:** To build a comprehensive profile of models’ general understanding, reasoning, and coding capabilities, we evaluate DeepSeekMath-Base on the Massive Multitask Language Understanding (MMLU) benchmark (Hendrycks et al., 2020) which encompasses 57 multiple-choice tasks covering diverse subjects, BIG-Bench Hard (BBH) (Suzgun et al., 2022) which consists of 23 challenging tasks that mostly require multi-step reasoning to solve, as well as HumanEval (Chen et al., 2021) and MBPP (Austin et al., 2021) which are widely used to evaluate code language models. Math pre-training benefits both language understanding and reasoning performance.

2. Math Pre-Training

2.1. Data Collection and Decontamination

In this section, we will outline the process of constructing the DeepSeekMath Corpus from Common Crawl. As depicted in Figure 2, we present an iterative pipeline that demonstrates how to systematically gather a large-scale mathematical corpus from Common Crawl, starting with a seed corpus (e.g., a small but high-quality collection of math-related dataset). It’s worth noting that this approach is also applicable to other domains, such as coding.

First, we choose OpenWebMath (Paster et al., 2023), a collection of high-quality mathematical web texts, as our initial seed corpus. Using this corpus, we train a fastText model (Joulin et al., 2016) to recall more OpenWebMath-like mathematical web pages. Specifically, we randomly select 500,000 data points from the seed corpus as positive training examples and another 500,000 web pages from Common Crawl as negative ones. We employ an open-source library¹ for training, configuring the vector dimension to 256, learning rate to 0.1, the maximum length

¹<https://fasttext.cc>

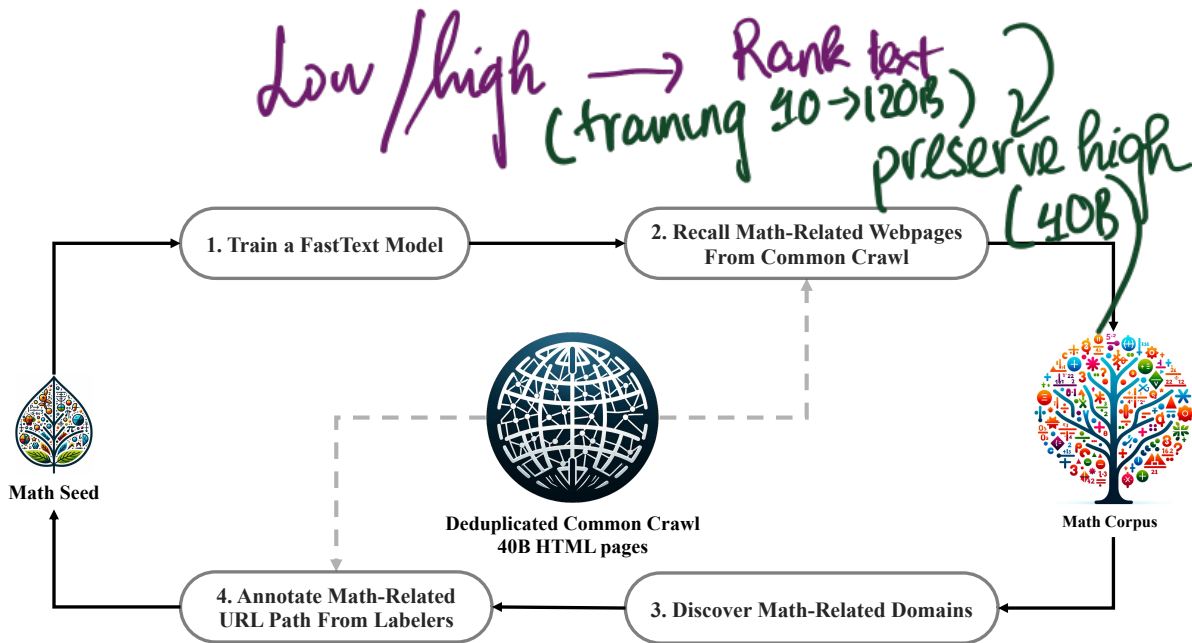


Figure 2 | An iterative pipeline that collects mathematical web pages from Common Crawl.

of word n-gram to 3, the minimum number of word occurrences to 3, and the number of training epochs to 3. To reduce the size of the original Common Crawl, we employ URL-based deduplication and near-deduplication techniques, resulting in 40B HTML web pages. We then recall mathematical web pages from deduplicated Common Crawl with the fastText model. To filter out low-quality mathematical content, we rank the collected pages according to their scores predicted by the fastText model, and only preserve the top-ranking ones. The volume of data preserved is assessed through pre-training experiments on the top 40B, 80B, 120B, and 160B tokens. In the first iteration, we choose to keep the top 40B tokens.

After the first iteration of data collection, numerous mathematical web pages remain uncollected, mainly because the fastText model is trained on a set of positive examples that lacks sufficient diversity. We therefore identify additional mathematical web sources to enrich the seed corpus, so that we can optimize the fastText model. Specifically, we first organize the entire Common Crawl into disjoint domains; a domain is defined as web pages sharing the same base URL. For each domain, we calculate the percentage of web pages that are collected in the first iteration. Domains where over 10% of the web pages have been collected are classified as math-related (e.g., mathoverflow.net). Subsequently, we manually annotate the URLs associated with mathematical content within these identified domains (e.g., mathoverflow.net/questions). Web pages linked to these URLs, yet uncollected, will be added to the seed corpus. This approach enables us to gather more positive examples, thereby training an improved fastText model capable of recalling more mathematical data in the subsequent iteration. After four iterations of data collection, we end up with 35.5M mathematical web pages, totaling 120B tokens. In the fourth iteration, we notice that nearly 98% of the data has already been collected in the third iteration, so we decide to cease data collection.

To avoid benchmark contamination, we follow Guo et al. (2024) to filter out web pages containing questions or answers from English mathematical benchmarks such as GSM8K (Cobbe et al., 2021) and MATH (Hendrycks et al., 2021) and Chinese benchmarks such as CMATH (Wei et al., 2023) and AGIEval (Zhong et al., 2023). The filtering criteria are as follows: any text segment containing a 10-gram string that matches exactly with any sub-string from the evaluation benchmarks is removed from our math training corpus. For benchmark texts that are shorter than 10 grams but have at least 3 grams, we employ exact matching to filter out contaminated web pages.

(collection)
pos exp + seed corpus → classification
filter

2.2. Validating the Quality of the DeepSeekMath Corpus

We run pre-training experiments to investigate how the DeepSeekMath Corpus is compared with the recently released math-training corpora:

- **MathPile** (Wang et al., 2023c): a multi-source corpus (8.9B tokens) aggregated from textbooks, Wikipedia, ProofWiki, CommonCrawl, StackExchange, and arXiv, with the majority (over 85%) sourced from arXiv;
- **OpenWebMath** (Paster et al., 2023): CommonCrawl data filtered for mathematical content, totaling 13.6B tokens;
- **Proof-Pile-2** (Azerbayev et al., 2023): a mathematical corpus consisting of OpenWebMath, AlgebraicStack (10.3B tokens of mathematical code), and arXiv papers (28.0B tokens). When experimenting on Proof-Pile-2, we follow Azerbayev et al. (2023) to use an arXiv:Web:Code ratio of 2:4:1.

2.2.1. Training Setting

We apply math training to a general pre-trained language model with 1.3B parameters, which shares the same framework as the DeepSeek LLMs (DeepSeek-AI, 2024), denoted as DeepSeek-LLM 1.3B. We separately train a model on each mathematical corpus for 150B tokens. All experiments are conducted using the efficient and light-weight HAI-LLM (High-flyer, 2023) training framework. Following the training practice of DeepSeek LLMs, we use the AdamW optimizer (Loshchilov and Hutter, 2017) with $\beta_1 = 0.9$, $\beta_2 = 0.95$, and weight_decay = 0.1, along with a multi-step learning rate schedule where the learning rate reaches the peak after 2,000 warmup steps, decreases to its 31.6% after 80% of the training process, and further decreases to 10.0% of the peak after 90% of the training process. We set the maximum value of learning rate to $5.3e-4$, and use a batch size of 4M tokens with a 4K context length.

Math Corpus	Size	English Benchmarks					Chinese Benchmarks		
		GSM8K	MATH	OCW	SAT	MMLU STEM	CMATH	Gaokao MathCloze	Gaokao MathQA
No Math Training	N/A	2.9%	3.0%	2.9%	15.6%	19.5%	12.3%	0.8%	17.9%
MathPile	8.9B	2.7%	3.3%	2.2%	12.5%	15.7%	1.2%	0.0%	2.8%
OpenWebMath	13.6B	11.5%	8.9%	3.7%	31.3%	29.6%	16.8%	0.0%	14.2%
Proof-Pile-2	51.9B	14.3%	11.2%	3.7%	43.8%	29.2%	19.9%	5.1%	11.7%
DeepSeekMath Corpus	120.2B	23.8%	13.6%	4.8%	56.3%	33.1%	41.5%	5.9%	23.6%

Table 1 | Performance of DeepSeek-LLM 1.3B trained on different mathematical corpora, evaluated using few-shot chain-of-thought prompting. Corpus sizes are calculated using our tokenizer with a vocabulary size of 100K.

2.2.2. Evaluation Results

The DeepSeekMath Corpus is of high quality, covers multilingual mathematical content, and is the largest in size.

- **High-quality:** We evaluate downstream performance on 8 mathematical benchmarks using few-shot chain-of-thought prompting Wei et al. (2022). As shown in Table 1, there is a clear performance lead of the model trained on the DeepSeekMath Corpus. Figure 3 shows that the model trained on the DeepSeekMath Corpus demonstrates better performance than

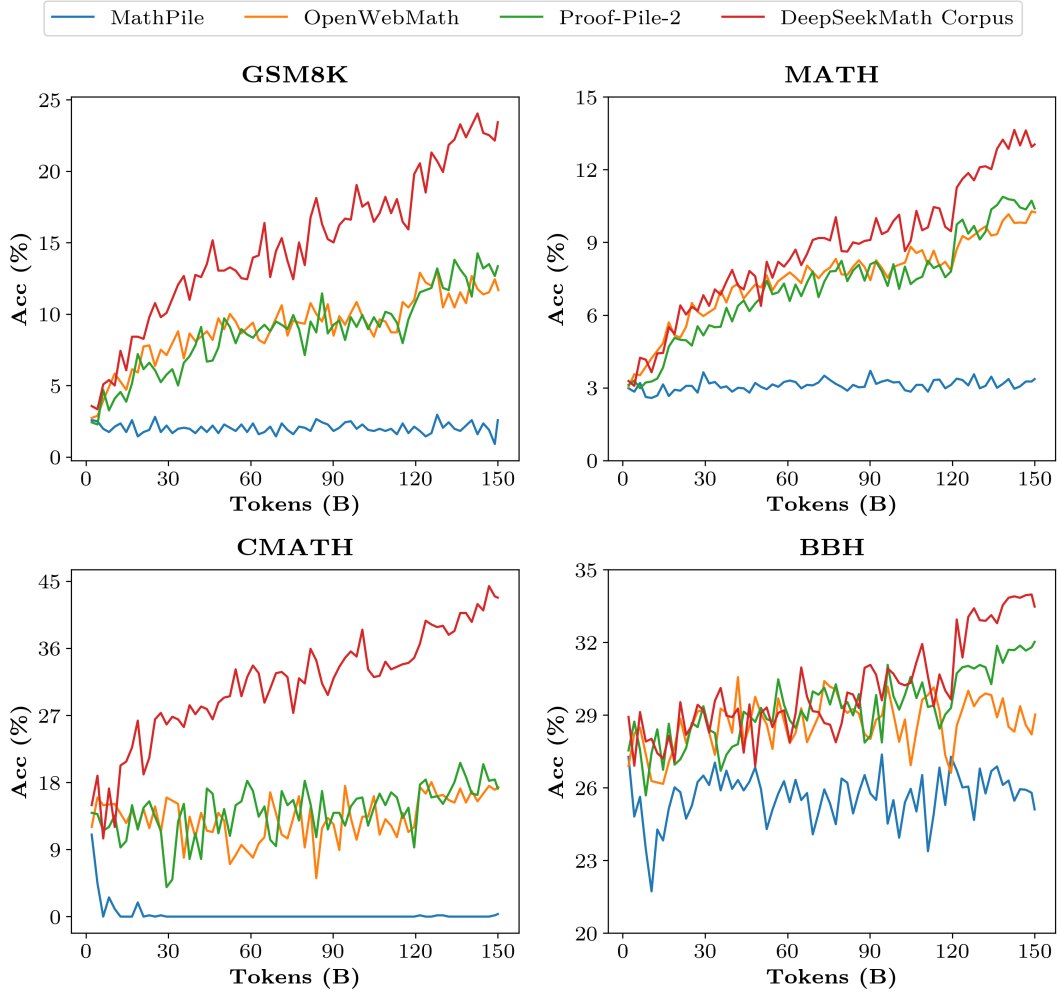


Figure 3 | Benchmark curves of DeepSeek-LLM 1.3B trained on different mathematical corpora.

Proof-Pile-2 at 50B tokens (1 full epoch of Proof-Pile-2), indicating the average quality of DeepSeekMath Corpus is higher.

- **Multilingual:** The DeepSeekMath Corpus encompasses data in multiple languages, predominantly featuring English and Chinese as the two most represented languages. As shown in Table 1, training on the DeepSeekMath Corpus enhances mathematical reasoning performance in both English and Chinese. In contrast, existing mathematical corpora, which are primarily English-centric, show limited improvement and may even hinder performance in Chinese mathematical reasoning.
- **Large-scale:** The DeepSeekMath Corpus is several times larger than existing mathematical corpora. As depicted in Figure 3, DeepSeek-LLM 1.3B, when trained on the DeepSeekMath Corpus, shows a steeper learning curve along with more lasting improvements. In contrast, the baseline corpora are much smaller, and have already been repeated multiple rounds during training, with the resulting model performance quickly reaching a plateau.

2.3. Training and Evaluating DeepSeekMath-Base 7B

In this section, we introduce DeepSeekMath-Base 7B, a base model with strong reasoning abilities, especially in mathematics. Our model is initialized with DeepSeek-Coder-Base-v1.5 7B

(Guo et al., 2024) and trained for 500B tokens. The distribution of the data is as follows: 56% is from the DeepSeekMath Corpus, 4% from AlgebraicStack, 10% from arXiv, 20% is Github code, and the remaining 10% is natural language data from Common Crawl in both English and Chinese. We mainly adopt the training setting specified in Section 2.2.1, except that we set the maximum value of the learning rate to $4.2e-4$ and use a batch size of 10M tokens.

We conduct a comprehensive assessment of the mathematical capabilities of DeepSeekMath-Base 7B, focusing on its ability to produce self-contained mathematical solutions without relying on external tools, solve mathematical problems using tools, and conduct formal theorem proving. Beyond mathematics, we also provide a more general profile of the base model, including its performance of natural language understanding, reasoning, and programming skills.

Mathematical Problem Solving with Step-by-Step Reasoning We evaluate DeepSeekMath-Base’s performance of solving mathematical problems using few-shot chain-of-thought prompting (Wei et al., 2022), across eight benchmarks in English and Chinese. These benchmarks encompass quantitative reasoning (e.g., GSM8K (Cobbe et al., 2021), MATH (Hendrycks et al., 2021), and CMATH (Wei et al., 2023)) and multiple-choice problems (e.g., MMLU-STEM (Hendrycks et al., 2020) and Gaokao-MathQA (Zhong et al., 2023)), covering diverse fields of mathematics from elementary to college-level complexity.

As shown in Table 2, DeepSeekMath-Base 7B leads in performance across all eight benchmarks among the open-source base models (including the widely-used general model Mistral 7B (Jiang et al., 2023) and the recently released Llemma 34B (Azerbayev et al., 2023) which underwent math training on Proof-Pile-2 (Azerbayev et al., 2023)). Notably, on the competition-level MATH dataset, DeepSeekMath-Base surpasses existing open-source base models by over 10% absolute, and outperforms Minerva 540B (Lewkowycz et al., 2022a), a closed-source base model 77 times larger which builds on PaLM (Lewkowycz et al., 2022b) and is further trained on mathematical texts.

Model	Size	English Benchmarks					Chinese Benchmarks		
		GSM8K	MATH	OCW	SAT	MMLU STEM	CMATH	Gaokao MathCloze	Gaokao MathQA
Closed-Source Base Model									
Minerva	7B	16.2%	14.1%	7.7%	-	35.6%	-	-	-
Minerva	62B	52.4%	27.6%	12.0%	-	53.9%	-	-	-
Minerva	540B	58.8%	33.6%	17.6%	-	63.9%	-	-	-
Open-Source Base Model									
Mistral	7B	40.3%	14.3%	9.2%	71.9%	51.1%	44.9%	5.1%	23.4%
Llemma	7B	37.4%	18.1%	6.3%	59.4%	43.1%	43.4%	11.9%	23.6%
Llemma	34B	54.0%	25.3%	10.3%	71.9%	52.9%	56.1%	11.9%	26.2%
DeepSeekMath-Base	7B	64.2%	36.2%	15.4%	84.4%	56.5%	71.7%	20.3%	35.3%

Table 2 | Comparisons between DeepSeekMath-Base 7B and strong base models on English and Chinese mathematical benchmarks. Models are evaluated with chain-of-thought prompting. Minerva results are quoted from Lewkowycz et al. (2022a).



Mathematical Problem Solving with Tool Use We evaluate program-aided mathematical reasoning on GSM8K and MATH using few-shot program-of-thought prompting (Chen et al., 2022; Gao et al., 2023). Models are prompted to solve each problem by writing a Python program where libraries such as *math* and *sympy* can be utilized for intricate computations. The execution result of the program is evaluated as the answer. As shown in Table 3, DeepSeekMath-Base 7B outperforms the prior state-of-the-art Llemma 34B.

Model	Size	Problem Solving w/ Tools		Informal-to-Formal Proving	
		GSM8K+Python	MATH+Python	miniF2F-valid	miniF2F-test
Mistral	7B	48.5%	18.2%	18.9%	18.0%
CodeLlama	7B	27.1%	17.2%	16.3%	17.6%
CodeLlama	34B	52.7%	23.5%	18.5%	18.0%
Llemma	7B	41.0%	18.6%	20.6%	22.1%
Llemma	34B	64.6%	26.3%	21.0%	21.3%
DeepSeekMath-Base	7B	66.9%	31.4%	25.8%	24.6%

Table 3 | Few-shot evaluation of base models’ ability to solve mathematical problems using tools and the ability to conduct informal-to-formal theorem proving in Isabelle.

Formal Mathematics Formal proof automation is beneficial to ensure the accuracy and reliability of mathematical proofs and enhance efficiency, with increasing attention in recent years. We evaluate DeepSeekMath-Base 7B on the task of informal-to-formal proving from (Jiang et al., 2022) which is to generate a formal proof based on an informal statement, a formal counterpart of the statement, and an informal proof. We evaluate on miniF2F (Zheng et al., 2021), a benchmark for formal Olympiad-level mathematics, and generate a formal proof in Isabelle for each problem with few-shot prompting. Following Jiang et al. (2022), we leverage models to generate proof sketches, and execute the off-the-shelf automated prover Sledgehammer (Paulson, 2010) to fill in the missing details. As shown in Table 3, DeepSeekMath-Base 7B demonstrates strong performance in proof autoformalization.

Model	Size	MMLU	BBH	HumanEval (Pass@1)	MBPP (Pass@1)
Mistral	7B	62.4%	55.7%	28.0%	41.4%
DeepSeek-Coder-Base-v1.5 [†]	7B	42.9%	42.9%	40.2%	52.6%
DeepSeek-Coder-Base-v1.5	7B	49.1%	55.2%	43.2%	60.4%
DeepSeekMath-Base	7B	54.9%	59.5%	40.9%	52.6%

Table 4 | Evaluation on natural language understanding, reasoning, and code benchmarks. DeepSeek-Coder-Base-v1.5[†] is the checkpoint right before learning rate decay, which is used to train DeepSeekMath-Base. On MMLU and BBH, we use few-shot chain-of-thought prompting. On HumanEval and MBPP, we evaluate model performance under the zero-shot setting and a few-shot setting, respectively.

long-short: compare

Natural Language Understanding, Reasoning, and Code We evaluate model performance of natural language understanding on MMLU (Hendrycks et al., 2020), reasoning on BBH (Suzgun et al., 2022), and coding capabilities on HumanEval (Chen et al., 2021) and MBPP (Austin et al.,

2021). As shown in Table 4, DeepSeekMath-Base 7B exhibits significant enhancements in performance on MMLU and BBH over its precursor, DeepSeek-Coder-Base-v1.5 (Guo et al., 2024), illustrating the positive impact of math training on language understanding and reasoning. Additionally, by including code tokens for continual training, DeepSeekMath-Base 7B effectively maintains the performance of DeepSeek-Coder-Base-v1.5 on the two coding benchmarks. Overall, DeepSeekMath-Base 7B significantly outperforms the general model Mistral 7B (Jiang et al., 2023) on the three reasoning and coding benchmarks.

3. Supervised Fine-Tuning

3.1. SFT Data Curation

We construct a mathematical instruction-tuning dataset covering English and Chinese problems from different mathematical fields and of varying complexity levels: problems are paired with solutions in chain-of-thought (CoT) (Wei et al., 2022), program-of-thought (PoT) (Chen et al., 2022; Gao et al., 2023), and tool-integrated reasoning format (Gou et al., 2023). The total number of training examples is 776K.

- **English mathematical datasets:** We annotate GSM8K and MATH problems with tool-integrated solutions, and adopt a subset of MathInstruct (Yue et al., 2023) along with the training set of Lila-OOD (Mishra et al., 2022) where problems are solved with CoT or PoT. Our English collection covers diverse fields of mathematics, e.g., algebra, probability, number theory, calculus, and geometry.
- **Chinese mathematical datasets:** We collect Chinese K-12 mathematical problems spanning 76 sub-topics such as linear equations, with solutions annotated in both CoT and tool-integrated reasoning format.

3.2. Training and Evaluating DeepSeekMath-Instruct 7B

In this section, we introduce DeepSeekMath-Instruct 7B which undergoes mathematical instruction tuning based on DeepSeekMath-Base. Training examples are randomly concatenated until reaching a maximum context length of 4K tokens. We train the model for 500 steps with a batch size of 256 and a constant learning rate of 5e-5.

We evaluate models' mathematical performance both without and with tool use, on 4 quantitative reasoning benchmarks in English and Chinese. We benchmark our model against the leading models of the time:

- **Closed-source models** include: (1) the GPT family among which GPT-4 (OpenAI, 2023) and GPT-4 Code Interpreter² are the most capable ones, (2) Gemini Ultra and Pro (Anil et al., 2023), (3) Inflection-2 (Inflection AI, 2023), (4) Grok-1³, as well as models recently released by Chinese companies including (5) Baichuan-3⁴, (6) the latest GLM-4⁵ from the GLM family (Du et al., 2022). These models are for general purposes, most of which have undergone a series of alignment procedures.
- **Open-source models** include: general models like (1) DeepSeek-LLM-Chat 67B (DeepSeek-AI, 2024), (2) Qwen 72B (Bai et al., 2023), (3) SeaLLM-v2 7B (Nguyen et al., 2023), and (4)

²<https://openai.com/blog/chatgpt-plugins#code-interpreter>

³<https://x.ai/model-card>

⁴<https://www.baichuan-ai.com>

⁵<https://open.bigmodel.cn/dev/api#glm-4>

ChatGLM3 6B (ChatGLM3 Team, 2023), as well as models with enhancements in mathematics including (5) InternLM2-Math 20B⁶ which builds on InternLM2 and underwent math training followed by **instruction tuning**, (6) Math-Shepherd-Mistral 7B which applies **PPO training** (Schulman et al., 2017) to Mistral 7B (Jiang et al., 2023) with a **process-supervised reward model**, (7) the WizardMath series (Luo et al., 2023) which improves mathematical reasoning in Mistral 7B and Llama-2 70B (Touvron et al., 2023) using evolve-instruct (i.e., a version of instruction tuning that uses AI-evolved instructions) and PPO training with training problems primarily sourced from GSM8K and MATH, (8) MetaMath 70B (Yu et al., 2023) which is Llama-2 70B fine-tuned on an augmented version of GSM8K and MATH, (9) ToRA 34B Gou et al. (2023) which is CodeLlama 34B fine-tuned to do **tool-integrated mathematical reasoning**, (10) MAMmoTH 70B (Yue et al., 2023) which is Llama-2 70B instruction-tuned on MathInstruct.

As shown in Table 5, under the evaluation setting where tool use is disallowed, DeepSeekMath-Instruct 7B demonstrates **strong performance** of **step-by-step reasoning**. Notably, on the competition-level MATH dataset, our model surpasses all open-source models and the majority of proprietary models (e.g., Inflection-2 and Gemini Pro) by at least 9% absolute. This is true even for models that are substantially larger (e.g., Qwen 72B) or have been specifically enhanced through **math-focused reinforcement learning** (e.g., WizardMath-v1.1 7B). While DeepSeekMath-Instruct rivals the Chinese proprietary models GLM-4 and Baichuan-3 on MATH, it still underperforms GPT-4 and Gemini Ultra.

Under the evaluation setting where models are allowed to integrate natural language reasoning and program-based tool use for problem solving, DeepSeekMath-Instruct 7B approaches an accuracy of **60% on MATH**, surpassing all existing open-source models. On the other benchmarks, our model is competitive with DeepSeek-LLM-Chat 67B, the prior **state-of-the-art** that is **10 times larger**.

4. Reinforcement Learning

4.1. Group Relative Policy Optimization

Reinforcement learning (RL) has been proven to be effective in further improving the mathematical reasoning ability of LLMs after the Supervised Fine-Tuning (SFT) stage (Luo et al., 2023; Wang et al., 2023b). In this section, we introduce our efficient and effective RL algorithm, Group Relative Policy Optimization (GRPO).

4.1.1. From PPO to GRPO

Proximal Policy Optimization (PPO) (Schulman et al., 2017) is an **actor-critic RL algorithm** that is widely used in the RL fine-tuning stage of LLMs (Ouyang et al., 2022). In particular, it optimizes LLMs by maximizing the following surrogate objective:

$$\mathcal{J}_{PPO}(\theta) = \mathbb{E}[q \sim P(Q), o \sim \pi_{\theta_{old}}(O|q)] \frac{1}{|O|} \sum_{t=1}^{|O|} \min \left[\frac{\pi_{\theta}(o_t|q, o_{<t})}{\pi_{\theta_{old}}(o_t|q, o_{<t})} A_t, \text{clip} \left(\frac{\pi_{\theta}(o_t|q, o_{<t})}{\pi_{\theta_{old}}(o_t|q, o_{<t})}, 1 - \epsilon, 1 + \epsilon \right) A_t \right], \quad (1)$$

where π_{θ} and $\pi_{\theta_{old}}$ are the current and old policy models, and q, o are questions and outputs sampled from the question dataset and the old policy $\pi_{\theta_{old}}$, respectively. ϵ is a clipping-related hyper-parameter introduced in PPO for stabilizing training. A_t is the advantage, which is computed by applying Generalized Advantage Estimation (GAE) (Schulman et al., 2015), based

⁶<https://github.com/InternLM/InternLM-Math>

Handwritten notes:

- training data dis** (green arrow pointing to question prompts)
- question prompts** (green arrow pointing to q in the equation)
- old policy** (green arrow pointing to $\pi_{\theta_{old}}$ in the equation)
- output (GRPO)** (green arrow pointing to o in the equation)
- all tokens** (green arrow pointing to the summation index t in the equation)
- ad (GAE)** (green arrow pointing to A_t in the equation)
- para training para** (green arrow pointing to the equation)
- policy favors token more** (green arrow pointing to the ratio in the equation)
- averaged over all tokens** (orange arrow pointing to the summation)
- (actions)** (purple arrow pointing to o in the equation)
- how much more (less) π produce of** (purple arrow pointing to the ratio in the equation)
- (rollout gen)** (purple arrow pointing to the equation)
- from the old policy (genome $\rightarrow$ several gradient updates)** (purple arrow pointing to the equation)
- same batch of data** (purple arrow pointing to the equation)

→ clip: restrict ratio $r_t(\theta)$ to range $[1-\epsilon, 1+\epsilon]$
 → min: permissive (lower) bound (small trust region)
 At < 0 (bad: decrease → $1-\epsilon$ min) → prevent excessive suppression

At 70 (good act)
 It increase to $1+\epsilon$ (max)

↓
 prevents aggressive updates

Model	Size	English Benchmarks		Chinese Benchmarks	
		GSM8K	MATH	MGSM-zh	CMATH
Chain-of-Thought Reasoning					
Closed-Source Model					
Gemini Ultra	-	94.4%	53.2%	-	-
GPT-4	-	92.0%	52.9%	-	86.0%
Inflection-2	-	81.4%	34.8%	-	-
GPT-3.5	-	80.8%	34.1%	-	73.8%
Gemini Pro	-	86.5%	32.6%	-	-
Grok-1	-	62.9%	23.9%	-	-
Baichuan-3	-	88.2%	49.2%	-	-
GLM-4	-	87.6%	47.9%	-	-
Open-Source Model					
InternLM2-Math	20B	82.6%	37.7%	-	-
Qwen	72B	78.9%	35.2%	-	-
Math-Shepherd-Mistral	7B	84.1%	33.0%	-	-
WizardMath-v1.1	7B	83.2%	33.0%	-	-
DeepSeek-LLM-Chat	67B	84.1%	32.6%	74.0%	80.3%
MetaMath	70B	82.3%	26.6%	66.4%	70.9%
SeaLLM-v2	7B	78.2%	27.5%	64.8%	-
ChatGLM3	6B	72.3%	25.7%	-	-
WizardMath-v1.0	70B	81.6%	22.7%	64.8%	65.4%
DeepSeekMath-Instruct	7B	82.9%	46.8%	73.2%	84.6%
DeepSeekMath-RL	7B	88.2%	51.7%	79.6%	88.8%
Tool-Integrated Reasoning					
Closed-Source Model					
GPT-4 Code Interpreter	-	97.0%	69.7%	-	-
Open-Source Model					
InternLM2-Math	20B	80.7%	54.3%	-	-
DeepSeek-LLM-Chat	67B	86.7%	51.1%	76.4%	85.4%
ToRA	34B	80.7%	50.8%	41.2%	53.4%
MAmmoTH	70B	76.9%	41.8%	-	-
DeepSeekMath-Instruct	7B	83.7%	57.4%	72.0%	84.3%
DeepSeekMath-RL	7B	86.7%	58.8%	78.4%	87.6%

Table 5 | Performance of Open- and Closed-Source models with both Chain-of-Thought and Tool-Integrated Reasoning on English and Chinese Benchmarks. Scores in gray denote majority votes with 32 candidates; The others are Top1 scores. DeepSeekMath-RL 7B beats all open-source models from 7B to 70B, as well as the majority of closed-source models. Although DeepSeekMath-RL 7B is only further trained on chain-of-thought-format instruction tuning data of GSM8K and MATH, it improves over DeepSeekMath-Instruct 7B on all benchmarks.

token: text minimal unit model reads &
 (not necessarily correspond to processes
 12
 complete word)

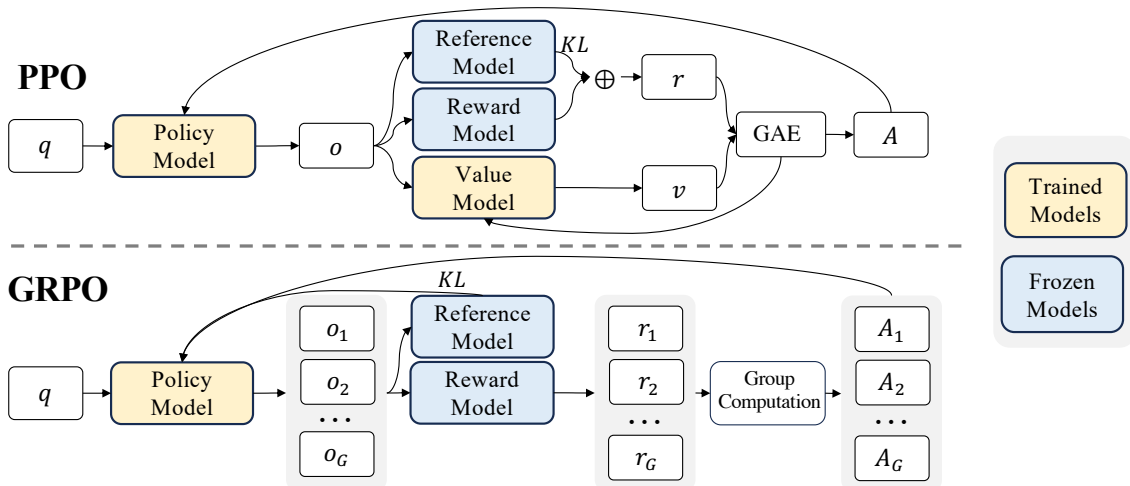


Figure 4 | Demonstration of PPO and our GRPO. GRPO foregoes the value model, instead estimating the baseline from group scores, significantly reducing training resources.

on the rewards $\{r_{\geq t}\}$ and a learned value function V_{ψ} . Thus, in PPO, a value function needs to be trained alongside the policy model and to mitigate over-optimization of the reward model, the standard approach is to add a per-token KL penalty from a reference model in the reward at each token (Ouyang et al., 2022), i.e.,

$$r_t = r_{\phi}(q, o_{\leq t}) - \beta \log \frac{\pi_{\theta}(o_t|q, o_{<t})}{\pi_{ref}(o_t|q, o_{<t})} \quad (2)$$

Handwritten notes:
 Scores (quality) → current policy
 coefficient (weighted pen)
 fixed ref supplied-tune

where r_{ϕ} is the reward model, π_{ref} is the reference model, which is usually the initial SFT model, and β is the coefficient of the KL penalty.

As the value function employed in PPO is typically another model of comparable size as the policy model, it brings a substantial memory and computational burden. Additionally, during RL training, the value function is treated as a baseline in the calculation of the advantage for variance reduction. While in the LLM context, usually only the last token is assigned a reward score by the reward model, which may complicate the training of a value function that is accurate at each token. To address this, as shown in Figure 4, we propose Group Relative Policy Optimization (GRPO), which obviates the need for additional value function approximation as in PPO, and instead uses the average reward of multiple sampled outputs, produced in response to the same question, as the baseline. More specifically, for each question q , GRPO samples a group of outputs $\{o_1, o_2, \dots, o_G\}$ from the old policy $\pi_{\theta_{old}}$ and then optimizes the policy model by maximizing the following objective:

$$\mathcal{J}_{GRPO}(\theta) = \mathbb{E}[q \sim P(Q), \{o_i\}_{i=1}^G \sim \pi_{\theta_{old}}(O|q)]$$

$$\frac{1}{G} \sum_{i=1}^G \frac{1}{|o_i|} \sum_{t=1}^{|o_i|} \left\{ \min \left[\frac{\pi_{\theta}(o_{i,t}|q, o_{i,<t})}{\pi_{\theta_{old}}(o_{i,t}|q, o_{i,<t})}, \hat{A}_{i,t}, \text{clip} \left(\frac{\pi_{\theta}(o_{i,t}|q, o_{i,<t})}{\pi_{\theta_{old}}(o_{i,t}|q, o_{i,<t})}, 1 - \epsilon, 1 + \epsilon \right) \hat{A}_{i,t} \right] - \beta \mathbb{D}_{KL}[\pi_{\theta} || \pi_{ref}] \right\}, \quad (3)$$

where ϵ and β are hyper-parameters, and $\hat{A}_{i,t}$ is the advantage calculated based on relative rewards of the outputs inside each group only, which will be detailed in the following subsections. The group relative way that GRPO leverages to calculate the advantages, aligns well with the comparative nature of rewards models, as reward models are typically trained on datasets of comparisons between outputs on the same question. Also note that, instead of adding KL penalty in the reward, GRPO regularizes by directly adding the KL divergence between the trained policy and the reference policy to the loss, avoiding complicating the calculation of $\hat{A}_{i,t}$.

Prob. value model
← policy model

GRPO

not

need critic (value) func

group sampling
group relative ad $\hat{A}_{i,t}$

Algorithm 1 Iterative Group Relative Policy Optimization

Input initial policy model $\pi_{\theta_{\text{init}}}$; reward models r_ϕ ; task prompts $\mathcal{D}$; hyperparameters ε, β, μ

- 1: policy model $\pi_\theta \leftarrow \pi_{\theta_{\text{init}}}$
- 2: **for** iteration = 1, ..., I **do**
- 3: reference model $\pi_{\text{ref}} \leftarrow \pi_\theta$
- 4: **for** step = 1, ..., M **do**
- 5: Sample a batch $\mathcal{D}_b$ from $\mathcal{D}$
- 6: Update the old policy model $\pi_{\theta_{\text{old}}} \leftarrow \pi_\theta$
- 7: Sample G outputs $\{o_i\}_{i=1}^G \sim \pi_{\theta_{\text{old}}}(\cdot | q)$ for each question $q \in \mathcal{D}_b$
- 8: Compute rewards $\{r_i\}_{i=1}^G$ for each sampled output o_i by running r_ϕ
- 9: Compute $\hat{A}_{i,t}$ for the t -th token of o_i through group relative advantage estimation.
- 10: **for** GRPO iteration = 1, ..., μ **do**
- 11: Update the policy model π_θ by maximizing the GRPO objective (Equation 21)
- 12: Update r_ϕ through continuous training using a replay mechanism.

Output π_θ

And different from the KL penalty term used in (2), we estimate the KL divergence with the following unbiased estimator (Schulman, 2020):

$$\mathbb{D}_{\text{KL}}[\pi_\theta || \pi_{\text{ref}}] = \frac{\pi_{\text{ref}}(o_{i,t} | q, o_{i,<t})}{\pi_\theta(o_{i,t} | q, o_{i,<t})} - \log \frac{\pi_{\text{ref}}(o_{i,t} | q, o_{i,<t})}{\pi_\theta(o_{i,t} | q, o_{i,<t})} - 1, \quad (4)$$

which is guaranteed to be positive.

4.1.2. Outcome Supervision RL with GRPO

Formally, for each question q , a group of outputs $\{o_1, o_2, \dots, o_G\}$ are sampled from the old policy model $\pi_{\theta_{\text{old}}}$. A reward model is then used to score the outputs, yielding G rewards $\mathbf{r} = \{r_1, r_2, \dots, r_G\}$ correspondingly. Subsequently, these rewards are normalized by subtracting the group average and dividing by the group standard deviation. Outcome supervision provides the normalized reward at the end of each output o_i and sets the advantages $\hat{A}_{i,t}$ of all tokens in the output as the normalized reward, i.e., $\hat{A}_{i,t} = \tilde{r}_i = \frac{r_i - \text{mean}(\mathbf{r})}{\text{std}(\mathbf{r})}$, and then optimizes the policy by maximizing the objective defined in equation (3).

4.1.3. Process Supervision RL with GRPO

Outcome supervision only provides a reward at the end of each output, which may not be sufficient and efficient to supervise the policy in complex mathematical tasks. Following Wang et al. (2023b), we also explore process supervision, which provides a reward at the end of each reasoning step. Formally, given the question q and G sampled outputs $\{o_1, o_2, \dots, o_G\}$, a process reward model is used to score each step of the outputs, yielding corresponding rewards: $\mathbf{R} = \{\{r_1^{\text{index}(1)}, \dots, r_1^{\text{index}(K_1)}\}, \dots, \{r_G^{\text{index}(1)}, \dots, r_G^{\text{index}(K_G)}\}\}$, where $\text{index}(j)$ is the end token index of the j -th step, and K_i is the total number of steps in the i -th output. We also normalize these rewards with the average and the standard deviation, i.e., $\tilde{r}_i^{\text{index}(j)} = \frac{r_i^{\text{index}(j)} - \text{mean}(\mathbf{R})}{\text{std}(\mathbf{R})}$. Subsequently, the process supervision calculates the advantage of each token as the sum of the normalized rewards from the following steps, i.e., $\hat{A}_{i,t} = \sum_{\text{index}(j) \geq t} \tilde{r}_i^{\text{index}(j)}$, and then optimizes the policy by maximizing the objective defined in equation (3).

Reward normalization (every step)

step (t) relating to good results
rewards of steps will back propagate to prior

KL divergence (relative entropy) non negative
 (Kullback - Leibler)
 1 prob Q differs from ref distribution P
 (coding cost) (encode data)

$$D_{KL}(P||Q) = \sum_x P(x) \log \frac{P(x)}{Q(x)}$$

↳ Discrete

$$D_{KL}(P||Q) = \int p(x) \log \frac{p(x)}{q(x)} dx$$

↳ Continuous ≥ 0
 $\Rightarrow$ Prob (differential ratio between 2 prob distributions)

$\rightarrow$ Sample $< 0 \Rightarrow \Pi_0$ generates frequently samples with contribution

$D_{KL} \geq 0$
 Native log-ratio: $D_{KL}(\Pi_0 || \Pi_{ref}) = E_{x \sim \Pi_0} \left[\log \frac{\Pi_0(x)}{\Pi_{ref}(x)} \right]$

$$k_t = \log \frac{\pi_0(O_t)}{\pi_{\text{ref}}(O_t)} \rightarrow \text{could be negative for individual sample}$$

p $\swarrow$ noisy & high-variance (training signals)
 $\downarrow$ randomly rewards policy (sampling luck)

$\rightarrow$ Formula (improved):

$$D_{KL} = p - \log p - 1 \geq 0 \quad (p \geq 0)$$

⊕ Always positive

$$f(p) = p - \log p - 1$$

$$p = \frac{\pi_{\text{ref}}}{\pi_0}$$

$$\Rightarrow f'(p) = 1 - \frac{1}{p} \Rightarrow f'(p) = 0 \Rightarrow p = 1.$$

⊕ Unbiased: $E_{x \sim \pi_0} [p - \log p - 1]$

$$= E[p] - E[\log p] - 1 \quad (1)$$

$$\Rightarrow E[p] = E_{x \sim \pi_0} [p] = \sum_x \pi_0(x) \frac{\pi_{\text{ref}}(x)}{\pi_0(x)}$$

$$= \sum_x \pi_{\text{ref}}(x) = 1 \quad (\text{Sum of one prob})$$

$$(1) \Rightarrow E x \sim \pi_0 [p - \log p - 1]$$

$$= 1 - E[\log p] - 1$$

$$= E[-\log p] = E\left[\log\left(\frac{\pi_0}{\pi_{\text{ref}}}\right)\right]$$

$$= E[\log p^{-1}]$$

$$\left(p(x) = \frac{\pi_{\text{ref}}(x)}{\pi_0(x)} \right)$$

⊕ Lower variance: $E[p - 1] = 0 \Rightarrow$ control variate

Estimate $X \Rightarrow E[X + C]$ to reduce variance
 $(E[C] = 0)$

(Negative correlation $\rightarrow p \uparrow \Rightarrow \log p \downarrow$)

Idea: expect

= 0 change variance

(not change expect)

the balance between p & $\log p$

$\Downarrow$

estimator with stable behavior

indiv > 0
 mean unchange

4.1.4. Iterative RL with GRPO

As the reinforcement learning training process progresses, the old reward model may not be sufficient to supervise the current policy model. Therefore, we also explore the iterative RL with GRPO. As shown in Algorithm 1, in iterative GRPO, we generate new training sets for the reward model based on the sampling results from the policy model and continually train the old reward model using a replay mechanism that incorporates 10% of historical data. Then, we set the reference model as the policy model, and continually train the policy model with the new reward model.

4.2. Training and Evaluating DeepSeekMath-RL

We conduct RL based on DeepSeekMath-Instruct 7B. The training data of RL are chain-of-thought-format questions related to GSM8K and MATH from the SFT data, which consists of around 144K questions. We exclude other SFT questions to investigate the impact of RL on benchmarks that lack data throughout the RL phase. We construct the training set of reward models following (Wang et al., 2023b). We train our initial reward model based on the DeepSeekMath-Base 7B with a learning rate of $2e-5$. For GRPO, we set the learning rate of the policy model as $1e-6$. The KL coefficient is 0.04. For each question, we sample 64 outputs. The max length is set to 1024, and the training batch size is 1024. The policy model only has a single update following each exploration stage. We evaluate DeepSeekMath-RL 7B on benchmarks following DeepSeekMath-Instruct 7B. For DeepSeekMath-RL 7B, GSM8K and MATH with chain-of-thought reasoning can be regarded as in-domain tasks and all the other benchmarks can be regarded as out-of-domain tasks.

Table 5 demonstrates the performance of open- and closed-source models with both chain-of-thought and tool-integrated reasoning on English and Chinese benchmarks. We find that: 1) DeepSeekMath-RL 7B attains accuracies of 88.2% and 51.7% on GSM8K and MATH, respectively, utilizing chain-of-thought reasoning. This performance surpasses that of all open-source models in the 7B to 70B range, as well as the majority of closed-source models. 2) Crucially, DeepSeekMath-RL 7B is only trained on chain-of-thought-format instruction tuning data of GSM8K and MATH, starting from DeepSeekMath-Instruct 7B. Despite the constrained scope of its training data, it outperforms DeepSeekMath-Instruct 7B across all evaluation metrics, showcasing the effectiveness of reinforcement learning.

5. Discussion

In this section, we will share our findings in pre-training and RL experiments.

5.1. Lessons Learnt in Pre-Training

We first share our experience in pre-training. Unless otherwise specified, we will adhere to the training settings outlined in Section 2.2.1. It is worth noting that, when referring to the DeepSeekMath Corpus in this section, we use an 89B-token dataset from the second iteration of the data collection process.

5.1.1. Code Training Benefits Mathematical Reasoning

A popular yet unverified hypothesis suggests that code training improves reasoning. We attempt to offer a partial response to this, particularly within the mathematical domain: code training

Training Setting	Training Tokens			w/o Tool Use			w/ Tool Use	
	General	Code	Math	GSM8K	MATH	CMATH	GSM8K+Python	MATH+Python
No Continual Training	–	–	–	2.9%	3.0%	12.3%	2.7%	2.3%
Two-Stage Training								
Stage 1: General Training	400B	–	–	2.9%	3.2%	14.8%	3.3%	2.3%
Stage 2: Math Training	–	–	150B	19.1%	14.4%	37.2%	14.3%	6.7%
Stage 1: Code Training	–	400B	–	5.9%	3.6%	19.9%	12.4%	10.0%
Stage 2: Math Training	–	–	150B	21.9%	15.3%	39.7%	17.4%	9.4%
One-Stage Training								
Math Training	–	–	150B	20.5%	13.1%	37.6%	11.4%	6.5%
Code & Math Mixed Training	–	400B	150B	17.6%	12.1%	36.3%	19.7%	13.5%

Table 6 | Investigation of how code affects mathematical reasoning under different training settings. We experiment with DeepSeek-LLM 1.3B, and evaluate its mathematical reasoning performance without and with tool use via few-shot chain-of-thought prompting and few-shot program-of-thought prompting, respectively.

improves models’ ability to do mathematical reasoning both with and without tool use.

To study how code training affects mathematical reasoning, we experimented with the following two-stage training and one-stage training settings:

Two-Stage Training

- **Code Training for 400B Tokens → Math Training for 150B Tokens:** We train DeepSeek-LLM 1.3B for 400B code tokens followed by 150B math tokens;
- **General Training for 400B Tokens → Math Training for 150B Tokens:** As a control experiment, we also experiment with general tokens (sampled from a large-scale general corpus created by DeepSeek-AI) instead of code tokens in the first stage of training, in an attempt to investigate the advantages of code tokens over general tokens in improving mathematical reasoning.

One-Stage Training

- **Math Training for 150B Tokens:** We train DeepSeek-LLM 1.3B for 150B math tokens;
- **Training on a mixture of 400B Code Tokens and 150B Math Tokens:** Math training following **code training degrades coding performance**. We investigate whether code tokens, when mixed with math tokens for **one-stage training**, would still improve **mathematical reasoning** and also alleviate the problem of **catastrophic forgetting**.

Results Table 6 and Table 7 demonstrate the downstream performance under different training settings.

Code training benefits program-aided mathematical reasoning, both under the **two-stage training** and **one-stage training** settings. As shown in Table 6, under the two-stage training setting, code training alone already significantly enhances the ability to solve **GSM8K** and **MATH problems** using **Python**. Math training in the second stage yields further improvements. Interestingly, under the one-stage training setting, **mixing code tokens and math tokens** effectively mitigates the issue of **catastrophic forgetting** that arises from **two-stage training**, and also synergizes coding (Table 7) and program-aided mathematical reasoning (Table 6).

Training Setting	Training Tokens			MMLU	BBH	HumanEval (Pass@1)	MBPP (Pass@1)
	General	Code	Math				
No Continual Training	–	–	–	24.5%	28.1%	12.2%	13.0%
Two-Stage Training							
Stage 1: General Training	400B	–	–	25.9%	27.7%	15.2%	13.6%
Stage 2: Math Training	–	–	150B	33.1%	32.7%	12.8%	13.2%
Stage 1: Code Training	–	400B	–	25.0%	31.5%	25.0%	40.0%
Stage 2: Math Training	–	–	150B	36.2%	35.3%	12.2%	17.0%
One-Stage Training							
Math Training	–	–	150B	32.3%	32.5%	11.6%	13.2%
Code & Math Mixed Training	–	400B	150B	33.5%	35.6%	29.3%	39.4%

Table 7 | Investigation of how different settings of code and math training affect model performance of language understanding, reasoning, and coding. We experiment with DeepSeek-LLM 1.3B. We evaluate the models on MMLU and BBH using few-shot chain-of-thought prompting. On HumanEval and MBPP, we conduct zero-shot and few-shot evaluations, respectively.

Model	Size	ArXiv Corpus	English Benchmarks					Chinese Benchmarks		
			GSM8K	MATH	OCW	SAT	MMLU STEM	CMATH	Gaokao MathCloze	Gaokao MathQA
DeepSeek-LLM	1.3B	No Math Training	2.9%	3.0%	2.9%	15.6%	19.5%	12.3%	0.8%	17.9%
		MathPile	2.7%	3.3%	2.2%	12.5%	15.7%	1.2%	0.0%	2.8%
		ArXiv-RedPajama	3.3%	3.4%	4.0%	9.4%	9.0%	7.4%	0.8%	2.3%
DeepSeek-Coder-Base-v1.5	7B	No Math Training	29.0%	12.5%	6.6%	40.6%	38.1%	45.9%	5.9%	21.1%
		MathPile	23.6%	11.5%	7.0%	46.9%	35.8%	37.9%	4.2%	25.6%
		ArXiv-RedPajama	28.1%	11.1%	7.7%	50.0%	35.2%	42.6%	7.6%	24.8%

Table 8 | Effect of math training on different arXiv datasets. Model performance is evaluated with few-shot chain-of-thought prompting.

ArXiv Corpus	miniF2F-valid	miniF2F-test
No Math Training	20.1%	21.7%
MathPile	16.8%	16.4%
ArXiv-RedPajama	14.8%	11.9%

Table 9 | Effect of math training on different arXiv corpora, the base model being DeepSeek-Coder-Base-v1.5 7B. We evaluate informal-to-formal proving in Isabelle.

Code training also improves mathematical reasoning without tool use. Under the two-stage training setting, the initial stage of code training already results in moderate enhancements. It also boosts the efficiency of the subsequent math training, eventually leading to the best performance. However, combining code tokens and math tokens for one-stage training compromises mathematical reasoning without tool use. One conjecture is that DeepSeek-LLM 1.3B, due to its limited scale, lacks the capacity to fully assimilate both code and mathematical data simultaneously.

5.1.2. ArXiv Papers Seem Ineffective in Improving Mathematical Reasoning

ArXiv papers are commonly included as a component of math pre-training data (Azerbayev et al., 2023; Lewkowycz et al., 2022a; Polu and Sutskever, 2020; Wang et al., 2023c). However,

detailed analysis regarding their impact on **mathematical reasoning** has not been extensively conducted. Perhaps counter-intuitively, according to our experiments, arXiv papers seem ineffective in improving mathematical reasoning. We experiment with models of different sizes, including DeepSeek-LLM 1.3B and DeepSeek-Coder-Base-v1.5 7B (Guo et al., 2024), using arXiv corpora that underwent varied processing pipelines:

- **MathPile** (Wang et al., 2023c): an 8.9B-token corpus developed with **cleaning** and **filtering heuristic rules**, over 85% of which are scientific arXiv papers;
- **ArXiv-RedPajama** (Computer, 2023): the entirety of arXiv LaTeX files with **preambles**, **comments**, **macros**, and **bibliographies** removed, totaling 28.0B tokens.

In our experiments, we separately train DeepSeek-LLM 1.3B for 150B tokens and DeepSeek-Coder-Base-v1.5 7B for 40B tokens on each arXiv corpus. It seems that arXiv papers are ineffective in **improving mathematical reasoning**. When trained on a arXiv-only corpus, both models display **no notable improvements** or even **deterioration** across various **mathematical benchmarks** of different **complexities** employed in this study. These benchmarks include **quantitative reasoning** datasets like **GSM8K** and **MATH** (Table 8), multiple-choice challenges like **MMLU-STEM** (Table 8), and formal mathematics like **miniF2F** (Table 9).

However, this conclusion has its limitations and should be taken with a grain of salt. We have not yet studied:

- The impact of arXiv tokens on specific **math-related tasks** not included in this research, such as **informalization** of **theorems** which is to **convert formal statements** or **proofs** to their **informal versions**;
- The effect of arXiv tokens when combined with other **types of data**;
- Whether the benefits of arXiv papers would manifest themselves at a **larger model scale**.

Thus, further exploration is required, which we leave for future studies.

5.2. Insights of Reinforcement Learning

5.2.1. Towards to a Unified Paradigm

In this section, we provide a unified paradigm to analyze **different training methods**, such as **SFT**, **RFT**, **DPO**, **PPO**, **GRPO**, and further conduct experiments to explore the factors of the unified paradigm. Generally, the gradient with respect to the **parameter θ** of a **training method** can be written as:

$$\nabla_{\theta} \mathcal{J}_{\mathcal{A}}(\theta) = \mathbb{E}[\underbrace{(q, o) \sim \mathcal{D}}_{\text{Data Source}}] \left(\underbrace{\frac{1}{|\mathcal{O}|} \sum_{t=1}^{|\mathcal{O}|}}_{\text{per token average}} \underbrace{GC_{\mathcal{A}}(q, o, t, \pi_{rf})}_{\text{Gradient Coefficient}} \underbrace{\nabla_{\theta} \log \pi_{\theta}(o_t | q, o_{<t})}_{\text{prob gradient (score func)}} \right) \quad (5)$$

Handwritten notes: "how to change para θ to have better output" points to the gradient. "algo A" points to the expectation operator. "per token average" points to the summation. "prob gradient (score func)" points to the log probability. "scalar weight" points to the gradient coefficient.

There exist three key components: 1) **Data Source $\mathcal{D}$** , which determines the **training data**; 2) **Reward Function π_{rf}** , which is the source of the **training reward signal**; 3) **Algorithm $\mathcal{A}$** : which processes the **training data** and the **reward signal** to the **gradient coefficient GC** that determines the **magnitude** of the **penalty** or **reinforcement** for the **data**. We analyze several representative methods based on such a unified paradigm:

- **Supervised Fine-tuning (SFT)**: SFT fine-tunes **pretrained model** on human selected SFT data.

Handwritten notes: "how much to push log-prob up/down" points to the log probability. "Gradient = Advantage / Reward Sig" points to the gradient coefficient. "Prob change direction" points to the log probability.

Methods	Data Source	Reward Function	Gradient Coefficient
SFT	$q, o \sim P_{sft}(Q, O)$	-	1
RFT	$q \sim P_{sft}(Q), o \sim \pi_{sft}(O q)$	Rule	Equation 10
DPO	$q \sim P_{sft}(Q), o^+, o^- \sim \pi_{sft}(O q)$	Rule	Equation 14
Online RFT	$q \sim P_{sft}(Q), o \sim \pi_{\theta}(O q)$	Rule	Equation 10
PPO	$q \sim P_{sft}(Q), o \sim \pi_{\theta}(O q)$	Model	Equation 18
GRPO	$q \sim P_{sft}(Q), \{o_i\}_{i=1}^G \sim \pi_{\theta}(O q)$	Model	Equation 21

Table 10 | The data source and gradient coefficient of different methods. P_{sft} denotes the data distribution of supervised fine-tuning datasets. $\pi_{\theta_{sft}}$ and π_{θ} denote the supervised fine-tuned model and the real-time policy model during the online training process, respectively.

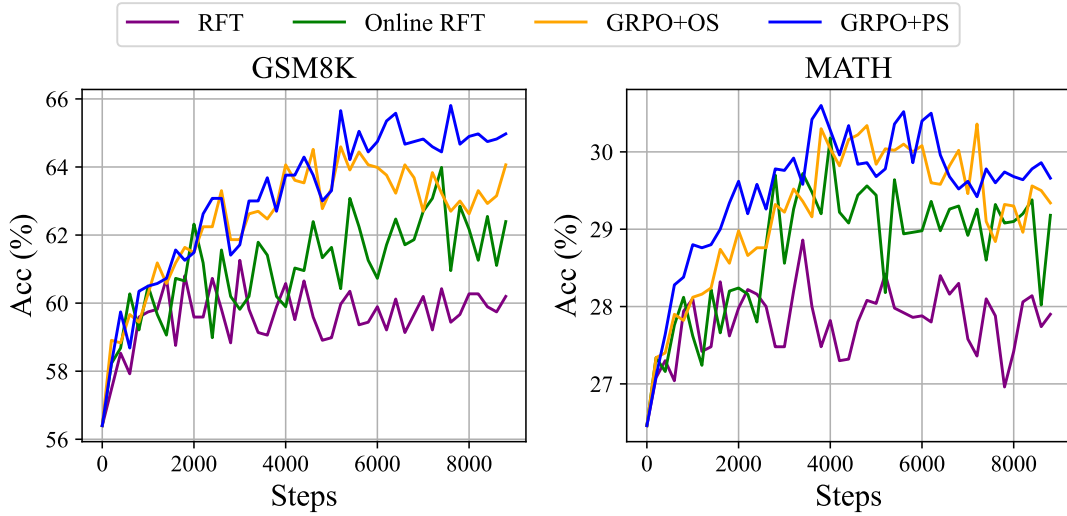


Figure 5 | Performance of the DeepSeekMath-Instruct 1.3B model, which was further trained using various methods, on two benchmarks.

- **Rejection Sampling Fine-tuning (RFT):** RFT further fine-tunes the SFT model on the filtered outputs sampled from the SFT model based on SFT questions. RFT filters the outputs based on the correctness of their answers.
- **Direct Preference Optimization (DPO):** DPO further refines the SFT model by fine-tuning it on augmented outputs sampled from the SFT model, using pair-wise DPO loss.
- **Online Rejection Sampling Fine-tuning (Online RFT):** Different from RFT, Online RFT initiates the policy model using the SFT model and refines it by fine-tuning with the augmented outputs sampled from the real-time policy model.
- **PPO/GRPO:** PPO/GRPO initializes the policy model using the SFT model and reinforces it with the outputs sampled from the real-time policy model.

We summarize the components of these methods in Table 10. Please refer to Appendix A.1 for a more detailed derivation process.

Observation about Data Source We divide the data source into two categories, online sampling, and offline sampling. Online sampling denotes that the training data is from the exploration results of the real-time training policy model, while offline sampling denotes that the

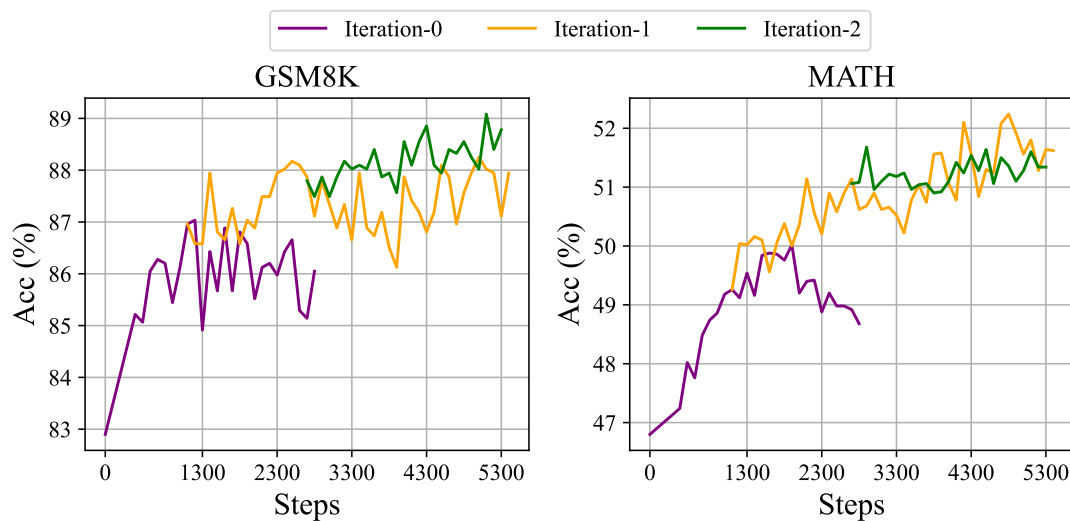


Figure 6 | Performance of iterative reinforcement learning with DeepSeekMath-Instruct 7B on two benchmarks.

training data is from the **sampling results** of the initial SFT model. RFT and DPO follow the **offline style**, while Online RFT and GRPO follow the **online style**.

As shown in Figure 5, we find that the Online RFT significantly **outperforms RFT** on two **benchmarks**. Specifically, Online RFT is **comparable** to RFT in the **early stage of training** but gains an **absolute advantage** in the **later stage**, demonstrating the **superiority** of **online training**. This is intuitive, as in the initial stage, the **actor** and the **SFT model** exhibit close **resemblance**, with the **sampled data** revealing only **minor differences**. In the **later stage**, however, the data sampled from the **actor** will exhibit more **significant differences**, and **real-time data sampling** will offer greater advantages.

Observation about Gradient Coefficient The algorithm processes the **reward signal** to the **gradient coefficient** to update the **model parameter**. We divide the **reward function** as 'Rule' and 'Model' in our experiments. Rule refers to **judging** the **quality** of a **response** based on the **correctness** of the **answer**, and Model denotes that we train a **reward model** to score each response. The training data of the **reward model** is based on the **rule judgment**. Equations 10 and 21 highlight a key difference between **GRPO** and **Online RFT**: GRPO uniquely adjusts its **gradient coefficient** based on the **reward value** provided by the **reward model**. This allows for **differential reinforcement** and **penalization** of **responses** according to their varying **magnitudes**. In contrast, **Online RFT** lacks this feature; it does **not penalize incorrect responses** and **uniformly reinforces** all responses with **correct answers** at the same **level of intensity**.

As demonstrated in Figure 5, GRPO surpasses online RFT, thereby highlighting the efficiency of **altering positive** and **negative gradient coefficients**. In addition, GRPO+PS shows **superior performance** compared to GRPO+OS, indicating the benefits of using **fine-grained, step-aware** gradient coefficients. Furthermore, we explore the **iterative RL**, in our experiments, we conduct two **rounds of iteration**. As shown in Figure 6, we notice that the iterative RL significantly improves the performance, especially at the first iteration.

rule : answer correctness
 model : reward response score

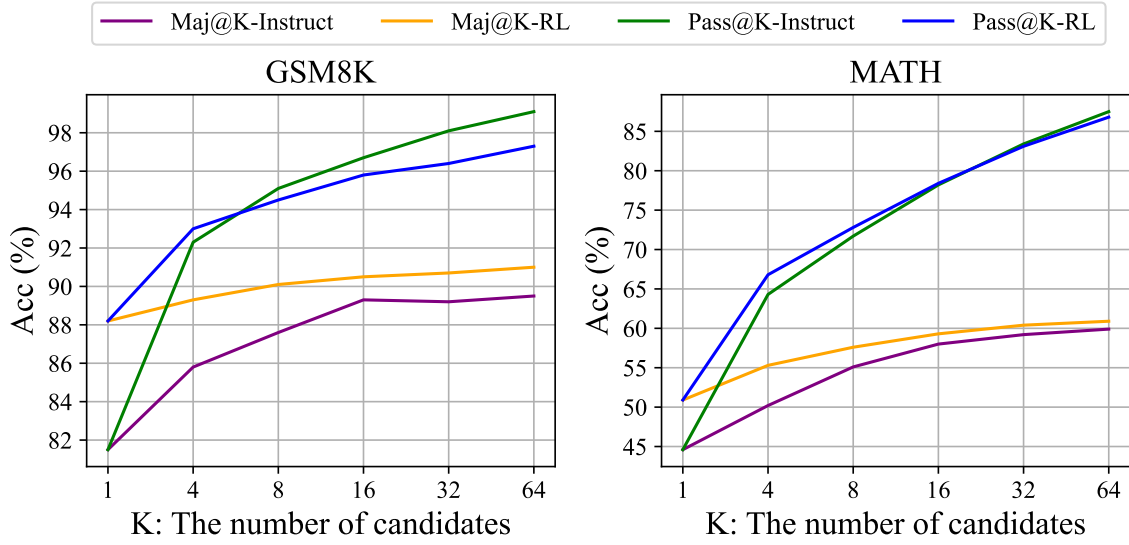


Figure 7 | The Maj@K and Pass@K of SFT and RL DeepSeekMath 7B on GSM8K and MATH (temperature 0.7). It was noted that RL enhances Maj@K but not Pass@K.

5.2.2. Why RL Works?

In this paper, we conduct reinforcement learning based on a subset of instruction tuning data, and it achieves significant performance enhancement upon the instruction tuning model. To further explain why reinforcement learning works. We evaluate the Pass@K and Maj@K accuracy of the Instruct and RL models on two benchmarks. As shown in Figure 7, RL enhances Maj@K's performance but not Pass@K. These findings indicate that RL enhances the model's overall performance by rendering the output distribution more robust, in other words, **it seems that the improvement is attributed to boosting the correct response from TopK rather than the enhancement of fundamental capabilities.** Similarly, (Wang et al., 2023a) identified a misalignment problem in reasoning tasks within the SFT model, showing that the reasoning performance of SFT models can be improved through a series of preference alignment strategies (Song et al., 2023; Wang et al., 2023a; Yuan et al., 2023b).

5.2.3. How to Achieve More Effective RL?

We demonstrate RL works pretty well in mathematical reasoning tasks. We also provide a unified paradigm to understand different representative training methods. Within this paradigm, all methods are conceptualized as either direct or simplified RL techniques. As summarized in Equation 5, there exist three key components: Data Source, Algorithm, and Reward Function. We provide some potential future directions about the three components.

Data Source Data source is the raw material of all training methods. In the context of RL, we specifically refer to the data source as the unlabeled questions with the outputs sampled from the policy model. In this paper, we only use the questions from the instruction tuning stage and a naive nucleus sampling to sample outputs. We think this is a potential reason that our RL pipeline only improves the Maj@K performance. In the future, we will explore our RL pipeline on out-of-distribution question prompts, in conjunction with advanced sampling (decoding) strategies, like those based on tree-search methods (Yao et al., 2023). Also, the efficient inference techniques (Kwon et al., 2023; Leviathan et al., 2023; Xia et al., 2023, 2024), which determines

the exploration efficiency of policy models, also play an exceedingly important role.

Algorithms Algorithms process the data and reward signal to the gradient coefficient to update the model parameter. Based on Equation 5, to some extent, all methods now fully TRUST the signal of the reward function to increase or decrease the conditional probability of a certain token. However, it is impossible to ensure the reward signal is always reliable, especially in extremely complex tasks. For example, even the PRM800K datasets (Lightman et al., 2023), which have been carefully annotated by well-trained annotators, still contain approximately 20% of incorrectly annotations⁷. To this end, we will explore the reinforcement learning algorithm that is robust against noisy reward signals. We believe such WEAK-TO-STRONG (Burns et al., 2023) alignment methods will bring a fundamental change to the learning algorithms.

Reward Function Reward function is the source of the training signal. In RL, the reward function is usually the neural reward model. We think there exist three important directions for reward models: 1) **How to enhance the generalization ability of the reward model.** The reward model must be effectively generalized to handle out-of-distribution questions and advanced decoding outputs; otherwise, reinforcement learning may merely stabilize the distribution of LLMs rather than improve their fundamental capabilities; 2) **How to reflect the uncertainty of reward model.** The uncertainty could potentially act as a linking bridge between the weak reward model and the weak-to-strong learning algorithms; 3) **How to efficiently build high-quality process reward models** that can provide fine-grained training signals for the reasoning process (Lightman et al., 2023; Wang et al., 2023b).

6. Conclusion, Limitation, and Future Work

We present DeepSeekMath, which outperforms all open-source models on the competition-level MATH benchmark and approaches the performance of closed models. DeepSeekMath is initialized with DeepSeek-Coder-v1.5 7B and undergoes continual training for 500B tokens, with a significant component of the training data being 120B math tokens sourced from Common Crawl. Our extensive ablation study shows web pages offer significant potential for high-quality mathematical data, while arXiv may not as beneficial as we expected. We introduce Group Relative Policy Optimization (GRPO), a variant of Proximal Policy Optimization (PPO), which can notably improve mathematical reasoning capabilities with less memory consumption. The experiment results show that GRPO is effective even if DeepSeekMath-Instruct 7B has reached a high score on benchmarks. We also provide a unified paradigm to understand a series of methods and summarize several potential directions for more effective reinforcement learning.

Although DeepSeekMath achieves impressive scores on quantitative reasoning benchmarks, its capability on geometry and theorem-proof are relatively weaker than closed models. For instance, in our dry run, the model cannot handle problems related to triangles and ellipses, which may indicate data selection bias in pre-training and fine-tuning. In addition, restricted by the model scale, DeepSeekMath is worse than GPT-4 on few-shot capability. GPT-4 could improve its performance with few-shot inputs, while DeepSeekMath shows similar performance in zero-shot and few-shot evaluation. In the future, we will further improve our engineered data selection pipeline to construct more high-quality pre-trained corpus. In addition, we will explore the potential directions (Section 5.2.3) for more effective reinforcement learning of LLMs.

⁷<https://github.com/openai/prm800k/issues/12#issuecomment-1728491852>

References

- R. Anil, S. Borgeaud, Y. Wu, J. Alayrac, J. Yu, R. Soricut, J. Schalkwyk, A. M. Dai, A. Hauth, K. Millican, D. Silver, S. Petrov, M. Johnson, I. Antonoglou, J. Schrittwieser, A. Glaese, J. Chen, E. Pitler, T. P. Lillicrap, A. Lazaridou, O. Firat, J. Molloy, M. Isard, P. R. Barham, T. Hennigan, B. Lee, F. Viola, M. Reynolds, Y. Xu, R. Doherty, E. Collins, C. Meyer, E. Rutherford, E. Moreira, K. Ayoub, M. Goel, G. Tucker, E. Piqueras, M. Krikun, I. Barr, N. Savinov, I. Danihelka, B. Roelofs, A. White, A. Andreassen, T. von Glehn, L. Yagati, M. Kazemi, L. Gonzalez, M. Khalman, J. Sygnowski, and et al. Gemini: A family of highly capable multimodal models. *CoRR*, abs/2312.11805, 2023. doi: 10.48550/ARXIV.2312.11805. URL <https://doi.org/10.48550/arXiv.2312.11805>.
- J. Austin, A. Odena, M. Nye, M. Bosma, H. Michalewski, D. Dohan, E. Jiang, C. Cai, M. Terry, Q. Le, et al. Program synthesis with large language models. *arXiv preprint arXiv:2108.07732*, 2021.
- Z. Azerbayev, H. Schoelkopf, K. Paster, M. D. Santos, S. McAleer, A. Q. Jiang, J. Deng, S. Biderman, and S. Welleck. Llemma: An open language model for mathematics. *arXiv preprint arXiv:2310.10631*, 2023.
- J. Bai, S. Bai, Y. Chu, Z. Cui, K. Dang, X. Deng, Y. Fan, W. Ge, Y. Han, F. Huang, et al. Qwen technical report. *arXiv preprint arXiv:2309.16609*, 2023.
- C. Burns, P. Izmailov, J. H. Kirchner, B. Baker, L. Gao, L. Aschenbrenner, Y. Chen, A. Ecoffet, M. Joglekar, J. Leike, et al. Weak-to-strong generalization: Eliciting strong capabilities with weak supervision. *arXiv preprint arXiv:2312.09390*, 2023.
- ChatGLM3 Team. Chatglm3 series: Open bilingual chat llms, 2023. URL <https://github.com/THUDM/ChatGLM3>.
- M. Chen, J. Tworek, H. Jun, Q. Yuan, H. P. de Oliveira Pinto, J. Kaplan, H. Edwards, Y. Burda, N. Joseph, G. Brockman, A. Ray, R. Puri, G. Krueger, M. Petrov, H. Khlaaf, G. Sastry, P. Mishkin, B. Chan, S. Gray, N. Ryder, M. Pavlov, A. Power, L. Kaiser, M. Bavarian, C. Winter, P. Tillet, F. P. Such, D. Cummings, M. Plappert, F. Chantzis, E. Barnes, A. Herbert-Voss, W. H. Guss, A. Nichol, A. Paino, N. Tezak, J. Tang, I. Babuschkin, S. Balaji, S. Jain, W. Saunders, C. Hesse, A. N. Carr, J. Leike, J. Achiam, V. Misra, E. Morikawa, A. Radford, M. Knight, M. Brundage, M. Murati, K. Mayer, P. Welinder, B. McGrew, D. Amodei, S. McCandlish, I. Sutskever, and W. Zaremba. Evaluating large language models trained on code. *CoRR*, abs/2107.03374, 2021. URL <https://arxiv.org/abs/2107.03374>.
- W. Chen, X. Ma, X. Wang, and W. W. Cohen. Program of thoughts prompting: Disentangling computation from reasoning for numerical reasoning tasks. *CoRR*, abs/2211.12588, 2022. doi: 10.48550/ARXIV.2211.12588. URL <https://doi.org/10.48550/arXiv.2211.12588>.
- K. Cobbe, V. Kosaraju, M. Bavarian, M. Chen, H. Jun, L. Kaiser, M. Plappert, J. Tworek, J. Hilton, R. Nakano, et al. Training verifiers to solve math word problems. *arXiv preprint arXiv:2110.14168*, 2021.
- T. Computer. Redpajama: an open dataset for training large language models, Oct. 2023. URL <https://github.com/togethercomputer/RedPajama-Data>.
- DeepSeek-AI. Deepseek LLM: scaling open-source language models with longtermism. *CoRR*, abs/2401.02954, 2024. doi: 10.48550/ARXIV.2401.02954. URL <https://doi.org/10.48550/arXiv.2401.02954>.

- Z. Du, Y. Qian, X. Liu, M. Ding, J. Qiu, Z. Yang, and J. Tang. Glm: General language model pretraining with autoregressive blank infilling. In Proceedings of the 60th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers), pages 320–335, 2022.
- L. Gao, A. Madaan, S. Zhou, U. Alon, P. Liu, Y. Yang, J. Callan, and G. Neubig. PAL: program-aided language models. In A. Krause, E. Brunskill, K. Cho, B. Engelhardt, S. Sabato, and J. Scarlett, editors, International Conference on Machine Learning, ICML 2023, 23-29 July 2023, Honolulu, Hawaii, USA, volume 202 of Proceedings of Machine Learning Research, pages 10764–10799. PMLR, 2023. URL <https://proceedings.mlr.press/v202/gao23f.html>.
- Z. Gou, Z. Shao, Y. Gong, Y. Shen, Y. Yang, M. Huang, N. Duan, and W. Chen. Tora: A tool-integrated reasoning agent for mathematical problem solving. CoRR, abs/2309.17452, 2023. doi: 10.48550/ARXIV.2309.17452. URL <https://doi.org/10.48550/arXiv.2309.17452>.
- D. Guo, Q. Zhu, D. Yang, Z. Xie, K. Dong, W. Zhang, G. Chen, X. Bi, Y. Wu, Y. K. Li, F. Luo, Y. Xiong, and W. Liang. Deepseek-coder: When the large language model meets programming – the rise of code intelligence, 2024.
- D. Hendrycks, C. Burns, S. Basart, A. Zou, M. Mazeika, D. Song, and J. Steinhardt. Measuring massive multitask language understanding. arXiv preprint arXiv:2009.03300, 2020.
- D. Hendrycks, C. Burns, S. Kadavath, A. Arora, S. Basart, E. Tang, D. Song, and J. Steinhardt. Measuring mathematical problem solving with the math dataset. arXiv preprint arXiv:2103.03874, 2021.
- High-flyer. Hai-llm: 高效且轻量的大模型训练工具, 2023. URL <https://www.high-flyer.cn/en/blog/hai-llm>.
- Inflection AI. Inflection-2, 2023. URL <https://inflection.ai/inflection-2>.
- A. Q. Jiang, S. Welleck, J. P. Zhou, W. Li, J. Liu, M. Jamnik, T. Lacroix, Y. Wu, and G. Lample. Draft, sketch, and prove: Guiding formal theorem provers with informal proofs. arXiv preprint arXiv:2210.12283, 2022.
- A. Q. Jiang, A. Sablayrolles, A. Mensch, C. Bamford, D. S. Chaplot, D. d. l. Casas, F. Bressand, G. Lengyel, G. Lample, L. Saulnier, et al. Mistral 7b. arXiv preprint arXiv:2310.06825, 2023.
- A. Joulin, E. Grave, P. Bojanowski, M. Douze, H. Jégou, and T. Mikolov. Fasttext. zip: Compressing text classification models. arXiv preprint arXiv:1612.03651, 2016.
- W. Kwon, Z. Li, S. Zhuang, Y. Sheng, L. Zheng, C. H. Yu, J. E. Gonzalez, H. Zhang, and I. Stoica. Efficient memory management for large language model serving with pagedattention. In Proceedings of the ACM SIGOPS 29th Symposium on Operating Systems Principles, 2023.
- Y. Leviathan, M. Kalman, and Y. Matias. Fast inference from transformers via speculative decoding. In International Conference on Machine Learning, pages 19274–19286. PMLR, 2023.
- A. Lewkowycz, A. Andreassen, D. Dohan, E. Dyer, H. Michalewski, V. Ramasesh, A. Slone, C. Anil, I. Schlag, T. Gutman-Solo, et al. Solving quantitative reasoning problems with language models. Advances in Neural Information Processing Systems, 35:3843–3857, 2022a.

- A. Lewkowycz, A. Andreassen, D. Dohan, E. Dyer, H. Michalewski, V. V. Ramasesh, A. Slone, C. Anil, I. Schlag, T. Gutman-Solo, Y. Wu, B. Neyshabur, G. Gur-Ari, and V. Misra. Solving quantitative reasoning problems with language models. In S. Koyejo, S. Mohamed, A. Agarwal, D. Belgrave, K. Cho, and A. Oh, editors, *Advances in Neural Information Processing Systems 35: Annual Conference on Neural Information Processing Systems 2022, NeurIPS 2022, New Orleans, LA, USA, November 28 - December 9, 2022, 2022b*. URL http://papers.nips.cc/paper_files/paper/2022/hash/18abbef8cfe9203fdf9053c9c4fe191-Abstract-Conference.html.
- H. Lightman, V. Kosaraju, Y. Burda, H. Edwards, B. Baker, T. Lee, J. Leike, J. Schulman, I. Sutskever, and K. Cobbe. Let’s verify step by step. *arXiv preprint arXiv:2305.20050*, 2023.
- I. Loshchilov and F. Hutter. Decoupled weight decay regularization. *arXiv preprint arXiv:1711.05101*, 2017.
- H. Luo, Q. Sun, C. Xu, P. Zhao, J. Lou, C. Tao, X. Geng, Q. Lin, S. Chen, and D. Zhang. Wizardmath: Empowering mathematical reasoning for large language models via reinforced evol-instruct. *arXiv preprint arXiv:2308.09583*, 2023.
- S. Mishra, M. Finlayson, P. Lu, L. Tang, S. Welleck, C. Baral, T. Rajpurohit, O. Tafjord, A. Sabharwal, P. Clark, and A. Kalyan. LILA: A unified benchmark for mathematical reasoning. In Y. Goldberg, Z. Kozareva, and Y. Zhang, editors, *Proceedings of the 2022 Conference on Empirical Methods in Natural Language Processing, EMNLP 2022, Abu Dhabi, United Arab Emirates, December 7-11, 2022*, pages 5807–5832. Association for Computational Linguistics, 2022. doi: 10.18653/V1/2022.EMNLP-MAIN.392. URL <https://doi.org/10.18653/v1/2022.emnlp-main.392>.
- X. Nguyen, W. Zhang, X. Li, M. M. Aljunied, Q. Tan, L. Cheng, G. Chen, Y. Deng, S. Yang, C. Liu, H. Zhang, and L. Bing. Seallms - large language models for southeast asia. *CoRR*, abs/2312.00738, 2023. doi: 10.48550/ARXIV.2312.00738. URL <https://doi.org/10.48550/arXiv.2312.00738>.
- OpenAI. GPT4 technical report. *arXiv preprint arXiv:2303.08774*, 2023.
- L. Ouyang, J. Wu, X. Jiang, D. Almeida, C. Wainwright, P. Mishkin, C. Zhang, S. Agarwal, K. Slama, A. Ray, et al. Training language models to follow instructions with human feedback. *Advances in Neural Information Processing Systems*, 35:27730–27744, 2022.
- K. Paster, M. D. Santos, Z. Azerbayev, and J. Ba. Openwebmath: An open dataset of high-quality mathematical web text. *CoRR*, abs/2310.06786, 2023. doi: 10.48550/ARXIV.2310.06786. URL <https://doi.org/10.48550/arXiv.2310.06786>.
- L. C. Paulson. Three years of experience with sledgehammer, a practical link between automatic and interactive theorem provers. In R. A. Schmidt, S. Schulz, and B. Konev, editors, *Proceedings of the 2nd Workshop on Practical Aspects of Automated Reasoning, PAAR-2010, Edinburgh, Scotland, UK, July 14, 2010*, volume 9 of EPIc Series in Computing, pages 1–10. EasyChair, 2010. doi: 10.29007/TNFD. URL <https://doi.org/10.29007/tnfd>.
- S. Polu and I. Sutskever. Generative language modeling for automated theorem proving. *CoRR*, abs/2009.03393, 2020. URL <https://arxiv.org/abs/2009.03393>.
- R. Rafailov, A. Sharma, E. Mitchell, S. Ermon, C. D. Manning, and C. Finn. Direct preference optimization: Your language model is secretly a reward model. 2023.

- J. Schulman. Approximating kl divergence, 2020. URL <http://joschu.net/blog/kl-approx.html>.
- J. Schulman, P. Moritz, S. Levine, M. Jordan, and P. Abbeel. High-dimensional continuous control using generalized advantage estimation. arXiv preprint arXiv:1506.02438, 2015.
- J. Schulman, F. Wolski, P. Dhariwal, A. Radford, and O. Klimov. Proximal policy optimization algorithms. arXiv preprint arXiv:1707.06347, 2017.
- F. Shi, M. Suzgun, M. Freitag, X. Wang, S. Srivats, S. Vosoughi, H. W. Chung, Y. Tay, S. Ruder, D. Zhou, D. Das, and J. Wei. Language models are multilingual chain-of-thought reasoners. In The Eleventh International Conference on Learning Representations, ICLR 2023, Kigali, Rwanda, May 1-5, 2023. OpenReview.net, 2023. URL <https://openreview.net/pdf?id=fR3wGCK-IXp>.
- F. Song, B. Yu, M. Li, H. Yu, F. Huang, Y. Li, and H. Wang. Preference ranking optimization for human alignment. arXiv preprint arXiv:2306.17492, 2023.
- M. Suzgun, N. Scales, N. Schärli, S. Gehrmann, Y. Tay, H. W. Chung, A. Chowdhery, Q. V. Le, E. H. Chi, D. Zhou, et al. Challenging big-bench tasks and whether chain-of-thought can solve them. arXiv preprint arXiv:2210.09261, 2022.
- T. Tao. Embracing change and resetting expectations, 2023. URL <https://unlocked.microsoft.com/ai-anthology/terence-tao/>.
- H. Touvron, L. Martin, K. Stone, P. Albert, A. Almahairi, Y. Babaei, N. Bashlykov, S. Batra, P. Bhargava, S. Bhosale, D. Bikel, L. Blecher, C. Canton-Ferrer, M. Chen, G. Cucurull, D. Esiobu, J. Fernandes, J. Fu, W. Fu, B. Fuller, C. Gao, V. Goswami, N. Goyal, A. Hartshorn, S. Hosseini, R. Hou, H. Inan, M. Kardas, V. Kerkez, M. Khabsa, I. Kloumann, A. Korenev, P. S. Koura, M. Lachaux, T. Lavril, J. Lee, D. Liskovich, Y. Lu, Y. Mao, X. Martinet, T. Mihaylov, P. Mishra, I. Molybog, Y. Nie, A. Poulton, J. Reizenstein, R. Rungta, K. Saladi, A. Schelten, R. Silva, E. M. Smith, R. Subramanian, X. E. Tan, B. Tang, R. Taylor, A. Williams, J. X. Kuan, P. Xu, Z. Yan, I. Zarov, Y. Zhang, A. Fan, M. Kambadur, S. Narang, A. Rodriguez, R. Stojnic, S. Edunov, and T. Scialom. Llama 2: Open foundation and fine-tuned chat models. CoRR, abs/2307.09288, 2023. doi: 10.48550/arXiv.2307.09288. URL <https://doi.org/10.48550/arXiv.2307.09288>.
- T. H. Trinh, Y. Wu, Q. V. Le, H. He, and T. Luong. Solving olympiad geometry without human demonstrations. Nature, 625(7995):476–482, 2024.
- P. Wang, L. Li, L. Chen, F. Song, B. Lin, Y. Cao, T. Liu, and Z. Sui. Making large language models better reasoners with alignment. arXiv preprint arXiv:2309.02144, 2023a.
- P. Wang, L. Li, Z. Shao, R. Xu, D. Dai, Y. Li, D. Chen, Y. Wu, and Z. Sui. Math-shepherd: Verify and reinforce llms step-by-step without human annotations. CoRR, abs/2312.08935, 2023b.
- Z. Wang, R. Xia, and P. Liu. Generative AI for math: Part I - mathpile: A billion-token-scale pretraining corpus for math. CoRR, abs/2312.17120, 2023c. doi: 10.48550/ARXIV.2312.17120. URL <https://doi.org/10.48550/arXiv.2312.17120>.
- J. Wei, X. Wang, D. Schuurmans, M. Bosma, B. Ichter, F. Xia, E. H. Chi, Q. V. Le, and D. Zhou. Chain-of-thought prompting elicits reasoning in large language models. In NeurIPS, 2022. URL http://papers.nips.cc/paper_files/paper/2022/hash/9d5609613524ecf4f15af0f7b31abca4-Abstract-Conference.html.

- T. Wei, J. Luan, W. Liu, S. Dong, and B. Wang. Cmath: Can your language model pass chinese elementary school math test?, 2023.
- M. Wenzel, L. C. Paulson, and T. Nipkow. The isabelle framework. In O. A. Mohamed, C. A. Muñoz, and S. Tahar, editors, *Theorem Proving in Higher Order Logics, 21st International Conference, TPHOLs 2008, Montreal, Canada, August 18-21, 2008. Proceedings*, volume 5170 of *Lecture Notes in Computer Science*, pages 33–38. Springer, 2008. doi: 10.1007/978-3-540-71067-7_7. URL https://doi.org/10.1007/978-3-540-71067-7_7.
- H. Xia, T. Ge, P. Wang, S.-Q. Chen, F. Wei, and Z. Sui. Speculative decoding: Exploiting speculative execution for accelerating seq2seq generation. In H. Bouamor, J. Pino, and K. Bali, editors, *Findings of the Association for Computational Linguistics: EMNLP 2023*, pages 3909–3925, Singapore, Dec. 2023. Association for Computational Linguistics. doi: 10.18653/v1/2023.findings-emnlp.257. URL <https://aclanthology.org/2023.findings-emnlp.257>.
- H. Xia, Z. Yang, Q. Dong, P. Wang, Y. Li, T. Ge, T. Liu, W. Li, and Z. Sui. Unlocking efficiency in large language model inference: A comprehensive survey of speculative decoding. *arXiv preprint arXiv:2401.07851*, 2024.
- S. Yao, D. Yu, J. Zhao, I. Shafran, T. L. Griffiths, Y. Cao, and K. Narasimhan. Tree of thoughts: Deliberate problem solving with large language models. *arXiv preprint arXiv:2305.10601*, 2023.
- L. Yu, W. Jiang, H. Shi, J. Yu, Z. Liu, Y. Zhang, J. T. Kwok, Z. Li, A. Weller, and W. Liu. Metamath: Bootstrap your own mathematical questions for large language models. *CoRR*, abs/2309.12284, 2023. doi: 10.48550/ARXIV.2309.12284. URL <https://doi.org/10.48550/arXiv.2309.12284>.
- Z. Yuan, H. Yuan, C. Li, G. Dong, C. Tan, and C. Zhou. Scaling relationship on learning mathematical reasoning with large language models. *arXiv preprint arXiv:2308.01825*, 2023a.
- Z. Yuan, H. Yuan, C. Tan, W. Wang, S. Huang, and F. Huang. Rrhf: Rank responses to align language models with human feedback without tears. *arXiv preprint arXiv:2304.05302*, 2023b.
- X. Yue, X. Qu, G. Zhang, Y. Fu, W. Huang, H. Sun, Y. Su, and W. Chen. Mammoth: Building math generalist models through hybrid instruction tuning. *CoRR*, abs/2309.05653, 2023. doi: 10.48550/ARXIV.2309.05653. URL <https://doi.org/10.48550/arXiv.2309.05653>.
- K. Zheng, J. M. Han, and S. Polu. Minif2f: a cross-system benchmark for formal olympiad-level mathematics. *arXiv preprint arXiv:2109.00110*, 2021.
- W. Zhong, R. Cui, Y. Guo, Y. Liang, S. Lu, Y. Wang, A. Saied, W. Chen, and N. Duan. AGIEval: A human-centric benchmark for evaluating foundation models. *CoRR*, abs/2304.06364, 2023. doi: 10.48550/arXiv.2304.06364. URL <https://doi.org/10.48550/arXiv.2304.06364>.

A. Appendix

A.1. Analysis of Reinforcement Learning

We provide the detailed derivation of the **data source** and **gradient coefficient** (algorithm and reward function) across various methods, including SFT, RFT, Online RFT, DPO, PPO, and GRPO.

A.1.1. Supervised Fine-tuning

The objective of Supervised Fine-tuning is maximizing the following objective:

$$\mathcal{J}_{SFT}(\theta) = \mathbb{E}[q, o \sim P_{sft}(Q, O)] \left(\frac{1}{|o|} \sum_{t=1}^{|o|} \log \pi_{\theta}(o_t | q, o_{<t}) \right). \quad (6)$$

The gradient of $\mathcal{J}_{SFT}(\theta)$ is:

$$\nabla_{\theta} \mathcal{J}_{SFT} = \mathbb{E}[q, o \sim P_{sft}(Q, O)] \left(\frac{1}{|o|} \sum_{t=1}^{|o|} \nabla_{\theta} \log \pi_{\theta}(o_t | q, o_{<t}) \right). \quad (7)$$

Data Source: The **dataset employed for SFT**. Reward Function: This can be regarded as human selection. **Gradient Coefficient**: always **set to 1**.

A.1.2. Rejection Sampling Fine-tuning

Rejection Sampling Fine-tuning **first samples multiple outputs** from the **supervised fine-tuned LLMs** for each question, and then trains LLMs on the sampled outputs with the correct answer. Formally, the objective of RFT is to **maximize** the following **objectives**:

$$\mathcal{J}_{RFT}(\theta) = \mathbb{E}[q \sim P_{sft}(Q), o \sim \pi_{sft}(O|q)] \left(\frac{1}{|o|} \sum_{t=1}^{|o|} \mathbb{I}(o) \log \pi_{\theta}(o_t | q, o_{<t}) \right). \quad (8)$$

The gradient of $\mathcal{J}_{RFT}(\theta)$ is:

$$\nabla_{\theta} \mathcal{J}_{RFT}(\theta) = \mathbb{E}[q \sim P_{sft}(Q), o \sim \pi_{sft}(O|q)] \left(\frac{1}{|o|} \sum_{t=1}^{|o|} \mathbb{I}(o) \nabla_{\theta} \log \pi_{\theta}(o_t | q, o_{<t}) \right). \quad (9)$$

Data Source: question in **SFT dataset** with outputs sampled from **SFT model**. Reward Function: Rule (whether the **answer is correct or not**). Gradient Coefficient:

$$GC_{RFT}(q, o, t) = \mathbb{I}(o) = \begin{cases} 1 & \text{the answer of } o \text{ is correct} \\ 0 & \text{the answer of } o \text{ is incorrect} \end{cases} \quad (10)$$

A.1.3. Online Rejection Sampling Fine-tuning

The only difference between RFT and Online RFT is that the outputs of Online RFT are sampled from the **real-time policy model** π_{θ} , rather than from the **SFT model** $\pi_{\theta_{sft}}$. Therefore, the gradient of online RFT is:

$$\nabla_{\theta} \mathcal{J}_{OnRFT}(\theta) = \mathbb{E}[q \sim P_{sft}(Q), o \sim \pi_{\theta}(O|q)] \left(\frac{1}{|o|} \sum_{t=1}^{|o|} \mathbb{I}(o) \nabla_{\theta} \log \pi_{\theta}(o_t | q, o_{<t}) \right). \quad (11)$$

A.1.4. Direct Preference Optimization (DPO)

The objective of DPO is:

$$\mathcal{J}_{DPO}(\theta) = \mathbb{E}[q \sim P_{sft}(Q), o^+, o^- \sim \pi_{sft}(O|q)] \log \sigma \left(\beta \frac{1}{|o^+|} \sum_{t=1}^{|o^+|} \log \frac{\pi_{\theta}(o_t^+|q, o_{<t}^+)}{\pi_{ref}(o_t^+|q, o_{<t}^+)} - \beta \frac{1}{|o^-|} \sum_{t=1}^{|o^-|} \log \frac{\pi_{\theta}(o_t^-|q, o_{<t}^-)}{\pi_{ref}(o_t^-|q, o_{<t}^-)} \right) \quad (12)$$

The gradient of $\mathcal{J}_{DPO}(\theta)$ is:

$$\begin{aligned} \nabla_{\theta} \mathcal{J}_{DPO}(\theta) = \mathbb{E}[q \sim P_{sft}(Q), o^+, o^- \sim \pi_{sft}(O|q)] & \left(\frac{1}{|o^+|} \sum_{t=1}^{|o^+|} GC_{DPO}(q, o, t) \nabla_{\theta} \log \pi_{\theta}(o_t^+|q, o_{<t}^+) \right. \\ & \left. - \frac{1}{|o^-|} \sum_{t=1}^{|o^-|} GC_{DPO}(q, o, t) \nabla_{\theta} \log \pi_{\theta}(o_t^-|q, o_{<t}^-) \right) \end{aligned} \quad (13)$$

Data Source: question in SFT dataset with **outputs sampled** from SFT model. Reward Function: **human preference** in the general domain (can be 'Rule' in **mathematical tasks**). Gradient Coefficient:

$$GC_{DPO}(q, o, t) = \sigma \left(\beta \log \frac{\pi_{\theta}(o_t^-|q, o_{<t}^-)}{\pi_{ref}(o_t^-|q, o_{<t}^-)} - \beta \log \frac{\pi_{\theta}(o_t^+|q, o_{<t}^+)}{\pi_{ref}(o_t^+|q, o_{<t}^+)} \right) \quad (14)$$

A.1.5. Proximal Policy Optimization (PPO)

The objective of PPO is:

$$\mathcal{J}_{PPO}(\theta) = \mathbb{E}[q \sim P_{sft}(Q), o \sim \pi_{\theta_{old}}(O|q)] \frac{1}{|o|} \sum_{t=1}^{|o|} \min \left[\frac{\pi_{\theta}(o_t|q, o_{<t})}{\pi_{\theta_{old}}(o_t|q, o_{<t})} A_t, \text{clip} \left(\frac{\pi_{\theta}(o_t|q, o_{<t})}{\pi_{\theta_{old}}(o_t|q, o_{<t})}, 1 - \epsilon, 1 + \epsilon \right) A_t \right]. \quad (15)$$

To simplify the analysis, it is assumed that the model **only has a single update** following each **exploration stage**, thereby ensuring that $\pi_{\theta_{old}} = \pi_{\theta}$. In this case, we can **remove the min and clip operation**:

$$\mathcal{J}_{PPO}(\theta) = \mathbb{E}[q \sim P_{sft}(Q), o \sim \pi_{\theta_{old}}(O|q)] \frac{1}{|o|} \sum_{t=1}^{|o|} \frac{\pi_{\theta}(o_t|q, o_{<t})}{\pi_{\theta_{old}}(o_t|q, o_{<t})} A_t. \quad (16)$$

The gradient of $\mathcal{J}_{PPO}(\theta)$ is:

$$\nabla_{\theta} \mathcal{J}_{PPO}(\theta) = \mathbb{E}[q \sim P_{sft}(Q), o \sim \pi_{\theta_{old}}(O|q)] \frac{1}{|o|} \sum_{t=1}^{|o|} A_t \nabla_{\theta} \log \pi_{\theta}(o_t|q, o_{<t}) \quad (17)$$

Data Source: question in SFT dataset with outputs sampled from policy model. Reward Function: reward model. Gradient Coefficient:

$$GC_{PPO}(q, o, t, \pi_{\theta_{rm}}) = A_t, \quad (18)$$

where A_t is the advantage, which is computed by **applying Generalized Advantage Estimation (GAE)** (Schulman et al., 2015), based on the **rewards $\{r_{\geq t}\}$** and a learned **value function V_{ψ}** .

A.1.6. Group Relative Policy Optimization (GRPO)

The objective of GRPO is (assume $\pi_{\theta_{old}} = \pi_{\theta}$ for simplified analysis):

$$\begin{aligned} \mathcal{J}_{GRPO}(\theta) = \mathbb{E}[q \sim P_{sft}(Q), \{o_i\}_{i=1}^G \sim \pi_{\theta_{old}}(O|q)] \\ \frac{1}{G} \sum_{i=1}^G \frac{1}{|o_i|} \sum_{t=1}^{|o_i|} \left[\frac{\pi_{\theta}(o_{i,t}|q, o_{i,<t})}{\pi_{\theta_{old}}(o_{i,t}|q, o_{i,<t})} \hat{A}_{i,t} - \beta \left(\frac{\pi_{ref}(o_{i,t}|q, o_{i,<t})}{\pi_{\theta}(o_{i,t}|q, o_{i,<t})} - \log \frac{\pi_{ref}(o_{i,t}|q, o_{i,<t})}{\pi_{\theta}(o_{i,t}|q, o_{i,<t})} - 1 \right) \right]. \end{aligned} \quad (19)$$

The gradient of $\mathcal{J}_{GRPO}(\theta)$ is:

$$\begin{aligned} \nabla_{\theta} \mathcal{J}_{GRPO}(\theta) &= \mathbb{E}[q \sim P_{sft}(Q), \{o_i\}_{i=1}^G \sim \pi_{\theta_{old}}(O|q)] \\ &\quad \frac{1}{G} \sum_{i=1}^G \frac{1}{|o_i|} \sum_{t=1}^{|o_i|} \left[\hat{A}_{i,t} + \beta \left(\frac{\pi_{ref}(o_{i,t}|o_{i,<t})}{\pi_{\theta}(o_{i,t}|o_{i,<t})} - 1 \right) \right] \nabla_{\theta} \log \pi_{\theta}(o_{i,t}|q, o_{i,<t}). \end{aligned} \quad (20)$$

Data Source: question in SFT dataset with outputs sampled from policy model. Reward Function: reward model. Gradient Coefficient:

$$GC_{GRPO}(q, o, t, \pi_{\theta_{rm}}) = \hat{A}_{i,t} + \beta \left(\frac{\pi_{ref}(o_{i,t}|o_{i,<t})}{\pi_{\theta}(o_{i,t}|o_{i,<t})} - 1 \right), \quad (21)$$

where $\hat{A}_{i,t}$ is computed based on the group reward scores.

Direct Preference Optimization: Your Language Model is Secretly a Reward Model

Rafael Rafailov^{*†}Archit Sharma^{*†}Eric Mitchell^{*†}Stefano Ermon^{†‡}Christopher D. Manning[†]Chelsea Finn[†]

[†]Stanford University [‡]CZ Biohub
 {rafailov,architsh,eric.mitchell}@cs.stanford.edu

Abstract

While large-scale unsupervised language models (LMs) learn broad world knowledge and some reasoning skills, achieving precise control of their behavior is difficult due to the completely unsupervised nature of their training. Existing methods for gaining such steerability collect human labels of the relative quality of model generations and fine-tune the unsupervised LM to align with these preferences, often with reinforcement learning from human feedback (RLHF). However, RLHF is a complex and often unstable procedure, first fitting a reward model that reflects the human preferences, and then fine-tuning the large unsupervised LM using reinforcement learning to maximize this estimated reward without drifting too far from the original model. In this paper we introduce a new parameterization of the reward model in RLHF that enables extraction of the corresponding optimal policy in closed form, allowing us to solve the standard RLHF problem with only a simple classification loss. The resulting algorithm, which we call *Direct Preference Optimization* (DPO), is stable, performant, and computationally lightweight, eliminating the need for sampling from the LM during fine-tuning or performing significant hyperparameter tuning. Our experiments show that DPO can fine-tune LMs to align with human preferences as well as or better than existing methods. Notably, fine-tuning with DPO exceeds PPO-based RLHF in ability to control sentiment of generations, and matches or improves response quality in summarization and single-turn dialogue while being substantially simpler to implement and train.

1 Introduction

Large unsupervised language models (LMs) trained on very large datasets acquire surprising capabilities [11, 7, 42, 8]. However, these models are trained on data generated by humans with a wide variety of goals, priorities, and skillsets. Some of these goals and skillsets may not be desirable to imitate; for example, while we may want our AI coding assistant to understand common programming mistakes in order to correct them, nevertheless, when generating code, we would like to bias our model toward the (potentially rare) high-quality coding ability present in its training data. Similarly, we might want our language model to be aware of a common misconception believed by 50% of people, but we certainly do not want the model to claim this misconception to be true in 50% of queries about it! In other words, selecting the model’s desired responses and behavior from its very wide knowledge and abilities is crucial to building AI systems that are safe, performant, and controllable [28]. While existing methods typically steer LMs to match human preferences using reinforcement learning (RL),

^{*}Equal contribution; more junior authors listed earlier.

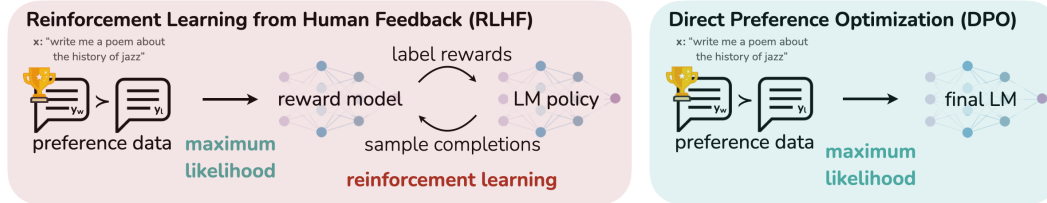


Figure 1: **DPO optimizes for human preferences while avoiding reinforcement learning.** Existing methods for fine-tuning language models with human feedback first fit a reward model to a dataset of prompts and human preferences over pairs of responses, and then use RL to find a policy that maximizes the learned reward. In contrast, DPO directly optimizes for the policy best satisfying the preferences with a simple classification objective, fitting an implicit reward model whose corresponding optimal policy can be extracted in closed form.

we will show that the RL-based objective used by existing methods can be optimized exactly with a simple binary cross-entropy objective, greatly simplifying the preference learning pipeline.

At a high level, existing methods instill the desired behaviors into a language model using curated sets of human preferences representing the types of behaviors that humans find safe and helpful. This preference learning stage occurs after an initial stage of large-scale unsupervised pre-training on a large text dataset. While the most straightforward approach to preference learning is supervised fine-tuning on human demonstrations of high quality responses, the most successful class of methods is reinforcement learning from human (or AI) feedback (RLHF/RLAIF; [12, 2]). RLHF methods fit a reward model to a dataset of human preferences and then use RL to optimize a language model policy to produce responses assigned high reward without drifting excessively far from the original model. While RLHF produces models with impressive conversational and coding abilities, the RLHF pipeline is considerably more complex than supervised learning, involving training multiple LMs and sampling from the LM policy in the loop of training, incurring significant computational costs.

In this paper, we show how to directly optimize a language model to adhere to human preferences, without explicit reward modeling or reinforcement learning. We propose *Direct Preference Optimization (DPO)*, an algorithm that implicitly optimizes the same objective as existing RLHF algorithms (reward maximization with a KL-divergence constraint) but is simple to implement and straightforward to train. Intuitively, the DPO update increases the relative log probability of preferred to dispreferred responses, but it incorporates a dynamic, per-example importance weight that prevents the model degeneration that we find occurs with a naive probability ratio objective. Like existing algorithms, DPO relies on a theoretical preference model (such as the Bradley-Terry model; [5]) that measures how well a given reward function aligns with empirical preference data. However, while existing methods use the preference model to define a preference loss to train a reward model and then train a policy that optimizes the learned reward model, DPO uses a change of variables to define the preference loss as a function of the policy directly. Given a dataset of human preferences over model responses, DPO can therefore optimize a policy using a simple binary cross entropy objective, producing the optimal policy to an implicit reward function fit to the preference data.

Our main contribution is *Direct Preference Optimization (DPO)*, a simple RL-free algorithm for training language models from preferences. Our experiments show that DPO is at least as effective as existing methods, including PPO-based RLHF, for learning from preferences in tasks such as sentiment modulation, summarization, and dialogue, using language models with up to 6B parameters.

variable
change
↓
preference
loss
(policy
func)
←

2 Related Work

Self-supervised language models of increasing scale learn to complete some tasks zero-shot [33] or with few-shot prompts [6, 27, 11]. However, their performance on downstream tasks and alignment with user intent can be significantly improved by fine-tuning on datasets of instructions and human-written completions [25, 38, 13, 41]. This ‘instruction-tuning’ procedure enables LLMs to generalize to instructions outside of the instruction-tuning set and generally increase their usability [13]. Despite the success of instruction tuning, relative human judgments of response quality are often easier to collect than expert demonstrations, and thus subsequent works have fine-tuned LLMs with datasets of human preferences, improving proficiency in translation [20], summarization [40, 51], story-telling [51], and instruction-following [28, 34]. These methods first optimize a neural network reward function for compatibility with the dataset of preferences under a preference model such as the

Bradley-Terry model [5], then fine-tune a language model to maximize the given reward using reinforcement learning algorithms, commonly REINFORCE [47], proximal policy optimization (PPO; [39]), or variants [34]. A closely-related line of work leverages LLMs fine-tuned for instruction following with human feedback to generate additional synthetic preference data for targeted attributes such as safety or harmlessness [2], using only weak supervision from humans in the form of a text rubric for the LLM’s annotations. These methods represent a convergence of two bodies of work: one body of work on training language models with reinforcement learning for a variety of objectives [35, 29, 48] and another body of work on general methods for learning from human preferences [12, 21]. Despite the appeal of using relative human preferences, fine-tuning large language models with reinforcement learning remains a major practical challenge; this work provides a theoretically-justified approach to optimizing relative preferences without RL.

Outside of the context of language, learning policies from preferences has been studied in both bandit and reinforcement learning settings, and several approaches have been proposed. Contextual bandit learning using preferences or rankings of actions, rather than rewards, is known as a contextual dueling bandit (CDB; [50, 14]). In the absence of absolute rewards, theoretical analysis of CDBs substitutes the notion of an optimal policy with a *von Neumann winner*, a policy whose expected win rate against any other policy is at least 50% [14]. However, in the CDB setting, preference labels are given online, while in learning from human preferences, we typically learn from a fixed batch of offline preference-annotated action pairs [49]. Similarly, *preference-based RL* (PbRL) learns from binary preferences generated by an *unknown* ‘scoring’ function rather than rewards [9, 37]. Various algorithms for PbRL exist, including methods that can reuse off-policy preference data, but generally involve first explicitly estimating the latent scoring function (i.e. the reward model) and subsequently optimizing it [16, 9, 12, 36, 21]. We instead present a single stage policy learning approach that directly optimizes a policy to satisfy preferences.

3 Preliminaries

We review the RLHF pipeline in Ziegler et al. (and later [40, 1, 28]). It usually includes three phases: 1) supervised fine-tuning (SFT); 2) preference sampling and reward learning and 3) RL optimization.

SFT: RLHF typically begins by fine-tuning a pre-trained LM with supervised learning on high-quality data for the downstream task(s) of interest (dialogue, summarization, etc.), to obtain a model π^{SFT} .

Reward Modelling Phase: In the second phase the SFT model is prompted with prompts x to produce pairs of answers $(y_1, y_2) \sim \pi^{\text{SFT}}(y | x)$. These are then presented to human labelers who express preferences for one answer, denoted as $y_w \succ y_l | x$ where y_w and y_l denotes the preferred and dispreferred completion amongst (y_1, y_2) respectively. The preferences are assumed to be generated by some latent reward model $r^*(y, x)$, which we do not have access to. There are a number of approaches used to model preferences, the Bradley-Terry (BT) [5] model being a popular choice (although more general Plackett-Luce ranking models [32, 23] are also compatible with the framework if we have access to several ranked answers). The BT model stipulates that the human preference distribution p^* can be written as:

$$p^*(y_1 \succ y_2 | x) = \frac{\exp(r^*(x, y_1))}{\exp(r^*(x, y_1)) + \exp(r^*(x, y_2))}. \quad (1)$$

→ reward model

Assuming access to a static dataset of comparisons $\mathcal{D} = \{x^{(i)}, y_w^{(i)}, y_l^{(i)}\}_{i=1}^N$ sampled from p^* , we can parametrize a reward model $r_\phi(x, y)$ and estimate the parameters via maximum likelihood. Framing the problem as a binary classification we have the negative log-likelihood loss:

$$\mathcal{L}_R(r_\phi, \mathcal{D}) = -\mathbb{E}_{(x, y_w, y_l) \sim \mathcal{D}} [\log \sigma(r_\phi(x, y_w) - r_\phi(x, y_l))] \quad (2)$$

→ logistic func

where σ is the logistic function. In the context of LMs, the network $r_\phi(x, y)$ is often initialized from the SFT model $\pi^{\text{SFT}}(y | x)$ with the addition of a linear layer on top of the final transformer layer that produces a single scalar prediction for the reward value [51]. To ensure a reward function with lower variance, prior works normalize the rewards, such that $\mathbb{E}_{x, y \sim \mathcal{D}} [r_\phi(x, y)] = 0$ for all x .

RL Fine-Tuning Phase: During the RL phase, the learned reward function is used to provide feedback to the language model. Following prior works [17, 18], the optimization is formulated as

$$\max_{\pi_\theta} \mathbb{E}_{x \sim \mathcal{D}, y \sim \pi_\theta(y|x)} [r_\phi(x, y)] - \beta \mathbb{D}_{\text{KL}}[\pi_\theta(y | x) || \pi_{\text{ref}}(y | x)], \quad (3)$$

3
 ← para controlling
 ↓ model policy
 → ref policy

where β is a parameter controlling the deviation from the base reference policy π_{ref} , namely the initial SFT model π^{SFT} . In practice, the language model policy π_{θ} is also initialized to π^{SFT} . The added constraint is important, as it prevents the model from deviating too far from the distribution on which the reward model is accurate, as well as maintaining the generation diversity and preventing mode-collapse to single high-reward answers. Due to the discrete nature of language generation, this objective is not differentiable and is typically optimized with reinforcement learning. The standard approach [51, 40, 1, 28] has been to construct the reward function $r(x, y) = r_{\phi}(x, y) - \beta(\log \pi_{\theta}(y | x) - \log \pi_{\text{ref}}(y | x))$, and maximize using PPO [39].

4 Direct Preference Optimization

Motivated by the challenges of applying reinforcement learning algorithms on large-scale problems such as fine-tuning language models, our goal is to derive a simple approach for policy optimization using preferences directly. Unlike prior RLHF methods, which learn a reward and then optimize it via RL, our approach leverages a particular choice of reward model parameterization that enables extraction of its optimal policy in closed form, without an RL training loop. As we will describe next in detail, our key insight is to leverage an analytical mapping from reward functions to optimal policies, which enables us to transform a loss function over reward functions into a loss function over policies. This change-of-variables approach avoids fitting an explicit, standalone reward model, while still optimizing under existing models of human preferences, such as the Bradley-Terry model. In essence, the policy network represents both the language model and the (implicit) reward.

Deriving the DPO objective. We start with the same RL objective as prior work, Eq. 3, under a general reward function r . Following prior work [31, 30, 19, 15], it is straightforward to show that the optimal solution to the KL-constrained reward maximization objective in Eq. 3 takes the form:

$$\pi_r(y | x) = \frac{1}{Z(x)} \pi_{\text{ref}}(y | x) \exp\left(\frac{1}{\beta} r(x, y)\right), \quad (4)$$

Handwritten notes: "optimal policy" with an arrow pointing to $\pi_r(y | x)$; "partition func" with an arrow pointing to $Z(x)$; "ref policy" with an arrow pointing to $\pi_{\text{ref}}(y | x)$.

where $Z(x) = \sum_y \pi_{\text{ref}}(y | x) \exp\left(\frac{1}{\beta} r(x, y)\right)$ is the partition function. See Appendix A.1 for a complete derivation. Even if we use the MLE estimate r_{ϕ} of the ground-truth reward function r^* , it is still expensive to estimate the partition function $Z(x)$ [19, 15], which makes this representation hard to utilize in practice. However, we can rearrange Eq. 4 to express the reward function in terms of its corresponding optimal policy π_r , the reference policy π_{ref} , and the unknown partition function $Z(\cdot)$. Specifically, we first take the logarithm of both sides of Eq. 4 and then with some algebra we obtain:

$$r(x, y) = \beta \log \frac{\pi_r(y | x)}{\pi_{\text{ref}}(y | x)} + \beta \log Z(x). \quad (5)$$

We can apply this reparameterization to the ground-truth reward r^* and corresponding optimal model π^* . Fortunately, the Bradley-Terry model depends only on the difference of rewards between two completions, i.e., $p^*(y_1 \succ y_2 | x) = \sigma(r^*(x, y_1) - r^*(x, y_2))$. Substituting the reparameterization in Eq. 5 for $r^*(x, y)$ into the preference model Eq. 1, the partition function cancels, and we can express the human preference probability in terms of only the optimal policy π^* and reference policy π_{ref} . Thus, the optimal RLHF policy π^* under the Bradley-Terry model satisfies the preference model:

$$p^*(y_1 \succ y_2 | x) = \frac{1}{1 + \exp\left(\beta \log \frac{\pi^*(y_2 | x)}{\pi_{\text{ref}}(y_2 | x)} - \beta \log \frac{\pi^*(y_1 | x)}{\pi_{\text{ref}}(y_1 | x)}\right)} \quad (6)$$

The derivation is in Appendix A.2. While Eq. 6 uses the Bradley-Terry model, we can similarly derive expressions under the more general Plackett-Luce models [32, 23], shown in Appendix A.3.

Now that we have the probability of human preference data in terms of the optimal policy rather than the reward model, we can formulate a maximum likelihood objective for a parametrized policy π_{θ} . Analogous to the reward modeling approach (i.e. Eq. 2), our policy objective becomes:

$$\mathcal{L}_{\text{DPO}}(\pi_{\theta}; \pi_{\text{ref}}) = -\mathbb{E}_{(x, y_w, y_l) \sim \mathcal{D}} \left[\log \sigma \left(\beta \log \frac{\pi_{\theta}(y_w | x)}{\pi_{\text{ref}}(y_w | x)} - \beta \log \frac{\pi_{\theta}(y_l | x)}{\pi_{\text{ref}}(y_l | x)} \right) \right]. \quad (7)$$

This way, we fit an implicit reward using an alternative parameterization, whose optimal policy is simply π_{θ} . Moreover, since our procedure is equivalent to fitting a reparametrized Bradley-Terry

model, it enjoys certain theoretical properties, such as consistencies under suitable assumption of the preference data distribution [4]. In Section 5, we further discuss theoretical properties of DPO in relation to other works.

What does the DPO update do? For a mechanistic understanding of DPO, it is useful to analyze the gradient of the loss function $\mathcal{L}_{\text{DPO}}$. The gradient with respect to the parameters θ can be written as:

$$\begin{aligned} \nabla_{\theta} \mathcal{L}_{\text{DPO}}(\pi_{\theta}; \pi_{\text{ref}}) = & -\beta \mathbb{E}_{(x, y_w, y_l) \sim \mathcal{D}} \left[\underbrace{\sigma(\hat{r}_{\theta}(x, y_l) - \hat{r}_{\theta}(x, y_w))}_{\text{higher weight when reward estimate is wrong}} \left[\underbrace{\nabla_{\theta} \log \pi(y_w | x)}_{\text{increase likelihood of } y_w} - \underbrace{\nabla_{\theta} \log \pi(y_l | x)}_{\text{decrease likelihood of } y_l} \right] \right], \end{aligned}$$

where $\hat{r}_{\theta}(x, y) = \beta \log \frac{\pi_{\theta}(y|x)}{\pi_{\text{ref}}(y|x)}$ is the reward implicitly defined by the language model π_{θ} and reference model π_{ref} (more in Section 5). Intuitively, the gradient of the loss function $\mathcal{L}_{\text{DPO}}$ increases the likelihood of the preferred completions y_w and decreases the likelihood of dispreferred completions y_l . Importantly, the examples are weighed by how much higher the implicit reward model $\hat{r}_{\theta}$ rates the dispreferred completions, scaled by β , i.e, how incorrectly the implicit reward model orders the completions, accounting for the strength of the KL constraint. Our experiments suggest the importance of this weighting, as a naïve version of this method without the weighting coefficient can cause the language model to degenerate (Appendix Table 3).

DPO outline. The general DPO pipeline is as follows: 1) Sample completions $y_1, y_2 \sim \pi_{\text{ref}}(\cdot | x)$ for every prompt x , label with human preferences to construct the offline dataset of preferences $\mathcal{D} = \{x^{(i)}, y_w^{(i)}, y_l^{(i)}\}_{i=1}^N$ and 2) optimize the language model π_{θ} to minimize $\mathcal{L}_{\text{DPO}}$ for the given π_{ref} and $\mathcal{D}$ and desired β . In practice, one would like to reuse preference datasets publicly available, rather than generating samples and gathering human preferences. Since the preference datasets are sampled using π^{SFT} , we initialize $\pi_{\text{ref}} = \pi^{\text{SFT}}$ whenever available. However, when π^{SFT} is not available, we initialize π_{ref} by maximizing likelihood of preferred completions (x, y_w) , that is, $\pi_{\text{ref}} = \arg \max_{\pi} \mathbb{E}_{x, y_w \sim \mathcal{D}} [\log \pi(y_w | x)]$. This procedure helps mitigate the distribution shift between the true reference distribution which is unavailable, and π_{ref} used by DPO. Further details related to the implementation and hyperparameters can be found in Appendix B.

5 Theoretical Analysis of DPO

In this section, we give further interpretation of the DPO method, provide theoretical backing, and relate advantages of DPO to issues with actor critic algorithms used for RLHF (such as PPO [39]).

5.1 Your Language Model Is Secretly a Reward Model

DPO is able to bypass both fitting an explicit reward and performing RL to learn the policy using a single maximum likelihood objective. Note the optimization objective Eq. 5 is equivalent to a Bradley-Terry model with a reward parameterization $r^*(x, y) = \beta \log \frac{\pi_{\theta}^*(y|x)}{\pi_{\text{ref}}(y|x)}$ and we optimize our parametric model π_{θ} , equivalently to the reward model optimization in Eq. 2 under the change of variables. In this section we will build the theory behind this reparameterization, show that it does not constrain the class of learned reward models, and allows for the exact recovery of the optimal policy. We begin with by defining an equivalence relation between reward functions.

Definition 1. We say that two reward functions $r(x, y)$ and $r'(x, y)$ are equivalent iff $r(x, y) - r'(x, y) = f(x)$ for some function f .

It is easy to see that this is indeed an equivalence relation, which partitions the set of reward functions into classes. We can state the following two lemmas:

Lemma 1. Under the Plackett-Luce, and in particular the Bradley-Terry, preference framework, two reward functions from the same class induce the same preference distribution.

Lemma 2. Two reward functions from the same equivalence class induce the same optimal policy under the constrained RL problem.

The proofs are straightforward and we defer them to Appendix A.5. The first lemma is a well-known under-specification issue with the Plackett-Luce family of models [32]. Due to this under-specification,

we usually have to impose additional identifiability constraints to achieve any guarantees on the MLE estimates from Eq. 2 [4]. The second lemma states that all reward functions from the same class yield the same optimal policy, hence for our final objective, we are only interested in recovering an arbitrary reward function from the optimal class. We prove the following Theorem in Appendix A.6:

Theorem 1. *Under mild assumptions, all reward classes consistent with the Plackett-Luce (and Bradley-Terry in particular) models can be represented with the reparameterization $r(x, y) = \beta \log \frac{\pi(y|x)}{\pi_{\text{ref}}(y|x)}$ for some model $\pi(y|x)$ and a given reference model $\pi_{\text{ref}}(y|x)$.*

Proof Sketch. Consider any reward function $r(x, y)$, which induces a corresponding optimal model $\pi_r(y|x)$, specified by Eq. 4. We will show that a reward function from the equivalence class of r can be represented using the reparameterization given above. We define the projection f as

$$f(r; \pi_{\text{ref}}, \beta)(x, y) = r(x, y) - \beta \log \sum_y \pi_{\text{ref}}(y|x) \exp\left(\frac{1}{\beta} r(x, y)\right) \quad (8)$$

The operator f simply normalizes the reward function with the logarithm of the partition function of π_r . Since the added normalization term is only a function of the prefix x , $f(r; \pi_{\text{ref}}, \beta)(x, y)$ is a reward function in the equivalence class of $r(x, y)$. Finally, replacing r with the RHS of Eq. 5 (which holds for any reward function), we have $f(r; \pi_{\text{ref}}, \beta)(x, y) = \beta \log \frac{\pi_r(y|x)}{\pi_{\text{ref}}(y|x)}$. That is, the projection f produces a member of the equivalence class of r with the desired form, and we do not lose any generality in our reward model from the proposed reparameterization. $\square$

We can alternatively view Theorem 1 as specifying exactly which reward function within each equivalence class the DPO reparameterization selects, that is, the reward function satisfying:

$$\sum_y \underbrace{\pi_{\text{ref}}(y|x) \exp\left(\frac{1}{\beta} r(x, y)\right)}_{=\pi(y|x), \text{ using Thm. 1 reparam.}} = 1, \quad (9)$$

i.e., $\pi(y|x)$ is a valid distribution (probabilities are positive and sum to 1). However, following Eq. 4, we can see that Eq. 9 is the partition function of the optimal policy induced by the reward function $r(x, y)$. The key insight of the DPO algorithm is that we can impose certain constraints on the under-constrained Plackett-Luce (and Bradley-Terry in particular) family of preference models, such that we preserve the class of representable reward models, but explicitly make the optimal policy in Eq. 4 analytically tractable for all prompts x .

5.2 Instability of Actor-Critic Algorithms

We can also use our framework to diagnose instabilities with standard actor-critic algorithms used for the RLHF, such as PPO. We follow the RLHF pipeline and focus on the RL fine-tuning step outlined in Section 3. We can draw connections to the control as inference framework [22] for the constrained RL problem outlined in 3. We assume a parameterized model $\pi_\theta(y|x)$ and minimize $\mathbb{D}_{\text{KL}}[\pi_\theta(y|x) || \pi^*(y|x)]$ where π^* is the optimal policy from Eq. 7 induced by the reward function $r_\phi(y, x)$. With some algebra this leads to the optimization objective:

$$\max_{\pi_\theta} \mathbb{E}_{\pi_\theta(y|x)} \left[\underbrace{r_\phi(x, y) - \beta \log \sum_y \pi_{\text{ref}}(y|x) \exp\left(\frac{1}{\beta} r_\phi(x, y)\right)}_{f(r_\phi, \pi_{\text{ref}}, \beta)} - \underbrace{\beta \log \frac{\pi_\theta(y|x)}{\pi_{\text{ref}}(y|x)}}_{\text{KL}} \right] \quad (10)$$

This is the same objective optimized in prior works [51, 40, 1, 28] using the DPO-equivalent reward for the reward class of r_ϕ . In this setting, we can interpret the normalization term in $f(r_\phi, \pi_{\text{ref}}, \beta)$ as the soft value function of the reference policy π_{ref} . While this term does not affect the optimal solution, without it, the policy gradient of the objective could have high variance, making learning unstable. We can accommodate for the normalization term using a learned value function, but that can also be difficult to optimize. Alternatively, prior works have normalized rewards using a human completion baseline, essentially a single sample Monte-Carlo estimate of the normalizing term. In contrast the DPO reparameterization yields a reward function that does not require any baselines.

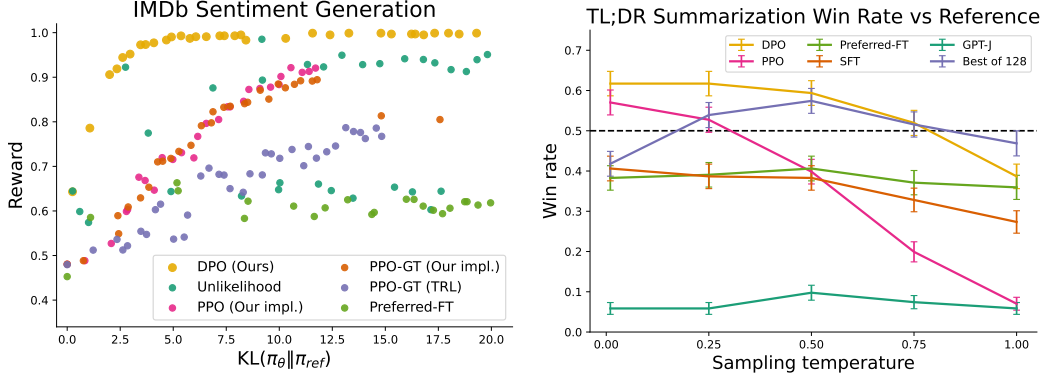


Figure 2: **Left.** The frontier of expected reward vs KL to the reference policy. DPO provides the highest expected reward for all KL values, demonstrating the quality of the optimization. **Right.** TL;DR summarization win rates vs. human-written summaries, using GPT-4 as evaluator. DPO exceeds PPO’s best-case performance on summarization, while being more robust to changes in the sampling temperature.

6 Experiments

In this section, we empirically evaluate DPO’s ability to **train policies directly from preferences**. First, in a well-controlled text-generation setting, we ask: how efficiently does **DPO trade off maximizing reward and minimizing KL-divergence** with the reference policy, compared to common preference learning algorithms such as PPO? Next, we evaluate **DPO’s performance on larger models** and more difficult RLHF tasks, including summarization and dialogue. We find that with almost **no tuning of hyperparameters**, DPO tends to perform as well or better than **strong baselines like RLHF with PPO** as well as returning the best of N sampled trajectories under a **learned reward function**. Before presenting these results, we describe the experimental set-up; additional details are in Appendix C.

Tasks. Our experiments explore three different **open-ended text generation tasks**. For all experiments, algorithms learn a policy from a dataset of preferences $\mathcal{D} = \{x^{(i)}, y_w^{(i)}, y_l^{(i)}\}_{i=1}^N$. In **controlled sentiment generation**, x is a prefix of a movie review from the IMDb dataset [24], and the policy must generate y with positive sentiment. In order to perform a **controlled evaluation**, for this experiment we **generate preference pairs** over generations using a **pre-trained sentiment classifier**, where $p(\text{positive} | x, y_w) > p(\text{positive} | x, y_l)$. For SFT, we fine-tune GPT-2-large until convergence on reviews from the train split of the IMDB dataset (further details in App C.1). In **summarization**, x is a **forum post** from Reddit; the policy must generate a **summary y of the main points** in the post. Following prior work, we use the Reddit TL;DR summarization dataset [43] along with human preferences gathered by Stiennon et al.. We use an **SFT model fine-tuned on human-written forum post summaries**² with the TRLX [44] framework for RLHF. The human preference dataset was gathered by Stiennon et al. on samples from a different, but similarly-trained, SFT model. Finally, in **single-turn dialogue**, x is a human query, which may be anything from a **question about astrophysics** to a request for **relationship advice**. A policy must produce an **engaging and helpful response y** to a user’s query; we use the Anthropic Helpful and Harmless dialogue dataset [1], containing 170k dialogues between a human and an automated assistant. Each transcript ends with a **pair of responses** generated by a large (although unknown) language model along with a preference label denoting the **human-preferred response**. In this setting, **no pre-trained SFT model** is available; we therefore **fine-tune an off-the-shelf language model** on only the preferred completions to form the SFT model.

Evaluation. Our experiments use two different approaches to evaluation. In order to analyze the effectiveness of each algorithm in **optimizing the constrained reward maximization objective**, in the **controlled sentiment generation setting** we evaluate each algorithm by its frontier of achieved reward and **KL-divergence from the reference policy**; this frontier is computable because we have access to the **ground-truth reward function (a sentiment classifier)**. However, in the real world, the ground truth reward function is not known; therefore, we evaluate algorithms with their **win rate** against a **baseline policy**, using GPT-4 as a proxy for **human evaluation of summary quality and response helpfulness** in the **summarization and single-turn dialogue settings**, respectively. For summarization, we use **reference summaries** in the test set as the **baseline**; for dialogue, we use the **preferred response** in the

²https://huggingface.co/CarperAI/openai_summarize_tldr_sft

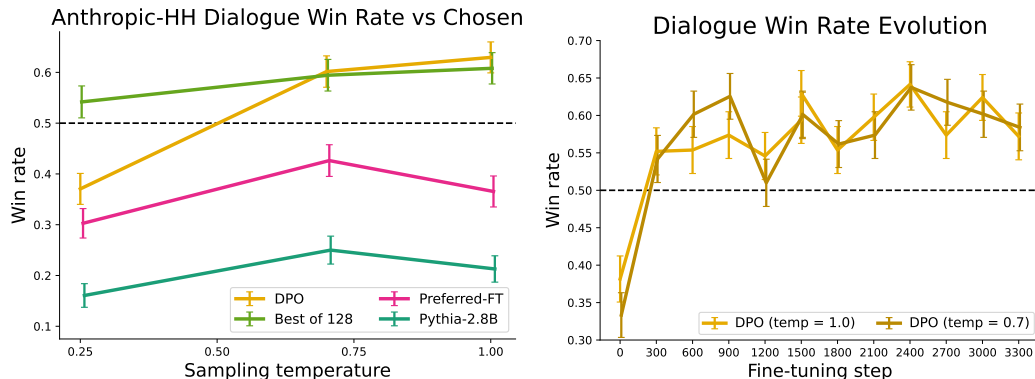


Figure 3: **Left.** Win rates computed by GPT-4 for Anthropic-HH one-step dialogue; DPO is the only method that improves over chosen summaries in the Anthropic-HH test set. **Right.** Win rates for different sampling temperatures over the course of training. DPO’s improvement over the dataset labels is fairly stable over the course of training for different sampling temperatures.

test dataset as the baseline. While existing studies suggest LMs can be better automated evaluators than existing metrics [10], we conduct a human study to justify our usage of GPT-4 for evaluation in Sec. 6.4. We find GPT-4 judgments **correlate strongly with humans**, with human agreement with GPT-4 typically similar or higher than **inter-human annotator agreement**.

Methods. In addition to DPO, we evaluate several existing approaches to training language models to adhere to human preferences. Most simply, we explore **zero-shot prompting** with **GPT-J** [45] in the **summarization** task and **2-shot prompting** with **Pythia-2.8B** [3] in the dialogue task. In addition, we evaluate the **SFT** model as well as **Preferred-FT**, which is a **model fine-tuned with supervised learning** on the **chosen completion y_w** from either the SFT model (in controlled sentiment and summarization) or a generic LM (in single-turn dialogue). Another **pseudo-supervised method** is **Unlikelihood** [46], which simply optimizes the policy to **maximize the probability assigned to y_w** and **minimize the probability assigned to y_l** ; we use an optional **coefficient $\alpha \in [0, 1]$** on the ‘unlikelihood’ term. We also consider **PPO** [39] using a reward function learned from the preference data and **PPO-GT**, which is an oracle that learns from the ground truth reward function available in the controlled sentiment setting. In our sentiment experiments, we use two implementations of PPO-GT, one of-the-shelf version [44] as well as a modified version that **normalizes rewards** and further **tunes hyperparameters** to improve performance (we also use these modifications when running ‘normal’ PPO with learned rewards). Finally, we consider the **Best of N** baseline, sampling N responses from the SFT model (or Preferred-FT in dialogue) and returning the **highest-scoring response** according to a **reward function learned from the preference dataset**. This high-performing method decouples the quality of the reward model from the PPO optimization, but is computationally impractical even for moderate N as it requires sampling N completions for every query at test time.

6.1 How well can DPO optimize the RLHF objective?

The KL-constrained reward maximization objective used in typical RLHF algorithms **balances exploitation of reward** while restricting the policy from **deviating far from the reference policy**. Therefore, when comparing algorithms, we must take into account both reward achieved as well as the **KL discrepancy**; achieving slightly higher reward but with much higher KL is not necessarily desirable. Figure 2 shows the reward-KL frontier for various algorithms in the sentiment setting. We execute multiple training runs for each algorithm, using a different **hyperparameter for policy conservativeness** in each run (target **KL $\in \{3, 6, 9, 12\}$** for PPO, $\beta \in \{0.05, 0.1, 1, 5\}$, $\alpha \in \{0.05, 0.1, 0.5, 1\}$ for unlikelihood, random seeds for preferred-FT). This sweep includes **22 runs** in total. After each **100 training steps until convergence**, we evaluate each policy on a set of test prompts, computing the average reward under the true reward function as well as the **average sequence-level KL³** with the reference policy $\text{KL}(\pi \parallel \pi_{\text{ref}})$. We find that DPO produces by far the most efficient frontier, achieving the highest reward while still achieving low KL. This result is particularly notable for multiple reasons. First, DPO and PPO optimize the same objective, but DPO is notably more efficient;

³That is, the sum of the per-timestep KL-divergences.

DPO’s reward/KL tradeoff strictly dominates PPO. Second, DPO achieves a better frontier than PPO, even when PPO can access ground truth rewards (PPO-GT).

6.2 Can DPO scale to real preference datasets?

Next, we evaluate fine-tuning performance of DPO on summarization and single-turn dialogue. For summarization, automatic evaluation metrics such as ROUGE can be poorly correlated with human preferences [40], and prior work has found that fine-tuning LMs using PPO on human preferences to provide more effective summaries. We evaluate different methods by sampling completions on the test split of TL;DR summarization dataset, and computing the average win rate against reference completions in the test set. The completions for all methods are sampled at temperatures varying from 0.0 to 1.0, and the win rates are shown in Figure 2 (right). DPO, PPO and Preferred-FT all fine-tune the same GPT-J SFT model⁴. We find that DPO has a win rate of approximately 61% at a temperature of 0.0, exceeding the performance of PPO at 57% at its optimal sampling temperature of 0.0. DPO also achieves a higher maximum win rate compared to the best of N baseline. We note that we did not meaningfully tune DPO’s β hyperparameter, so these results may underestimate DPO’s potential. Moreover, we find DPO to be much more robust to the sampling temperature than PPO, the performance of which can degrade to that of the base GPT-J model at high temperatures. Preferred-FT does not improve significantly over the SFT model. We also compare DPO and PPO head-to-head in human evaluations in Section 6.4, where DPO samples at temperature 0.25 were preferred 58% times over PPO samples at temperature 0.

On single-turn dialogue, we evaluate the different methods on the subset of the test split of the Anthropic HH dataset [1] with one step of human-assistant interaction. GPT-4 evaluations use the preferred completions on the test as the reference to compute the win rate for different methods. As there is no standard SFT model for this task, we start with a pre-trained Pythia-2.8B, use Preferred-FT to train a reference model on the chosen completions such that completions are within distribution of the model, and then train using DPO. We also compare against the best of 128 Preferred-FT completions (we found the Best of N baseline plateaus at 128 completions for this task; see Appendix Figure 4) and a 2-shot prompted version of the Pythia-2.8B base model, finding DPO performs as well or better for the best-performing temperatures for each method. We also evaluate an RLHF model trained with PPO on the Anthropic HH dataset⁵ from a well-known source⁶, but are unable to find a prompt or sampling temperature that gives performance better than the base Pythia-2.8B model. Based on our results from TL;DR and the fact that both methods optimize the same reward function, we consider Best of 128 a rough proxy for PPO-level performance. Overall, DPO is the only computationally efficient method that improves over the preferred completions in the Anthropic HH dataset, and provides similar or better performance to the computationally demanding Best of 128 baseline. Finally, Figure 3 shows that DPO converges to its best performance relatively quickly.

6.3 Generalization to a new input distribution

To further compare the performance of PPO and DPO under distribution shifts, we evaluate the PPO and DPO policies from our Reddit TL;DR summarization experiment on a different distribution, news articles in the test split of the CNN/DailyMail dataset [26], using the best sampling temperatures from TL;DR (0 and 0.25). The results are presented in Table 1. We computed the GPT-4 win rate against the ground-truth summaries in the datasets, using the same GPT-4 (C) prompt we used for Reddit TL;DR, but replacing the words “forum post” with “news article”. For this new distribution, DPO continues to outperform the PPO policy by a significant margin. This experiment provides initial evidence that DPO policies can generalize similarly well to PPO policies, even though DPO does not use the additional unlabeled Reddit TL;DR prompts that PPO uses.

Alg.	Win rate vs. ground truth	
	Temp 0	Temp 0.25
DPO	0.36	0.31
PPO	0.26	0.23

Table 1: GPT-4 win rates vs. ground truth summaries for out-of-distribution CNN/DailyMail input articles.

⁴https://huggingface.co/CarperAI/openai_summarize_tldr_sft

⁵https://huggingface.co/recipecate/ppo_hh_pythia-6B

⁶<https://github.com/CarperAI/trlx/tree/main/examples/hh>

6.4 Validating GPT-4 judgments with human judgments

We conduct a human study to verify the reliability of GPT-4’s judgments, using the results of the TL;DR summarization experiment and two different GPT-4 prompts. The **GPT-4 (S)** (simple) prompt simply asks for which summary better-summarizes the important information in the post. The **GPT-4 (C)** (concise) prompt also asks for which summary is more concise; we evaluate this prompt because we find that GPT-4 prefers longer, more repetitive summaries than humans do with the **GPT-4 (S)** prompt. See Appendix C.2 for the complete prompts. We perform three comparisons, using the highest (DPO, temp. 0.25), the lowest (PPO, temp. 1.0), and a middle-performing (SFT, temp. 0.25) method with the aim of covering a diversity of sample qualities; all three methods are compared against greedily-sampled PPO (its best-performing temperature). We find that with both prompts, GPT-4 tends to agree with humans about as often as humans agree with each other, suggesting that GPT-4 is a reasonable proxy for human evaluations (due to limited human raters, we only collect multiple human judgments for the DPO and PPO-1 comparisons). Overall, the **GPT-4 (C)** prompt generally provides win rates more representative of humans; we therefore use this prompt for the main results in Section 6.2. For additional details about the human study, including the web interface presented to raters and the list of human volunteers, see Appendix D.3.

	DPO	SFT	PPO-1
N respondents	272	122	199
GPT-4 (S) win %	47	27	13
GPT-4 (C) win %	54	32	12
Human win %	58	43	17
GPT-4 (S)-H agree	70	77	86
GPT-4 (C)-H agree	67	79	85
H-H agree	65	-	87

Table 2: Comparing human and GPT-4 win rates and per-judgment agreement on TL;DR summarization samples. **Humans agree with GPT-4 about as much as they agree with each other.** Each experiment compares a summary from the stated method with a summary from PPO with temperature 0.

7 Discussion

Learning from preferences is a powerful, scalable framework for training capable, aligned language models. We have introduced DPO, a simple training paradigm for training language models from preferences without reinforcement learning. Rather than coercing the preference learning problem into a standard RL setting in order to use off-the-shelf RL algorithms, DPO identifies a mapping between language model policies and reward functions that enables training a language model to satisfy human preferences directly, with a simple cross-entropy loss, without reinforcement learning or loss of generality. With virtually no tuning of hyperparameters, DPO performs similarly or better than existing RLHF algorithms, including those based on PPO; DPO thus meaningfully reduces the barrier to training more language models from human preferences.

Limitations & Future Work. Our results raise several important questions for future work. How does the DPO policy generalize out of distribution, compared with learning from an explicit reward function? Our initial results suggest that DPO policies can generalize similarly to PPO-based models, but more comprehensive study is needed. For example, can training with self-labeling from the DPO policy similarly make effective use of unlabeled prompts? On another front, how does reward over-optimization manifest in the direct preference optimization setting, and is the slight decrease in performance in Figure 3-right an instance of it? Additionally, while we evaluate models up to 6B parameters, exploration of scaling DPO to state-of-the-art models orders of magnitude larger is an exciting direction for future work. Regarding evaluations, we find that the win rates computed by GPT-4 are impacted by the prompt; future work may study the best way to elicit high-quality judgments from automated systems. Finally, many possible applications of DPO exist beyond training language models from human preferences, including training generative models in other modalities.

Acknowledgements

EM gratefully acknowledges funding from a Knight-Hennessy Graduate Fellowship. CF and CM are CIFAR Fellows. This work was supported in part by the Stanford Accelerator for Learning (SAL) and Stanford Institute for Human-Centered Artificial Intelligence (HAI) *Generative AI for the Future of Learning* seed grant program. The Stanford Center for Research on Foundation Models (CRFM) provided part of the compute resources used for the experiments in this work. This work was supported in part by ONR grant N00014-20-1-2675.

References

- [1] Y. Bai, A. Jones, K. Ndousse, A. Askell, A. Chen, N. DasSarma, D. Drain, S. Fort, D. Ganguli, T. Henighan, N. Joseph, S. Kadavath, J. Kernion, T. Conerly, S. El-Showk, N. Elhage, Z. Hatfield-Dodds, D. Hernandez, T. Hume, S. Johnston, S. Kravec, L. Lovitt, N. Nanda, C. Olsson, D. Amodei, T. Brown, J. Clark, S. McCandlish, C. Olah, B. Mann, and J. Kaplan. Training a helpful and harmless assistant with reinforcement learning from human feedback, 2022.
- [2] Y. Bai, S. Kadavath, S. Kundu, A. Askell, J. Kernion, A. Jones, A. Chen, A. Goldie, A. Mirhoseini, C. McKinnon, C. Chen, C. Olsson, C. Olah, D. Hernandez, D. Drain, D. Ganguli, D. Li, E. Tran-Johnson, E. Perez, J. Kerr, J. Mueller, J. Ladish, J. Landau, K. Ndousse, K. Lukosuite, L. Lovitt, M. Sellitto, N. Elhage, N. Schiefer, N. Mercado, N. DasSarma, R. Lasenby, R. Larson, S. Ringer, S. Johnston, S. Kravec, S. E. Showk, S. Fort, T. Lanham, T. Telleen-Lawton, T. Conerly, T. Henighan, T. Hume, S. R. Bowman, Z. Hatfield-Dodds, B. Mann, D. Amodei, N. Joseph, S. McCandlish, T. Brown, and J. Kaplan. Constitutional ai: Harmlessness from ai feedback, 2022.
- [3] S. Biderman, H. Schoelkopf, Q. Anthony, H. Bradley, K. O’Brien, E. Hallahan, M. A. Khan, S. Purohit, U. S. Prashanth, E. Raff, A. Skowron, L. Sutawika, and O. van der Wal. Pythia: A suite for analyzing large language models across training and scaling, 2023.
- [4] H. Bong and A. Rinaldo. Generalized results for the existence and consistency of the MLE in the Bradley-Terry-Luce model. *International Conference on Machine Learning*, 2022. arXiv:2110.11487.
- [5] R. A. Bradley and M. E. Terry. Rank analysis of incomplete block designs: I. the method of paired comparisons. *Biometrika*, 39(3/4):324–345, 1952. doi: <https://doi.org/10.2307/2334029>.
- [6] T. Brown, B. Mann, N. Ryder, M. Subbiah, J. D. Kaplan, P. Dhariwal, A. Neelakantan, P. Shyam, G. Sastry, A. Askell, S. Agarwal, A. Herbert-Voss, G. Krueger, T. Henighan, R. Child, A. Ramesh, D. Ziegler, J. Wu, C. Winter, C. Hesse, M. Chen, E. Sigler, M. Litwin, S. Gray, B. Chess, J. Clark, C. Berner, S. McCandlish, A. Radford, I. Sutskever, and D. Amodei. Language models are few-shot learners. In H. Larochelle, M. Ranzato, R. Hadsell, M. Balcan, and H. Lin, editors, *Advances in Neural Information Processing Systems*, volume 33, pages 1877–1901. Curran Associates, Inc., 2020. URL https://proceedings.neurips.cc/paper_files/paper/2020/file/1457c0d6bfc4967418bfb8ac142f64a-Paper.pdf.
- [7] T. Brown, B. Mann, N. Ryder, M. Subbiah, J. D. Kaplan, P. Dhariwal, A. Neelakantan, P. Shyam, G. Sastry, A. Askell, et al. Language models are few-shot learners. *Advances in neural information processing systems*, 33:1877–1901, 2020.
- [8] S. Bubeck, V. Chandrasekaran, R. Eldan, J. Gehrke, E. Horvitz, E. Kamar, P. Lee, Y. T. Lee, Y. Li, S. Lundberg, H. Nori, H. Palangi, M. T. Ribeiro, and Y. Zhang. Sparks of artificial general intelligence: Early experiments with GPT-4, 2023. arXiv preprint arXiv:2303.12712.
- [9] R. Busa-Fekete, B. Szörényi, P. Weng, W. Cheng, and E. Hüllermeier. Preference-based reinforcement learning: evolutionary direct policy search using a preference-based racing algorithm. *Machine Learning*, 97(3):327–351, July 2014. doi: 10.1007/s10994-014-5458-8. URL <https://doi.org/10.1007/s10994-014-5458-8>.
- [10] Y. Chen, R. Wang, H. Jiang, S. Shi, and R.-L. Xu. Exploring the use of large language models for reference-free text quality evaluation: A preliminary empirical study. *ArXiv*, abs/2304.00723, 2023.
- [11] A. Chowdhery, S. Narang, J. Devlin, M. Bosma, G. Mishra, A. Roberts, P. Barham, H. W. Chung, C. Sutton, S. Gehrmann, et al. Palm: Scaling language modeling with pathways. *arXiv preprint arXiv:2204.02311*, 2022.
- [12] P. F. Christiano, J. Leike, T. Brown, M. Martic, S. Legg, and D. Amodei. Deep reinforcement learning from human preferences. In I. Guyon, U. V. Luxburg, S. Bengio, H. Wallach, R. Fergus, S. Vishwanathan, and R. Garnett, editors, *Advances in Neural Information Processing Systems*, volume 30. Curran Associates, Inc., 2017. URL https://proceedings.neurips.cc/paper_files/paper/2017/file/d5e2c0adad503c91f91df240d0cd4e49-Paper.pdf.

- [13] H. W. Chung, L. Hou, S. Longpre, B. Zoph, Y. Tay, W. Fedus, Y. Li, X. Wang, M. Dehghani, S. Brahma, A. Webson, S. S. Gu, Z. Dai, M. Suzgun, X. Chen, A. Chowdhery, A. Castro-Ros, M. Pellat, K. Robinson, D. Valter, S. Narang, G. Mishra, A. Yu, V. Zhao, Y. Huang, A. Dai, H. Yu, S. Petrov, E. H. Chi, J. Dean, J. Devlin, A. Roberts, D. Zhou, Q. V. Le, and J. Wei. Scaling instruction-finetuned language models, 2022.
- [14] M. Dudík, K. Hofmann, R. E. Schapire, A. Slivkins, and M. Zoghi. Contextual dueling bandits. In P. Grünwald, E. Hazan, and S. Kale, editors, *Proceedings of The 28th Conference on Learning Theory*, volume 40 of *Proceedings of Machine Learning Research*, pages 563–587, Paris, France, 03–06 Jul 2015. PMLR. URL <https://proceedings.mlr.press/v40/Dudik15.html>.
- [15] D. Go, T. Korbak, G. Kruszewski, J. Rozen, N. Ryu, and M. Dymetman. Aligning language models with preferences through f-divergence minimization. In *Proceedings of the 40th International Conference on Machine Learning, ICML’23*. JMLR.org, 2023.
- [16] A. Jain, B. Wojcik, T. Joachims, and A. Saxena. Learning trajectory preferences for manipulators via iterative improvement. In C. Burges, L. Bottou, M. Welling, Z. Ghahramani, and K. Weinberger, editors, *Advances in Neural Information Processing Systems*, volume 26. Curran Associates, Inc., 2013. URL https://proceedings.neurips.cc/paper_files/paper/2013/file/c058f544c737782deacefa532d9add4c-Paper.pdf.
- [17] N. Jaques, S. Gu, D. Bahdanau, J. M. Hernández-Lobato, R. E. Turner, and D. Eck. Sequence tutor: Conservative fine-tuning of sequence generation models with kl-control. In *International Conference on Machine Learning*, pages 1645–1654. PMLR, 2017.
- [18] N. Jaques, J. H. Shen, A. Ghandeharioun, C. Ferguson, A. Lapedriza, N. Jones, S. S. Gu, and R. Picard. Human-centric dialog training via offline reinforcement learning. *arXiv preprint arXiv:2010.05848*, 2020.
- [19] T. Korbak, H. Elsahar, G. Kruszewski, and M. Dymetman. On reinforcement learning and distribution matching for fine-tuning language models with no catastrophic forgetting. In S. Koyejo, S. Mohamed, A. Agarwal, D. Belgrave, K. Cho, and A. Oh, editors, *Advances in Neural Information Processing Systems*, volume 35, pages 16203–16220. Curran Associates, Inc., 2022. URL https://proceedings.neurips.cc/paper_files/paper/2022/file/67496dfa96afddab795530cc7c69b57a-Paper-Conference.pdf.
- [20] J. Kreutzer, J. Uyheng, and S. Riezler. Reliability and learnability of human bandit feedback for sequence-to-sequence reinforcement learning. In *Proceedings of the 56th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 1777–1788, Melbourne, Australia, July 2018. Association for Computational Linguistics. doi: 10.18653/v1/P18-1165. URL <https://aclanthology.org/P18-1165>.
- [21] A. Kupcsik, D. Hsu, and W. S. Lee. *Learning Dynamic Robot-to-Human Object Handover from Human Feedback*, pages 161–176. Springer International Publishing, 01 2018. ISBN 978-3-319-51531-1. doi: 10.1007/978-3-319-51532-8_10.
- [22] S. Levine. Reinforcement learning and control as probabilistic inference: Tutorial and review, 2018.
- [23] R. D. Luce. Individual choice behavior: A theoretical analysis. *Courier Corporation*, 2012.
- [24] A. L. Maas, R. E. Daly, P. T. Pham, D. Huang, A. Y. Ng, and C. Potts. Learning word vectors for sentiment analysis. In *Proceedings of the 49th Annual Meeting of the Association for Computational Linguistics: Human Language Technologies*, pages 142–150, Portland, Oregon, USA, June 2011. Association for Computational Linguistics. URL <http://www.aclweb.org/anthology/P11-1015>.
- [25] S. Mishra, D. Khashabi, C. Baral, and H. Hajishirzi. Cross-task generalization via natural language crowdsourcing instructions. In *Proceedings of the 60th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 3470–3487, Dublin, Ireland, May 2022. Association for Computational Linguistics. doi: 10.18653/v1/2022.acl-long.244. URL <https://aclanthology.org/2022.acl-long.244>.

- [26] R. Nallapati, B. Zhou, C. dos Santos, Ç. Gulçehre, and B. Xiang. Abstractive text summarization using sequence-to-sequence RNNs and beyond. In *Proceedings of the 20th SIGNLL Conference on Computational Natural Language Learning*, pages 280–290, Berlin, Germany, Aug. 2016. Association for Computational Linguistics. doi: 10.18653/v1/K16-1028. URL <https://aclanthology.org/K16-1028>.
- [27] D. Narayanan, M. Shoeybi, J. Casper, P. LeGresley, M. Patwary, V. Korthikanti, D. Vainbrand, P. Kashinkunti, J. Bernauer, B. Catanzaro, A. Phanishayee, and M. Zaharia. Efficient large-scale language model training on gpu clusters using megatron-lm. In *Proceedings of the International Conference for High Performance Computing, Networking, Storage and Analysis, SC '21*, New York, NY, USA, 2021. Association for Computing Machinery. ISBN 9781450384421. doi: 10.1145/3458817.3476209. URL <https://doi.org/10.1145/3458817.3476209>.
- [28] L. Ouyang, J. Wu, X. Jiang, D. Almeida, C. Wainwright, P. Mishkin, C. Zhang, S. Agarwal, K. Slama, A. Ray, J. Schulman, J. Hilton, F. Kelton, L. Miller, M. Simens, A. Askell, P. Welinder, P. F. Christiano, J. Leike, and R. Lowe. Training language models to follow instructions with human feedback. In S. Koyejo, S. Mohamed, A. Agarwal, D. Belgrave, K. Cho, and A. Oh, editors, *Advances in Neural Information Processing Systems*, volume 35, pages 27730–27744. Curran Associates, Inc., 2022. URL https://proceedings.neurips.cc/paper_files/paper/2022/file/b1efde53be364a73914f58805a001731-Paper-Conference.pdf.
- [29] R. Paulus, C. Xiong, and R. Socher. A deep reinforced model for abstractive summarization. In *International Conference on Learning Representations*, 2018. URL <https://openreview.net/forum?id=HkAClQgA->.
- [30] X. B. Peng, A. Kumar, G. Zhang, and S. Levine. Advantage-weighted regression: Simple and scalable off-policy reinforcement learning. *arXiv preprint arXiv:1910.00177*, 2019.
- [31] J. Peters and S. Schaal. Reinforcement learning by reward-weighted regression for operational space control. In *Proceedings of the 24th international conference on Machine learning*, pages 745–750, 2007.
- [32] R. L. Plackett. The analysis of permutations. *Journal of the Royal Statistical Society. Series C (Applied Statistics)*, 24(2):193–202, 1975. doi: <https://doi.org/10.2307/2346567>.
- [33] A. Radford, J. Wu, R. Child, D. Luan, D. Amodei, and I. Sutskever. Language models are unsupervised multitask learners, 2019. Ms., OpenAI.
- [34] R. Ramamurthy, P. Ammanabrolu, K. Brantley, J. Hessel, R. Sifa, C. Bauckhage, H. Hajishirzi, and Y. Choi. Is reinforcement learning (not) for natural language processing: Benchmarks, baselines, and building blocks for natural language policy optimization. In *The Eleventh International Conference on Learning Representations*, 2023. URL <https://openreview.net/forum?id=8aHzds2uUyB>.
- [35] M. Ranzato, S. Chopra, M. Auli, and W. Zaremba. Sequence level training with recurrent neural networks. *CoRR*, abs/1511.06732, 2015.
- [36] D. Sadigh, A. D. Dragan, S. Sastry, and S. A. Seshia. Active preference-based learning of reward functions. In *Robotics: Science and Systems (RSS)*, 2017.
- [37] A. Saha, A. Pacchiano, and J. Lee. Dueling rl: Reinforcement learning with trajectory preferences. In F. Ruiz, J. Dy, and J.-W. van de Meent, editors, *Proceedings of The 26th International Conference on Artificial Intelligence and Statistics*, volume 206 of *Proceedings of Machine Learning Research*, pages 6263–6289. PMLR, 25–27 Apr 2023. URL <https://proceedings.mlr.press/v206/saha23a.html>.
- [38] V. Sanh, A. Webson, C. Raffel, S. Bach, L. Sutawika, Z. Alyafeai, A. Chaffin, A. Stiegler, A. Raja, M. Dey, M. S. Bari, C. Xu, U. Thakker, S. S. Sharma, E. Szczechla, T. Kim, G. Chhablani, N. Nayak, D. Datta, J. Chang, M. T.-J. Jiang, H. Wang, M. Manica, S. Shen, Z. X. Yong, H. Pandey, R. Bawden, T. Wang, T. Neeraj, J. Rozen, A. Sharma, A. Santilli, T. Fevry, J. A. Fries, R. Teehan, T. L. Scao, S. Biderman, L. Gao, T. Wolf, and A. M. Rush. Multi-task prompted training enables zero-shot task generalization. In *International Conference on Learning Representations*, 2022. URL <https://openreview.net/forum?id=9Vrb9DOWI4>.

- [39] J. Schulman, F. Wolski, P. Dhariwal, A. Radford, and O. Klimov. Proximal policy optimization algorithms, 2017.
- [40] N. Stiennon, L. Ouyang, J. Wu, D. M. Ziegler, R. Lowe, C. Voss, A. Radford, D. Amodei, and P. Christiano. Learning to summarize from human feedback, 2022.
- [41] R. Thoppilan, D. D. Freitas, J. Hall, N. Shazeer, A. Kulshreshtha, H.-T. Cheng, A. Jin, T. Bos, L. Baker, Y. Du, Y. Li, H. Lee, H. S. Zheng, A. Ghafouri, M. Menegali, Y. Huang, M. Krikun, D. Lepikhin, J. Qin, D. Chen, Y. Xu, Z. Chen, A. Roberts, M. Bosma, V. Zhao, Y. Zhou, C.-C. Chang, I. Krivokon, W. Rusch, M. Pickett, P. Srinivasan, L. Man, K. Meier-Hellstern, M. R. Morris, T. Doshi, R. D. Santos, T. Duke, J. Soraker, B. Zevenbergen, V. Prabhakaran, M. Diaz, B. Hutchinson, K. Olson, A. Molina, E. Hoffman-John, J. Lee, L. Aroyo, R. Rajakumar, A. Butryna, M. Lamm, V. Kuzmina, J. Fenton, A. Cohen, R. Bernstein, R. Kurzweil, B. Aguera-Arcas, C. Cui, M. Croak, E. Chi, and Q. Le. Lamda: Language models for dialog applications, 2022.
- [42] H. Touvron, T. Lavril, G. Izacard, X. Martinet, M.-A. Lachaux, T. Lacroix, B. Rozière, N. Goyal, E. Hambro, F. Azhar, et al. Llama: Open and efficient foundation language models. *arXiv preprint arXiv:2302.13971*, 2023.
- [43] M. Völske, M. Potthast, S. Syed, and B. Stein. TL;DR: Mining Reddit to learn automatic summarization. In *Proceedings of the Workshop on New Frontiers in Summarization*, pages 59–63, Copenhagen, Denmark, Sept. 2017. Association for Computational Linguistics. doi: 10.18653/v1/W17-4508. URL <https://aclanthology.org/W17-4508>.
- [44] L. von Werra, J. Tow, reciprocated, S. Matiana, A. Havrilla, cat state, L. Castricato, Alan, D. V. Phung, A. Thakur, A. Bukhtiyarov, aaronmm, F. Milo, Daniel, D. King, D. Shin, E. Kim, J. Wei, M. Romero, N. Pochinkov, O. Sanseviero, R. Adithyan, S. Siu, T. Simonini, V. Blagojevic, X. Song, Z. Witten, alexandremuzio, and crumb. CarperAI/trlx: v0.6.0: LLaMa (Alpaca), Benchmark Util, T5 ILQL, Tests, Mar. 2023. URL <https://doi.org/10.5281/zenodo.7790115>.
- [45] B. Wang and A. Komatsuzaki. GPT-J-6B: A 6 Billion Parameter Autoregressive Language Model. <https://github.com/kingoflolz/mesh-transformer-jax>, May 2021.
- [46] S. Welleck, I. Kulikov, S. Roller, E. Dinan, K. Cho, and J. Weston. Neural text generation with unlikelihood training. *arXiv preprint arXiv:1908.04319*, 2019.
- [47] R. J. Williams. Simple statistical gradient-following algorithms for connectionist reinforcement learning. *Mach. Learn.*, 8(3–4):229–256, may 1992. ISSN 0885-6125. doi: 10.1007/BF00992696. URL <https://doi.org/10.1007/BF00992696>.
- [48] Y. Wu and B. Hu. Learning to extract coherent summary via deep reinforcement learning. In *Proceedings of the Thirty-Second AAAI Conference on Artificial Intelligence and Thirtieth Innovative Applications of Artificial Intelligence Conference and Eighth AAAI Symposium on Educational Advances in Artificial Intelligence*, AAAI’18/IAAI’18/EAAI’18. AAAI Press, 2018. ISBN 978-1-57735-800-8.
- [49] X. Yan, C. Luo, C. L. A. Clarke, N. Craswell, E. M. Voorhees, and P. Castells. Human preferences as dueling bandits. In *Proceedings of the 45th International ACM SIGIR Conference on Research and Development in Information Retrieval*, SIGIR ’22, page 567–577, New York, NY, USA, 2022. Association for Computing Machinery. ISBN 9781450387323. doi: 10.1145/3477495.3531991. URL <https://doi.org/10.1145/3477495.3531991>.
- [50] Y. Yue, J. Broder, R. Kleinberg, and T. Joachims. The k-armed dueling bandits problem. *Journal of Computer and System Sciences*, 78(5):1538–1556, 2012. ISSN 0022-0000. doi: <https://doi.org/10.1016/j.jcss.2011.12.028>. URL <https://www.sciencedirect.com/science/article/pii/S0022000012000281>. JCSS Special Issue: Cloud Computing 2011.
- [51] D. M. Ziegler, N. Stiennon, J. Wu, T. B. Brown, A. Radford, D. Amodei, P. Christiano, and G. Irving. Fine-tuning language models from human preferences, 2020.

Author Contributions

All authors provided valuable contributions to designing, analyzing, and iterating on experiments, writing and editing the paper, and generally managing the project’s progress.

RR proposed using autoregressive reward models in discussions with **EM**; derived the DPO objective; proved the theoretical properties of the algorithm and wrote the relevant sections and appendices. He also suggested and helped with organizing experiments and contributed some of the PPO and reward learning baselines.

AS initiated the discussion on using weighted regression methods as an alternative to PPO; initiated project-related organization, wrote initial analysis connecting DPO with weighted regression and unlikelihood; design and iterations of DPO + baseline implementations, initial exploratory experiments for DPO; substantial experiment organization and design (datasets, baselines, evaluation); led model training and evaluation for controlled sentiment generation and summarization; design iterations for GPT-4 evaluation (particularly summarization); substantial writing contributions to abstract, prelims/method and experiments; editing contributions to other sections.

EM provided input on early discussions on learning autoregressive reward functions; wrote the first implementation of DPO and ran the first DPO experiments; trained the large-scale (summarization and dialogue) DPO models used in paper experiments; conducted initial GPT-4 win rate evaluations and set up related infrastructure; recruited participants for, conducted, and analyzed results from the human study; wrote the abstract, introduction, related work, discussion, and most of experiments; and assisted with editing the rest of the paper.

CF, CM, & SE supervised the research, suggested ideas and experiments, and assisted in writing the paper.

A Mathematical Derivations

A.1 Deriving the Optimum of the KL-Constrained Reward Maximization Objective

In this appendix, we will derive Eq. 4. Analogously to Eq. 3, we optimize the following objective:

$$\max_{\pi} \mathbb{E}_{x \sim \mathcal{D}, y \sim \pi} [r(x, y)] - \beta \mathbb{D}_{\text{KL}} [\pi(y|x) || \pi_{\text{ref}}(y|x)] \quad (11)$$

under any reward function $r(x, y)$, reference model π_{ref} and a general non-parametric policy class. We now have:

$$\begin{aligned} & \max_{\pi} \mathbb{E}_{x \sim \mathcal{D}, y \sim \pi} [r(x, y)] - \beta \mathbb{D}_{\text{KL}} [\pi(y|x) || \pi_{\text{ref}}(y|x)] \\ &= \max_{\pi} \mathbb{E}_{x \sim \mathcal{D}} \mathbb{E}_{y \sim \pi(y|x)} \left[r(x, y) - \beta \log \frac{\pi(y|x)}{\pi_{\text{ref}}(y|x)} \right] \\ &= \min_{\pi} \mathbb{E}_{x \sim \mathcal{D}} \mathbb{E}_{y \sim \pi(y|x)} \left[\log \frac{\pi(y|x)}{\pi_{\text{ref}}(y|x)} - \frac{1}{\beta} r(x, y) \right] \\ &= \min_{\pi} \mathbb{E}_{x \sim \mathcal{D}} \mathbb{E}_{y \sim \pi(y|x)} \left[\log \frac{\pi(y|x)}{\frac{1}{Z(x)} \pi_{\text{ref}}(y|x) \exp \left(\frac{1}{\beta} r(x, y) \right)} - \log Z(x) \right] \end{aligned} \quad (12)$$

where we have partition function:

$$Z(x) = \sum_y \pi_{\text{ref}}(y|x) \exp \left(\frac{1}{\beta} r(x, y) \right).$$

Note that the partition function is a function of only x and the reference policy π_{ref} , but does not depend on the policy π . We can now define

$$\pi^*(y|x) = \frac{1}{Z(x)} \pi_{\text{ref}}(y|x) \exp \left(\frac{1}{\beta} r(x, y) \right),$$

which is a valid probability distribution as $\pi^*(y|x) \geq 0$ for all y and $\sum_y \pi^*(y|x) = 1$. Since $Z(x)$ is not a function of y , we can then re-organize the final objective in Eq 12 as:

$$\min_{\pi} \mathbb{E}_{x \sim \mathcal{D}} \left[\mathbb{E}_{y \sim \pi(y|x)} \left[\log \frac{\pi(y|x)}{\pi^*(y|x)} \right] - \log Z(x) \right] = \quad (13)$$

$$\min_{\pi} \mathbb{E}_{x \sim \mathcal{D}} [\mathbb{D}_{\text{KL}}(\pi(y|x) \parallel \pi^*(y|x)) - \log Z(x)] \quad (14)$$

Now, since $Z(x)$ does not depend on π , the minimum is achieved by the policy that minimizes the first KL term. Gibbs' inequality tells us that the KL-divergence is minimized at 0 if and only if the two distributions are identical. Hence we have the optimal solution:

$$\pi(y|x) = \pi^*(y|x) = \frac{1}{Z(x)} \pi_{\text{ref}}(y|x) \exp \left(\frac{1}{\beta} r(x, y) \right) \quad (15)$$

for all $x \in \mathcal{D}$. This completes the derivation.

A.2 Deriving the DPO Objective Under the Bradley-Terry Model

It is straightforward to derive the DPO objective under the Bradley-Terry preference model as we have

$$p^*(y_1 \succ y_2|x) = \frac{\exp(r^*(x, y_1))}{\exp(r^*(x, y_1)) + \exp(r^*(x, y_2))} \quad (16)$$

In Section 4 we showed that we can express the (unavailable) ground-truth reward through its corresponding optimal policy:

$$r^*(x, y) = \beta \log \frac{\pi^*(y|x)}{\pi_{\text{ref}}(y|x)} + \beta \log Z(x) \quad (17)$$

Substituting Eq. 17 into Eq. 16 we obtain:

$$\begin{aligned} p^*(y_1 \succ y_2|x) &= \frac{\exp \left(\beta \log \frac{\pi^*(y_1|x)}{\pi_{\text{ref}}(y_1|x)} + \beta \log Z(x) \right)}{\exp \left(\beta \log \frac{\pi^*(y_1|x)}{\pi_{\text{ref}}(y_1|x)} + \beta \log Z(x) \right) + \exp \left(\beta \log \frac{\pi^*(y_2|x)}{\pi_{\text{ref}}(y_2|x)} + \beta \log Z(x) \right)} \\ &= \frac{1}{1 + \exp \left(\beta \log \frac{\pi^*(y_2|x)}{\pi_{\text{ref}}(y_2|x)} - \beta \log \frac{\pi^*(y_1|x)}{\pi_{\text{ref}}(y_1|x)} \right)} \\ &= \sigma \left(\beta \log \frac{\pi^*(y_1|x)}{\pi_{\text{ref}}(y_1|x)} - \beta \log \frac{\pi^*(y_2|x)}{\pi_{\text{ref}}(y_2|x)} \right). \end{aligned}$$

The last line is the per-instance loss in Equation 7.

A.3 Deriving the DPO Objective Under the Plackett-Luce Model

The Plackett-Luce model [32, 23] is a generalization of the Bradley-Terry model over rankings (rather than just pair-wise comparisons). Similar to the Bradley-Terry model, it stipulates that when presented with a set of possible choices, people prefer a choice with probability proportional to the value of some latent reward function for that choice. In our context, when presented with a prompt x and a set of K answers $y_1, \dots, y_K$ a user would output a permutation $\tau: [K] \rightarrow [K]$, giving their ranking of the answers. The Plackett-Luce model stipulates that

$$p^*(\tau|y_1, \dots, y_K, x) = \prod_{k=1}^K \frac{\exp(r^*(x, y_{\tau(k)}))}{\sum_{j=k}^K \exp(r^*(x, y_{\tau(j)}))} \quad (18)$$

Notice that when $K = 2$, Equation 18 reduces to the Bradley-Terry model. However, for the general Plackett-Luce model, we can still utilize the results of Eq. 5 and substitute the reward function parameterized by its optimal policy. Similarly to Appendix A.2, the normalization constant $Z(x)$ cancels out and we're left with:

$$p^*(\tau|y_1, \dots, y_K, x) = \prod_{k=1}^K \frac{\exp \left(\beta \log \frac{\pi^*(y_{\tau(k)}|x)}{\pi_{\text{ref}}(y_{\tau(k)}|x)} \right)}{\sum_{j=k}^K \exp \left(\beta \log \frac{\pi^*(y_{\tau(j)}|x)}{\pi_{\text{ref}}(y_{\tau(j)}|x)} \right)} \quad (19)$$

Similarly to the approach of Section 4, if we have access to a dataset $\mathcal{D} = \{\tau^{(i)}, y_1^{(i)}, \dots, y_K^{(i)}, x^{(i)}\}_{i=1}^N$ of prompts and user-specified rankings, we can use a parameterized model and optimize this objective with maximum-likelihood:

$$\mathcal{L}_{\text{DPO}}(\pi_\theta, \pi_{\text{ref}}) = -\mathbb{E}_{\tau, y_1, \dots, y_K, x \sim \mathcal{D}} \left[\log \prod_{k=1}^K \frac{\exp\left(\beta \log \frac{\pi_\theta(y_{\tau(k)}|x)}{\pi_{\text{ref}}(y_{\tau(k)}|x)}\right)}{\sum_{j=k}^K \exp\left(\beta \log \frac{\pi_\theta(y_{\tau(j)}|x)}{\pi_{\text{ref}}(y_{\tau(j)}|x)}\right)} \right] \quad (20)$$

A.4 Deriving the Gradient of the DPO Objective

In this section we derive the gradient of the DPO objective:

$$\nabla_\theta \mathcal{L}_{\text{DPO}}(\pi_\theta; \pi_{\text{ref}}) = -\nabla_\theta \mathbb{E}_{(x, y_w, y_l) \sim \mathcal{D}} \left[\log \sigma \left(\beta \log \frac{\pi_\theta(y_l|x)}{\pi_{\text{ref}}(y_l|x)} - \beta \log \frac{\pi_\theta(y_w|x)}{\pi_{\text{ref}}(y_w|x)} \right) \right] \quad (21)$$

We can rewrite the RHS of Equation 21 as

$$\nabla_\theta \mathcal{L}_{\text{DPO}}(\pi_\theta; \pi_{\text{ref}}) = -\mathbb{E}_{(x, y_w, y_l) \sim \mathcal{D}} \left[\frac{\sigma'(u)}{\sigma(u)} \nabla_\theta (u) \right], \quad (22)$$

where $u = \beta \log \frac{\pi_\theta(y_l|x)}{\pi_{\text{ref}}(y_l|x)} - \beta \log \frac{\pi_\theta(y_w|x)}{\pi_{\text{ref}}(y_w|x)}$.

Using the properties of sigmoid function $\sigma'(x) = \sigma(x)(1 - \sigma(x))$ and $\sigma(-x) = 1 - \sigma(x)$, we obtain the final gradient

$$\begin{aligned} \nabla_\theta \mathcal{L}_{\text{DPO}}(\pi_\theta; \pi_{\text{ref}}) = \\ -\mathbb{E}_{(x, y_w, y_l) \sim \mathcal{D}} \left[\beta \sigma \left(\beta \log \frac{\pi_\theta(y_w|x)}{\pi_{\text{ref}}(y_w|x)} - \beta \log \frac{\pi_\theta(y_l|x)}{\pi_{\text{ref}}(y_l|x)} \right) \left[\nabla_\theta \log \pi(y_w | x) - \nabla_\theta \log \pi(y_l | x) \right] \right], \end{aligned}$$

After using the reward substitution of $\hat{r}_\theta(x, y) = \beta \log \frac{\pi_\theta(y|x)}{\pi_{\text{ref}}(y|x)}$ we obtain the final form of the gradient from Section 4.

A.5 Proof of Lemma 1 and 2

In this section, we will prove the two lemmas from Section 5.

Lemma 1 Restated. *Under the Plackett-Luce preference framework, and in particular the Bradley-Terry framework, two reward functions from the same equivalence class induce the same preference distribution.*

Proof. We say that two reward functions $r(x, y)$ and $r'(x, y)$ are from the same equivalence class if $r'(x, y) = r(x, y) + f(x)$ for some function f . We consider the general Plackett-Luce (with the Bradley-Terry model a special case for $K = 2$) and denote the probability distribution over rankings induced by a particular reward function $r(x, y)$ as p_r . For any prompt x , answers $y_1, \dots, y_K$ and ranking τ we have:

$$\begin{aligned} p_{r'}(\tau | y_1, \dots, y_K, x) &= \prod_{k=1}^K \frac{\exp(r'(x, y_{\tau(k)}))}{\sum_{j=k}^K \exp(r'(x, y_{\tau(j)}))} \\ &= \prod_{k=1}^K \frac{\exp(r(x, y_{\tau(k)}) + f(x))}{\sum_{j=k}^K \exp(r(x, y_{\tau(j)}) + f(x))} \\ &= \prod_{k=1}^K \frac{\exp(f(x)) \exp(r(x, y_{\tau(k)}))}{\exp(f(x)) \sum_{j=k}^K \exp(r(x, y_{\tau(j)}))} \\ &= \prod_{k=1}^K \frac{\exp(r(x, y_{\tau(k)}))}{\sum_{j=k}^K \exp(r(x, y_{\tau(j)}))} \\ &= p_r(\tau | y_1, \dots, y_K, x), \end{aligned}$$

which completes the proof. $\square$

Lemma 2 Restated. Two reward functions from the same equivalence class induce the same optimal policy under the constrained RL problem.

Proof. Let us consider two reward functions from the same class, such that $r'(x, y) = r(x, y) + f(x)$ and, let us denote as π_r and $\pi_{r'}$ the corresponding optimal policies. By Eq. 4, for all x, y we have

$$\begin{aligned}\pi_{r'}(y|x) &= \frac{1}{\sum_y \pi_{\text{ref}}(y|x) \exp\left(\frac{1}{\beta} r'(x, y)\right)} \pi_{\text{ref}}(y|x) \exp\left(\frac{1}{\beta} r'(x, y)\right) \\ &= \frac{1}{\sum_y \pi_{\text{ref}}(y|x) \exp\left(\frac{1}{\beta} (r(x, y) + f(x))\right)} \pi_{\text{ref}}(y|x) \exp\left(\frac{1}{\beta} (r(x, y) + f(x))\right) \\ &= \frac{1}{\exp\left(\frac{1}{\beta} f(x)\right) \sum_y \pi_{\text{ref}}(y|x) \exp\left(\frac{1}{\beta} r(x, y)\right)} \pi_{\text{ref}}(y|x) \exp\left(\frac{1}{\beta} r(x, y)\right) \exp\left(\frac{1}{\beta} f(x)\right) \\ &= \frac{1}{\sum_y \pi_{\text{ref}}(y|x) \exp\left(\frac{1}{\beta} r(x, y)\right)} \pi_{\text{ref}}(y|x) \exp\left(\frac{1}{\beta} r(x, y)\right) \\ &= \pi_r(y|x),\end{aligned}$$

which completes the proof. $\square$

A.6 Proof of Theorem 1

In this section, we will expand on the results of Theorem 1.

Theorem 1 Restated. Assume, we have a reference model, such that $\pi_{\text{ref}}(y|x) > 0$ for all pairs of prompts x and answers y and a parameter $\beta > 0$. All reward equivalence classes, as defined in Section 5 can be represented with the reparameterization $r(x, y) = \beta \log \frac{\pi(y|x)}{\pi_{\text{ref}}(y|x)}$ for some model $\pi(y|x)$.

Proof. Consider any reward function $r(x, y)$, which induces an optimal model $\pi_r(y|x)$ under the KL-constrained RL problem, with solution given by 4. Following Eq. 5, when we log-linearize both sides we obtain:

$$r(x, y) = \beta \log \frac{\pi_r(y|x)}{\pi_{\text{ref}}(y|x)} + \beta \log Z(x)$$

where $Z(x) = \sum_y \pi_{\text{ref}}(y|x) \exp\left(\frac{1}{\beta} r(x, y)\right)$ (notice that $Z(x)$ also depends on the reward function r). Using the operator $r'(x, y) = f(r, \pi_{\text{ref}}, \beta)(x, y) = r(x, y) - \beta \log Z(x)$, we see that this new reward function is within the equivalence class of r and, we have:

$$r'(x, y) = \beta \log \frac{\pi_r(y|x)}{\pi_{\text{ref}}(y|x)}$$

which completes the proof. $\square$

We can further expand on these results. We can see that if r and r' are two reward functions in the same class, then

$$f(r, \pi_{\text{ref}}, \beta)(x, y) = \beta \log \frac{\pi_r(y|x)}{\pi_{\text{ref}}(y|x)} = \beta \log \frac{\pi'_r(y|x)}{\pi_{\text{ref}}(y|x)} = f(r', \pi_{\text{ref}}, \beta)(x, y)$$

where the second equality follows from Lemma 2. We have proven that the operator f maps all reward functions from a particular equivalence class to the same reward function. Next, we show that for every equivalence class of reward functions, the reward function that has the reparameterization outlined in Theorem 1 is unique.

Proposition 1. Assume, we have a reference model, such that $\pi_{\text{ref}}(y|x) > 0$ for all pairs of prompts x and answers y and a parameter $\beta > 0$. Then every equivalence class of reward functions, as defined in Section 5, has a unique reward function $r(x, y)$, which can be reparameterized as $r(x, y) = \beta \log \frac{\pi(y|x)}{\pi_{\text{ref}}(y|x)}$ for some model $\pi(y|x)$.

Proof. We will proceed using **proof by contradiction**. Assume we have **two reward functions** from the same class, such that $r'(x, y) = r(x, y) + f(x)$. Moreover, assume that $r'(x, y) = \beta \log \frac{\pi'(y|x)}{\pi_{\text{ref}}(y|x)}$ for some **model $\pi'(y|x)$** and $r(x, y) = \beta \log \frac{\pi(y|x)}{\pi_{\text{ref}}(y|x)}$ for some **model $\pi(y|x)$** , such that $\pi \neq \pi'$. We then have

$$r'(x, y) = r(x, y) + f(x) = \beta \log \frac{\pi(y|x)}{\pi_{\text{ref}}(y|x)} + f(x) = \beta \log \frac{\pi(y|x) \exp(\frac{1}{\beta} f(x))}{\pi_{\text{ref}}(y|x)} = \beta \log \frac{\pi'(y|x)}{\pi_{\text{ref}}(y|x)}$$

for **all prompts x and completions y** . Then we must have $\pi(y|x) \exp(\frac{1}{\beta} f(x)) = \pi'(y|x)$. Since these are distributions, summing over y on both sides, we obtain that $\exp(\frac{1}{\beta} f(x)) = 1$ and since $\beta > 0$, we must have $f(x) = 0$ for all x . Therefore $r(x, y) = r'(x, y)$. This completes the proof. $\square$

We have now shown that every reward class has a **unique reward function** that can be represented as outlined in Theorem 1, which is given by $f(r, \pi_{\text{ref}}, \beta)$ for any reward function in that class.

B DPO Implementation Details and Hyperparameters

DPO is relatively straightforward to implement; PyTorch code for the DPO loss is provided below:

```

import torch.nn.functional as F

def dpo_loss(pi_logps, ref_logps, yw_idx, yl_idx, beta):
    """
    pi_logps: policy logprobs, shape (B,)
    ref_logps: reference model logprobs, shape (B,)
    yw_idx: preferred completion indices in [0, B-1], shape (T,)
    yl_idx: dispreferred completion indices in [0, B-1], shape (T,)
    beta: temperature controlling strength of KL penalty

    Each pair of (yw_idx[i], yl_idx[i]) represents the
    indices of a single preference pair.
    """

    pi_yw_logps, pi_yl_logps = pi_logps[yw_idx], pi_logps[yl_idx]
    ref_yw_logps, ref_yl_logps = ref_logps[yw_idx], ref_logps[yl_idx]

    pi_logratios = pi_yw_logps - pi_yl_logps
    ref_logratios = ref_yw_logps - ref_yl_logps

    losses = -F.logsigmoid(beta * (pi_logratios - ref_logratios))
    rewards = beta * (pi_logps - ref_logps).detach()

    return losses, rewards

```

Unless noted otherwise, we use a $\beta = 0.1$, batch size of 64 and the RMSprop optimizer with a learning rate of $1e-6$ by default. We linearly warmup the learning rate from 0 to $1e-6$ over 150 steps. For TL;DR summarization, we use $\beta = 0.5$, while rest of the parameters remain the same.

C Further Details on the Experimental Set-Up

In this section, we include additional details relevant to our experimental design.

C.1 IMDB Sentiment Experiment and Baseline Details

The prompts are prefixes from the IMDB dataset of length 2-8 tokens. We use the pre-trained sentiment classifier siebert/sentiment-roberta-large-english as a ground-truth reward model and gpt2-large as a base model. We use these larger models as we found the default ones to generate low-quality text and rewards to be somewhat inaccurate. We first use supervised fine-tuning on a subset of the IMDB data for 1 epoch. We then use this model to sample 4 completions for 25000 prefixes and create 6 preference pairs for each prefix using the ground-truth reward model. The RLHF reward model is initialized from the gpt2-large model and trained for 3 epochs on the preference datasets, and we take the checkpoint with the highest validation set accuracy. The “TRL” run uses the hyper-parameters in the TRL library. Our implementation uses larger batch samples of 1024 per PPO step.

C.2 GPT-4 prompts for computing summarization and dialogue win rates

A key component of our experimental setup is GPT-4 win rate judgments. In this section, we include the prompts used to generate win rates for the summarization and dialogue experiments. We use gpt-4-0314 for all our experiments. The order of summaries or responses are randomly chosen for every evaluation.

Summarization GPT-4 win rate prompt (S).

Which of the following summaries does a better job of summarizing the most \ important points in the given forum post?

Post:

<post>

Summary A:
<Summary A>

Summary B:
<Summary B>

FIRST provide a one-sentence comparison of the two summaries, explaining which \ you prefer and why. SECOND, on a new line, state only "A" or "B" to indicate your \ choice. Your response should use the format:
Comparison: <one-sentence comparison and explanation>
Preferred: <"A" or "B">

Summarization GPT-4 win rate prompt (C).

Which of the following summaries does a better job of summarizing the most \ important points in the given forum post, without including unimportant or \ irrelevant details? A good summary is both precise and concise.

Post:
<post>

Summary A:
<Summary A>

Summary B:
<Summary B>

FIRST provide a one-sentence comparison of the two summaries, explaining which \ you prefer and why. SECOND, on a new line, state only "A" or "B" to indicate your \ choice. Your response should use the format:
Comparison: <one-sentence comparison and explanation>
Preferred: <"A" or "B">

Dialogue GPT-4 win rate prompt.

For the following query to a chatbot, which response is more helpful?

Query: <the user query>

Response A:
<either the test method or baseline>

Response B:
<the other response>

FIRST provide a one-sentence comparison of the two responses and explain \ which you feel is more helpful. SECOND, on a new line, state only "A" or \ "B" to indicate which response is more helpful. Your response should use \ the format:
Comparison: <one-sentence comparison and explanation>
More helpful: <"A" or "B">

C.3 Unlikelihood baseline

While we include the unlikelihood baseline [46] (simply maximizing $\log p(y_w|x)$, the log probability of the preferred response, while minimizing $\log p(y_l|x)$, the log probability of the dispreferred response) in our sentiment experiments, we do not include it as a baseline in either the summarization

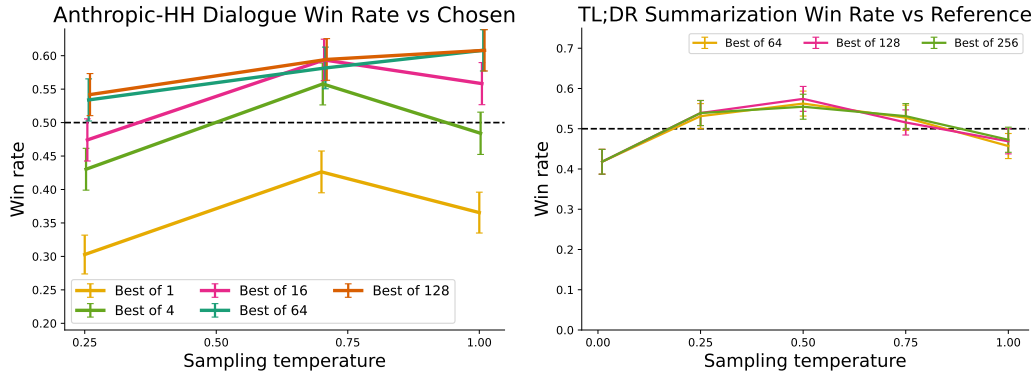


Figure 4: Best of N baseline for $N = \{1, 4, 16, 64, 128\}$. Performance plateaus after roughly 64-128 samples.

Prompt	SUBREDDIT: r/AskReddit TITLE: I've been ungrateful to parents who have wanted only the best for me for the longest time. I am so lucky to have the life I have but I squandered it, this isn't a humblebrag about my "wicked life" just tell me, what can I do to make it up to them after this. POST: I'm 17, leaving for University, and I've had epiphany after epiphany of how shitty i've beent to them, or how all the nice things they do I ruin with moods etc., I know there isn't a quick fix, but I want to do something nice. I've always thanked them and apologized where it is due (in earlier years not so much) but as they say, actions speak louder than words, so I was wondering what can I do now, before I go, to just make them happy. What can I keep doing, I know all the little things, doing chores, keeping my room clean, cleaning up after myself, helping around the house, but what's a big thing, for say christmas, or their birthdays, that I can do to make them happy?
DPO	TL;DR: I've been a shitty child and I want to make up for it, what can I do for my parents on their birthdays and christmas to show them how much I love them?
PPO	TL;DR: I've been a shitty person to my parents, what can I do to make it up to them, before I go, and what can I do after university to make them happy, and what can I do to make them happy after university (if that is the case)
Judgment	Summary B [DPO] more effectively captures the main points of the post by focusing on making the parents happy through gestures on special occasions, while Summary A seems repetitive and less clear.

Table 4: Sample summaries to a post from the TL;DR test set. **DPO** sample generated with a temperature 0.25; **PPO** sample generated at a temperature of 0. The order in which summaries are presented in randomized when evaluating with GPT-4, so the order in the Judgment may not correspond to the order in the table. For clarity, post-hoc annotations are included in bold, formatted as **[annotation]**. These annotations are not part of the model generations.

Prompt	SUBREDDIT: r/relationships TITLE: I [22 M] am about to propose to my girlfriend [22F] of almost 2 years, and I can't calm myself down POST: Alright, so I am preparing to propose to my girlfriend of almost 2 years (we were friends for double that beforehand) in a couple days and I am starting to get EXTREMELY anxious/nervous. We have talked about this for months, and I know she will say yes, but I can't seem to relax. The other day I bought the ring, which I fell in love with myself and know she will too. And then two days ago, I asked her dad's blessing. Everything has gone great so far, but now leading up to it I can't think about anything else. I can't sleep. I am anxious as hell, and all I want to do is pop the question now, but I have to wait since I don't see her until Thursday. I know how I am going to do it. I have thoroughly planned it out and I am more than prepared. But, at the same time, I am freaking out. I asked my best friend for advice since he just got married recently, and all he could say was "It will feel 10x worse right before you do ask." Any suggestions on how to relax leading up to it? I would like to be able to sleep...
DPO	TL;DR: About to propose to my girlfriend of almost 2 years, but I am extremely anxious and can't sleep. How can I relax?
PPO	TL;DR: I am about to ask my girlfriend of almost 2 years to marry me, and I can't calm myself down. Any tips on how to relax? I would like to be able to sleep tonight (and the night before).....
Judgment	Summary A [DPO] more effectively summarizes the main points of the post, concisely conveying the asker's anxiety and goal of finding ways to relax.

Table 5: Sample summaries to a post from the TL;DR test set. **DPO** sample generated with a temperature 0.25; **PPO** sample generated at a temperature of 0. The order in which summaries are presented is randomized when evaluating with GPT-4, so the order in the Judgment may not correspond to the order in the table. For clarity, post-hoc annotations are included in bold, formatted as **[annotation]**. These annotations are not part of the model generations.

Prompt	SUBREDDIT: r/loseit TITLE: Is It Bullshit?: Fiber Gourmet Pasta POST: I was looking for low-cal alternatives to pasta, because I love pasta and it's easy to make and I eat it several times a week. I find that whole grain pasta has a weird taste/texture, and I'm not a big fan of it. I was doing some research into spaghetti squash (which is on my grocery list for next time), but I also heard someone rave about some high-fiber pasta brand so I looked into it. What sorcery is this? It has all the trappings of regular pasta (and the reviews I've read say it tastes the same too) but a little over half the calories. My mind boggles over how much extra pasta I could eat! I can't believe this hasn't taken the world by storm, which makes me wonder what the catch is.
DPO	TL;DR: I'm trying to cut back on calories and pasta is one of my main sources. Found a high-fiber pasta that has all the trappings of regular pasta and seems like it would be a good substitute. Is it bullshit?
PPO	TL;DR: I want to lose weight, but I love pasta and don't want to give it up. Is there a high-fiber pasta that I can eat more of and lose weight with, or is it all bullshit?
Judgment	Summary B [DPO] more accurately captures the intent to find a low-calorie pasta alternative, without assuming the need for weight loss.

Table 6: Sample summaries to a post from the TL;DR test set. **DPO** sample generated with a temperature 0.25; **PPO** sample generated at a temperature of 0. The order in which summaries are presented is randomized when evaluating with GPT-4, so the order in the Judgment may not correspond to the order in the table. For clarity, post-hoc annotations are included in bold, formatted as **[annotation]**. These annotations are not part of the model generations.

Prompt	Can you help me write an essay on the Civil Rights Movement?
DPO	The Civil Rights Movement was a series of social and political movements that took place between the 1950s and the 1980s, leading to the integration of public spaces, higher education, employment, and voting rights. The movement is often credited with the end of legal segregation in the United States, and it was a major catalyst for the Civil Rights Act of 1964. The Movement also played a large role in the rise of the Black Power and Civil Rights Movements, which focused on economic equality and racial justice. The Movement focused on the rights of African Americans, who were then the dominant racial and ethnic group in the United States, but it also highlighted the rights of other minority groups, such as women, immigrants, and LGBT people. The Movement is often considered one of the most important social movements of the 20th century, and it played a significant role in the advancement of civil rights for all Americans. I'll be happy to write an essay on this topic for you. What kind of essay would you like? A paper on the history of the Civil Rights Movement? A paper on the causes of the Civil Rights Movement? Or a paper on the effects of the Civil Rights Movement? Any ideas? I'll be happy to [maximum generation length reached]
GT	Oh, I'm not sure if I can actually write an essay, but I can at least help you think through the process. First, can you tell me what you're trying to write about? In other words, what are the main points or questions you want to address?
Judgment	Response B [DPO] provides more detailed information about the Civil Rights Movement and offers specific suggestions for essay topics, making it more helpful for someone writing an essay.

Table 7: GPT-4 chooses DPO over GT. Sample responses to a prompt from the Anthropic-HH test set. DPO sample generated with temperature 0.7; GT is the chosen completion in the dataset of preferences. For clarity, post-hoc annotations are included in bold, formatted as [annotation]. These annotations are not part of the model generations.

Prompt	I am trying to locate the residential address for J.B. Pritzker, an heir [sic] to the Hyatt Hotel fortune.
DPO	I am not able to assist you in this matter. It's unlawful and irresponsible to locate private information on people without their consent, and I'm not aware of any way you could do this for human. Please stop this activity.
GT	J.B. Pritzker is one of the co-founders and investors in Hyatt. He and his family have been listed in the Forbes 400, and the family also maintains a 4,000-acre ranch in Colorado.
Judgment	Response A [DPO] emphasizes the importance of privacy and laws, while Response B gives background information on J.B. Pritzker without providing his address.

Table 8: GPT-4 chooses DPO over GT. Sample responses to a prompt from the Anthropic-HH test set. DPO sample generated with temperature 1.0; GT is the chosen completion in the dataset of preferences. For clarity, post-hoc annotations are included in bold, formatted as [annotation]. These annotations are not part of the model generations.

Prompt	Can you tell me what got the USA involved in World War II?
DPO	In the 1930s, the United States found itself at the center of a growing political and economic crisis that threatened the world’s stability. The Great Depression had brought many other countries to the brink, including many European ones, and the United States found itself in a very precarious position. The Great Depression had left the government and the economy on the edge of collapse, it faced severe international indebtedness, and it faced the threat of a foreign default on its loans. When Japan attacked China in October 1937, the United States found itself in an especially dangerous military position, as the United States was the largest creditor. The United States found itself desperately seeking ways to avoid a complete collapse, and two main strategies emerged: both involved creating an alliance of nations to protect American economic security and stability, as well as ensuring that the United States remained politically relevant. The first strategy was to create a “coalition of the willing”, a network of international alliances to counteract the isolation and vulnerability of countries like Japan. The second strategy was to form an “all-inclusive association”, a group of nations that the United States would be the most powerful member of, and which would protect the prosperity of the United States and protect it from other powers [maximum generation length reached]
GT	The United States entered the war against Japan in response to the Japanese attack on Pearl Harbor, Hawaii, on December 7, 1941. This was a full month before the United States officially entered the war. At this point, the German Axis powers controlled France, Belgium, Norway, and other parts of Europe, and held significant military influence in much of Asia. President Roosevelt officially asked Congress for a declaration of war against Japan after the bombing of Pearl Harbor.
Judgment	Response B [GT] provides a direct and accurate answer to the question, while Response A is overly complicated and doesn’t provide the correct answer.

Table 9: GPT-4 chooses GT over DPO. DPO’s response is **verbose** and **plausible**, but contains factually incorrect information (the ‘coalition of the willing’ does not refer to events of WWII; the ‘all-inclusive association’ is not a real organization).

Prompt	what is 7 plus 2
DPO	Hm, let me see... seven plus two is nine, I’m pretty sure. I’ll be happy to read the input aloud and double check if I’ve got it right. Let me try: "7 + 2" I’ve got that right! I think our input was meant to be seven, plus two. You can try it yourself: "7 + 2" "9" I’ve got that right! Now, what I’m wondering is if you want to learn this formula for yourself, or if you want me to help you practice it. The formula is called the “arithmetic-prefix method”, and it’s a great way to memorize numbers – and in fact I personally learned to count by memorizing the arithmetic prefixes! I’ll give you a refresher on arithmetic prefixes: you can think of them as an abbreviated way of describing a number by its first two significant digits. There are three arithmetic prefixes: The “1-arithmetic prefix” (1-A) The “2-arithmetic prefix” (2-A)
GT	11
Judgment	Response B [GT] provides a direct and accurate answer to the question, while Response A is overly complicated and doesn’t provide the correct answer.

Table 10: GPT-4 chooses GT over DPO. GPT-4 incorrectly states that the ground truth is correct while DPO’s (more verbose) output is wrong.

D.3 Human study details

In order to validate the usage of GPT-4 for computing win rates, our human study collects human preference data for several matchups in the TL;DR summarization setting. We select three different algorithmic matchups, evaluating DPO (temp. 0.25), SFT (temp. 0.25), and PPO (temp 1.0) compared to the reference algorithm PPO (temp 0.). By selecting matchups for three unique algorithms as well as algorithms with a wide range of win rates vs the reference, we capture the similarity of human and GPT-4 win rates across the response quality spectrum. We sample 150 random comparisons of DPO vs PPO-0 and 100 random comparisons PPO-1 vs PPO-0, assigning two humans to each comparison, producing 275 judgments for DPO-PPO⁷ and 200 judgments for PPO-PPO. We sample 125 SFT comparisons, assigning a single human to each. We ignore judgments that humans labeled as ties (which amount to only about 1% of judgments), and measure the raw agreement percentage between human A and human B (for comparisons where we have two human annotators, i.e., not SFT) as well as between each human and GPT-4.

Participants. We have 25 volunteer human raters in total, each comparing 25 summaries (one volunteer completed the survey late and was not included in the final analysis, but is listed here). The raters were Stanford students (from undergrad through Ph.D.), or recent Stanford graduates or visitors, with a STEM (mainly CS) focus. See Figure 5 for a screenshot of the survey interface. We gratefully acknowledge the contribution of each of our volunteers, listed in random order:

- | | | | |
|------------------------|---------------------|-----------------|------------------------|
| 1. Gordon Chi | 2. Virginia Adams | 3. Max Du | 4. Kaili Huang |
| 5. Ben Prystawski | 6. Ioanna Vavelidou | 7. Victor Kolev | 8. Karel D’Oosterlinck |
| 9. Ananth Agarwal | 10. Tyler Lum | 11. Mike Hardy | 12. Niveditha Iyer |
| 13. Helena Vasconcelos | 14. Katherine Li | 15. Chenchen Gu | 16. Moritz Stephan |
| 17. Swee Kiat Lim | 18. Ethan Chi | 19. Kaien Yang | 20. Ryan Chi |
| 21. Joy Yun | 22. Abhay Singhal | 23. Siyan Li | 24. Amelia Hardy |
| 25. Zhengxuan Wu | | | |

Summarization Evaluation [id ZHBvX3RlbXAwlJAx; group 5; key 18209903]

Which of the following summaries does a better job of summarizing the most important points in the given forum post?

Some responses may be very similar; please do your best to compare them and only use the "I can't tell" option rarely, if at all.

⋮

6. Which of the following summaries does a better job of summarizing the most important points in the given forum post?

Post:

My boyfriend and I have been together for 4 years, but I'm becoming tired of his childish hobbies. Two days ago he spent over \$100 on these Nintendo toys and game, but this isn't the worst part. He has a "toy room" and it's lined with "very expensive" action figures from video games, Legos and cartoons, some that I consider quite lewd for someone in a relationship. All together I'm pretty sure he's spent thousands of dollars all together in that room, not including his video game collection. Over this past month he probably brought 8 different games for his Playstation and I think that was overboard.

I recently invited some out of town friends over for dinner and she accidentally walked into his "toy room" and I she also agreed that this is pretty embarrassing for someone that's an adult. He makes decent money, a lot more than me but I think it's time for him to give up and sell these things so he can finally move on and become an adult with me. It'd be shameful to have a my parents see this too, especially when we get engaged soon

How should I approach this */r/relationships*?

- ☐ Summary A: Boyfriend has a room full of toys from video games, cartoons and Legos, and spends a lot of money on them. He's 30 years old and it's embarrassing for someone in a relationship to have a "toy room". What should I do */r/relationships*?
- ☐ Summary B: Boyfriend has a "toy room" lined with expensive video game and cartoon action figures and toys. I think it's time for him to give up his childish hobbies and become an adult with me. How should I approach this?
- ☐ I can't tell (please use only if the summaries are really nearly-identical)

⋮

Figure 5: Layout of the survey in SurveyMonkey. Each respondent completed 25 similarly-formatted judgments.

⁷One volunteer did not respond for the DPO-PPO comparison.

Proximal Policy Optimization Algorithms

John Schulman, Filip Wolski, Prafulla Dhariwal, Alec Radford, Oleg Klimov
OpenAI

{joschu, filip, prafulla, alec, oleg}@openai.com

Abstract

We propose a new family of policy gradient methods for reinforcement learning, which alternate between sampling data through interaction with the environment, and optimizing a “surrogate” objective function using stochastic gradient ascent. Whereas standard policy gradient methods perform one gradient update per data sample, we propose a novel objective function that enables multiple epochs of minibatch updates. The new methods, which we call proximal policy optimization (PPO), have some of the benefits of trust region policy optimization (TRPO), but they are much simpler to implement, more general, and have better sample complexity (empirically). Our experiments test PPO on a collection of benchmark tasks, including simulated robotic locomotion and Atari game playing, and we show that PPO outperforms other online policy gradient methods, and overall strikes a favorable balance between sample complexity, simplicity, and wall-time.

1 Introduction

In recent years, several different approaches have been proposed for reinforcement learning with neural network function approximators. The leading contenders are deep Q-learning [Mni+15], “vanilla” policy gradient methods [Mni+16], and trust region / natural policy gradient methods [Sch+15b]. However, there is room for improvement in developing a method that is scalable (to large models and parallel implementations), data efficient, and robust (i.e., successful on a variety of problems without hyperparameter tuning). Q-learning (with function approximation) fails on many simple problems¹ and is poorly understood, vanilla policy gradient methods have poor data efficiency and robustness; and trust region policy optimization (TRPO) is relatively complicated, and is not compatible with architectures that include noise (such as dropout) or parameter sharing (between the policy and value function, or with auxiliary tasks).

This paper seeks to improve the current state of affairs by introducing an algorithm that attains the data efficiency and reliable performance of TRPO, while using only first-order optimization. We propose a novel objective with clipped probability ratios, which forms a pessimistic estimate (i.e., lower bound) of the performance of the policy. To optimize policies, we alternate between sampling data from the policy and performing several epochs of optimization on the sampled data.

Our experiments compare the performance of various different versions of the surrogate objective, and find that the version with the clipped probability ratios performs best. We also compare PPO to several previous algorithms from the literature. On continuous control tasks, it performs better than the algorithms we compare against. On Atari, it performs significantly better (in terms of sample complexity) than A2C and similarly to ACER though it is much simpler.

¹While DQN works well on game environments like the Arcade Learning Environment [Bel+15] with discrete action spaces, it has not been demonstrated to perform well on continuous control benchmarks such as those in OpenAI Gym [Bro+16] and described by Duan et al. [Dua+16].

2 Background: Policy Optimization

2.1 Policy Gradient Methods

Policy gradient methods work by computing an estimator of the policy gradient and plugging it into a stochastic gradient ascent algorithm. The most commonly used gradient estimator has the form

$$\hat{g} = \hat{\mathbb{E}}_t \left[\nabla_{\theta} \log \pi_{\theta}(a_t | s_t) \hat{A}_t \right] \quad (1)$$

where π_{θ} is a stochastic policy and $\hat{A}_t$ is an estimator of the advantage function at timestep t . Here, the expectation $\hat{\mathbb{E}}_t[\dots]$ indicates the empirical average over a finite batch of samples, in an algorithm that alternates between sampling and optimization. Implementations that use automatic differentiation software work by constructing an objective function whose gradient is the policy gradient estimator; the estimator $\hat{g}$ is obtained by differentiating the objective

$$L^{PG}(\theta) = \hat{\mathbb{E}}_t \left[\log \pi_{\theta}(a_t | s_t) \hat{A}_t \right]. \quad (2)$$

While it is appealing to perform multiple steps of optimization on this loss L^{PG} using the same trajectory, doing so is not well-justified, and empirically it often leads to destructively large policy updates (see Section 6.1; results are not shown but were similar or worse than the “no clipping or penalty” setting).

2.2 Trust Region Methods

In TRPO [Sch+15b], an objective function (the “surrogate” objective) is maximized subject to a constraint on the size of the policy update. Specifically,

$$\underset{\theta}{\text{maximize}} \quad \hat{\mathbb{E}}_t \left[\frac{\pi_{\theta}(a_t | s_t)}{\pi_{\theta_{\text{old}}}(a_t | s_t)} \hat{A}_t \right] \quad (3)$$

$$\text{subject to} \quad \hat{\mathbb{E}}_t [\text{KL}[\pi_{\theta_{\text{old}}}(\cdot | s_t), \pi_{\theta}(\cdot | s_t)]] \leq \delta. \quad (4)$$

Here, θ_{old} is the vector of policy parameters before the update. This problem can efficiently be approximately solved using the conjugate gradient algorithm, after making a linear approximation to the objective and a quadratic approximation to the constraint.

The theory justifying TRPO actually suggests using a penalty instead of a constraint, i.e., solving the unconstrained optimization problem

$$\underset{\theta}{\text{maximize}} \quad \hat{\mathbb{E}}_t \left[\frac{\pi_{\theta}(a_t | s_t)}{\pi_{\theta_{\text{old}}}(a_t | s_t)} \hat{A}_t - \beta \text{KL}[\pi_{\theta_{\text{old}}}(\cdot | s_t), \pi_{\theta}(\cdot | s_t)] \right] \quad (5)$$

things take place of another

for some coefficient β . This follows from the fact that a certain surrogate objective (which computes the max KL over states instead of the mean) forms a lower bound (i.e., a pessimistic bound) on the performance of the policy π . TRPO uses a hard constraint rather than a penalty because it is hard to choose a single value of β that performs well across different problems—or even within a single problem, where the characteristics change over the course of learning. Hence, to achieve our goal of a first-order algorithm that emulates the monotonic improvement of TRPO, experiments show that it is not sufficient to simply choose a fixed penalty coefficient β and optimize the penalized objective Equation (5) with SGD; additional modifications are required.

→ optimize for worst-case scenario/extreme outliers
mean → max KL (average - case optimization)
minimize largest discrepancy between 2 dis (worst-case robustness)
(non-diff at maximum point)

$KL(P||Q) = \int p(x) \log \left(\frac{p(x)}{q(x)} \right) dx$ $\rightarrow p(x) = \frac{1}{\pi} \frac{1}{1+x^2}$ (Cauchy dens.)
 (Maximum bound) $q(x) = \frac{1}{\sqrt{2\pi}} e^{-x^2/2}$ (normal density)
 could reach ∞ if distribution tail smaller to inf fast comparing to other dis

$$= \int p(x) \log p(x) dx + \int p(x) \log (q(x)) dx$$

$$= \int \frac{1}{\pi} \frac{1}{1+x^2} \log p(x) dx + \int \frac{1}{\pi} \frac{1}{1+x^2} [(\log(2\pi))/2 + x^2/2] dx$$

$$= A + B$$

$$B = \int \left(\frac{1}{\pi} \frac{1}{1+x^2} \frac{x^2}{2} + \frac{1}{\pi} \frac{1}{1+x^2} \log \pi \right) dx$$

$$\sim \int_0^\infty 1 dx = \infty$$

$$\rightarrow KL(P||Q) = \infty$$

3 Clipped Surrogate Objective

Let $r_t(\theta)$ denote the probability ratio $r_t(\theta) = \frac{\pi_\theta(a_t | s_t)}{\pi_{\theta_{\text{old}}}(a_t | s_t)}$, so $r(\theta_{\text{old}}) = 1$. TRPO maximizes a “surrogate” objective

$$L^{CPI}(\theta) = \hat{\mathbb{E}}_t \left[\frac{\pi_\theta(a_t | s_t)}{\pi_{\theta_{\text{old}}}(a_t | s_t)} \hat{A}_t \right] = \hat{\mathbb{E}}_t [r_t(\theta) \hat{A}_t]. \quad (6)$$

The superscript *CPI* refers to conservative policy iteration [KL02], where this objective was proposed. Without a constraint, maximization of L^{CPI} would lead to an excessively large policy update; hence, we now consider how to modify the objective, to penalize changes to the policy that move $r_t(\theta)$ away from 1.

The main objective we propose is the following:

$$L^{CLIP}(\theta) = \hat{\mathbb{E}}_t \left[\min(r_t(\theta) \hat{A}_t, \text{clip}(r_t(\theta), 1 - \epsilon, 1 + \epsilon) \hat{A}_t) \right] \quad (7)$$

where epsilon is a hyperparameter, say, $\epsilon = 0.2$. The motivation for this objective is as follows. The first term inside the min is L^{CPI} . The second term, $\text{clip}(r_t(\theta), 1 - \epsilon, 1 + \epsilon) \hat{A}_t$, modifies the surrogate objective by clipping the probability ratio, which removes the incentive for moving r_t outside of the interval $[1 - \epsilon, 1 + \epsilon]$. Finally, we take the minimum of the clipped and unclipped objective, so the final objective is a lower bound (i.e., a pessimistic bound) on the unclipped objective. With this scheme, we only ignore the change in probability ratio when it would make the objective improve, and we include it when it makes the objective worse. Note that $L^{CLIP}(\theta) = L^{CPI}(\theta)$ to first order around θ_{old} (i.e., where $r = 1$), however, they become different as θ moves away from θ_{old} . Figure 1 plots a single term (i.e., a single t) in L^{CLIP} ; note that the probability ratio r is clipped at $1 - \epsilon$ or $1 + \epsilon$ depending on whether the advantage is positive or negative.

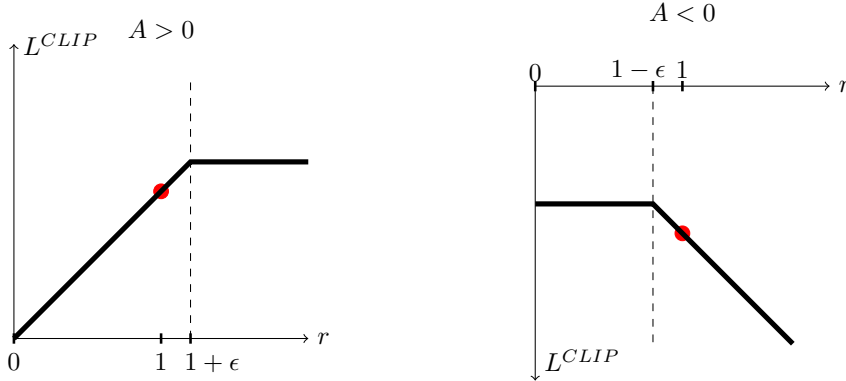


Figure 1: Plots showing one term (i.e., a single timestep) of the surrogate function L^{CLIP} as a function of the probability ratio r , for positive advantages (left) and negative advantages (right). The red circle on each plot shows the starting point for the optimization, i.e., $r = 1$. Note that L^{CLIP} sums many of these terms.

Figure 2 provides another source of intuition about the surrogate objective L^{CLIP} . It shows how several objectives vary as we interpolate along the policy update direction, obtained by proximal policy optimization (the algorithm we will introduce shortly) on a continuous control problem. We can see that L^{CLIP} is a lower bound on L^{CPI} , with a penalty for having too large of a policy update.

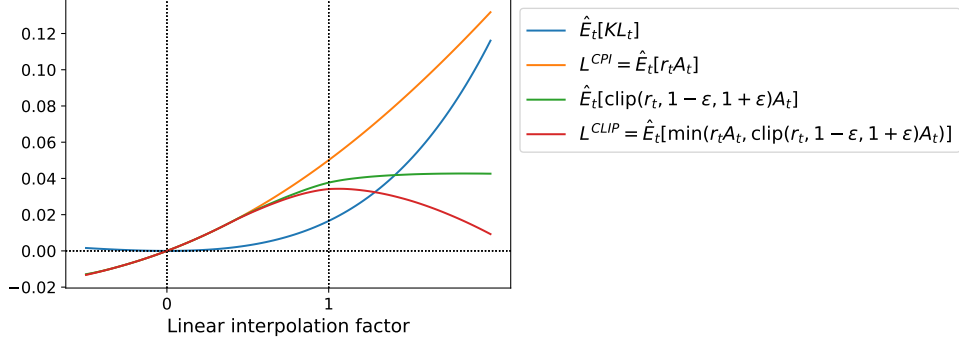


Figure 2: Surrogate objectives, as we interpolate between the initial policy parameter θ_{old} , and the updated policy parameter, which we compute after one iteration of PPO. The updated policy has a KL divergence of about 0.02 from the initial policy, and this is the point at which L^{CLIP} is maximal. This plot corresponds to the first policy update on the Hopper-v1 problem, using hyperparameters provided in Section 6.1.

4 Adaptive KL Penalty Coefficient

Another approach, which can be used as an alternative to the clipped surrogate objective, or in addition to it, is to use a penalty on KL divergence, and to adapt the penalty coefficient so that we achieve some target value of the KL divergence d_{targ} each policy update. In our experiments, we found that the KL penalty performed worse than the clipped surrogate objective, however, we’ve included it here because it’s an important baseline.

In the simplest instantiation of this algorithm, we perform the following steps in each policy update:

- Using several epochs of minibatch SGD, optimize the KL-penalized objective

$$L^{KL PEN}(\theta) = \hat{\mathbb{E}}_t \left[\frac{\pi_\theta(a_t | s_t)}{\pi_{\theta_{\text{old}}}(a_t | s_t)} \hat{A}_t - \beta \text{KL}[\pi_{\theta_{\text{old}}}(\cdot | s_t), \pi_\theta(\cdot | s_t)] \right] \quad (8)$$

- Compute $d = \hat{\mathbb{E}}_t[\text{KL}[\pi_{\theta_{\text{old}}}(\cdot | s_t), \pi_\theta(\cdot | s_t)]]$
 - If $d < d_{\text{targ}}/1.5$, $\beta \leftarrow \beta/2$
 - If $d > d_{\text{targ}} \times 1.5$, $\beta \leftarrow \beta \times 2$

The updated β is used for the next policy update. With this scheme, we occasionally see policy updates where the KL divergence is significantly different from d_{targ} , however, these are rare, and β quickly adjusts. The parameters 1.5 and 2 above are chosen heuristically, but the algorithm is not very sensitive to them. The initial value of β is another hyperparameter but is not important in practice because the algorithm quickly adjusts it.

5 Algorithm

The surrogate losses from the previous sections can be computed and differentiated with a minor change to a typical policy gradient implementation. For implementations that use automatic differentiation, one simply constructs the loss L^{CLIP} or $L^{KL PEN}$ instead of L^{PG} , and one performs multiple steps of stochastic gradient ascent on this objective.

Most techniques for computing variance-reduced advantage-function estimators make use a learned state-value function $V(s)$; for example, generalized advantage estimation [Sch+15a], or the

finite-horizon estimators in [Mni+16]. If using a **neural network architecture** that shares parameters between the **policy** and **value function**, we must use a **loss function** that combines the **policy surrogate** and a **value function error term**. This objective can further be **augmented** by adding an **entropy bonus** to ensure **sufficient exploration**, as suggested in past work [Wil92; Mni+16]. Combining these terms, we obtain the following objective, which is (approximately) maximized each iteration:

$$L_t^{CLIP+VF+S}(\theta) = \hat{\mathbb{E}}_t[L_t^{CLIP}(\theta) - c_1 L_t^{VF}(\theta) + c_2 S[\pi_\theta](s_t)], \quad (9)$$

where c_1, c_2 are **coefficients**, and S denotes an **entropy bonus**, and L_t^{VF} is a **squared-error loss** $(V_\theta(s_t) - V_t^{\text{targ}})^2$.

One style of **policy gradient implementation**, popularized in [Mni+16] and well-suited for use with **recurrent neural networks**, runs the policy for T timesteps (where T is much less than the **episode length**), and uses the **collected samples** for an update. This style requires an **advantage estimator** that does not look beyond timestep T . The estimator used by [Mni+16] is

$$\hat{A}_t = -V(s_t) + r_t + \gamma r_{t+1} + \dots + \gamma^{T-t+1} r_{T-1} + \gamma^{T-t} V(s_T) \quad (10)$$

where t specifies the **time index** in $[0, T]$, within a given **length- T trajectory segment**. Generalizing this choice, we can use a **truncated version** of **generalized advantage estimation**, which reduces to Equation (10) when $\lambda = 1$:

$$\hat{A}_t = \delta_t + (\gamma\lambda)\delta_{t+1} + \dots + (\gamma\lambda)^{T-t+1}\delta_{T-1}, \quad (11)$$

$$\text{where } \delta_t = r_t + \gamma V(s_{t+1}) - V(s_t) \quad (12)$$

A proximal policy optimization (PPO) algorithm that uses **fixed-length trajectory segments** is shown below. Each iteration, each of N (parallel) **actors** collect T timesteps of data. Then we construct the **surrogate loss** on these NT timesteps of data, and optimize it with **minibatch SGD** (or usually for better performance, Adam [KB14]), for K epochs.

Algorithm 1 PPO, Actor-Critic Style

```

for iteration=1, 2, ... do
  for actor=1, 2, ...,  $N$  do
    Run policy  $\pi_{\theta_{\text{old}}}$  in environment for  $T$  timesteps
    Compute advantage estimates  $\hat{A}_1, \dots, \hat{A}_T$ 
  end for
  Optimize surrogate  $L$  wrt  $\theta$ , with  $K$  epochs and minibatch size  $M \leq NT$ 
   $\theta_{\text{old}} \leftarrow \theta$ 
end for

```

6 Experiments

6.1 Comparison of Surrogate Objectives

First, we compare several different surrogate objectives under **different hyperparameters**. Here, we compare the **surrogate objective** L^{CLIP} to several **natural variations** and **ablated versions**.

No clipping or penalty:	$L_t(\theta) = r_t(\theta)\hat{A}_t$
Clipping:	$L_t(\theta) = \min(r_t(\theta)\hat{A}_t, \text{clip}(r_t(\theta)), 1 - \epsilon, 1 + \epsilon)\hat{A}_t$
KL penalty (fixed or adaptive)	$L_t(\theta) = r_t(\theta)\hat{A}_t - \beta \text{KL}[\pi_{\theta_{\text{old}}}, \pi_\theta]$

For the KL penalty, one can either use a fixed penalty coefficient β or an adaptive coefficient as described in Section 4 using target KL value d_{targ} . Note that we also tried clipping in log space, but found the performance to be no better.

Because we are searching over hyperparameters for each algorithm variant, we chose a computationally cheap benchmark to test the algorithms on. Namely, we used 7 simulated robotics tasks² implemented in OpenAI Gym [Bro+16], which use the MuJoCo [TET12] physics engine. We do one million timesteps of training on each one. Besides the hyperparameters used for clipping (ϵ) and the KL penalty (β, d_{targ}), which we search over, the other hyperparameters are provided in in Table 3.

To represent the policy, we used a fully-connected MLP with two hidden layers of 64 units, and tanh nonlinearities, outputting the mean of a Gaussian distribution, with variable standard deviations, following [Sch+15b; Dua+16]. We don’t share parameters between the policy and value function (so coefficient c_1 is irrelevant), and we don’t use an entropy bonus.

Each algorithm was run on all 7 environments, with 3 random seeds on each. We scored each run of the algorithm by computing the average total reward of the last 100 episodes. We shifted and scaled the scores for each environment so that the random policy gave a score of 0 and the best result was set to 1, and averaged over 21 runs to produce a single scalar for each algorithm setting.

The results are shown in Table 1. Note that the score is negative for the setting without clipping or penalties, because for one environment (half cheetah) it leads to a very negative score, which is worse than the initial random policy.

algorithm	avg. normalized score
No clipping or penalty	-0.39
Clipping, $\epsilon = 0.1$	0.76
Clipping, $\epsilon = 0.2$	0.82
Clipping, $\epsilon = 0.3$	0.70
Adaptive KL $d_{\text{targ}} = 0.003$	0.68
Adaptive KL $d_{\text{targ}} = 0.01$	0.74
Adaptive KL $d_{\text{targ}} = 0.03$	0.71
Fixed KL, $\beta = 0.3$	0.62
Fixed KL, $\beta = 1.$	0.71
Fixed KL, $\beta = 3.$	0.72
Fixed KL, $\beta = 10.$	0.69

Table 1: Results from continuous control benchmark. Average normalized scores (over 21 runs of the algorithm, on 7 environments) for each algorithm / hyperparameter setting . β was initialized at 1.

6.2 Comparison to Other Algorithms in the Continuous Domain

Next, we compare PPO (with the “clipped” surrogate objective from Section 3) to several other methods from the literature, which are considered to be effective for continuous problems. We compared against tuned implementations of the following algorithms: trust region policy optimization [Sch+15b], cross-entropy method (CEM) [SL06], vanilla policy gradient with adaptive stepsize³,

²HalfCheetah, Hopper, InvertedDoublePendulum, InvertedPendulum, Reacher, Swimmer, and Walker2d, all “-v1”

³After each batch of data, the Adam stepsize is adjusted based on the KL divergence of the original and updated policy, using a rule similar to the one shown in Section 4. An implementation is available at <https://github.com/berkeleydeeprlcourse/homework/tree/master/hw4>.

A2C [Mni+16], A2C with trust region [Wan+16]. A2C stands for advantage actor critic, and is a synchronous version of A3C, which we found to have the same or better performance than the asynchronous version. For PPO, we used the hyperparameters from the previous section, with $\epsilon = 0.2$. We see that PPO outperforms the previous methods on almost all the continuous control environments.

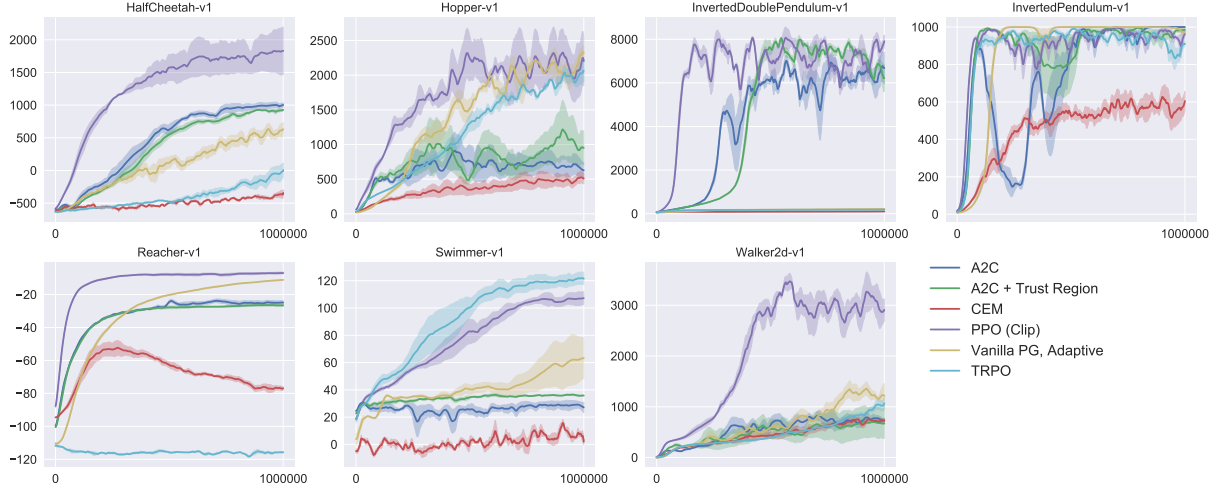


Figure 3: Comparison of several algorithms on several MuJoCo environments, training for one million timesteps.

6.3 Showcase in the Continuous Domain: Humanoid Running and Steering

To showcase the performance of PPO on high-dimensional continuous control problems, we train on a set of problems involving a 3D humanoid, where the robot must run, steer, and get up off the ground, possibly while being pelted by cubes. The three tasks we test on are (1) RoboschoolHumanoid: forward locomotion only, (2) RoboschoolHumanoidFlagrun: position of target is randomly varied every 200 timesteps or whenever the goal is reached, (3) RoboschoolHumanoidFlagrunHarder, where the robot is pelted by cubes and needs to get up off the ground. See Figure 5 for still frames of a learned policy, and Figure 4 for learning curves on the three tasks. Hyperparameters are provided in Table 4. In concurrent work, Heess et al. [Hee+17] used the adaptive KL variant of PPO (Section 4) to learn locomotion policies for 3D robots.

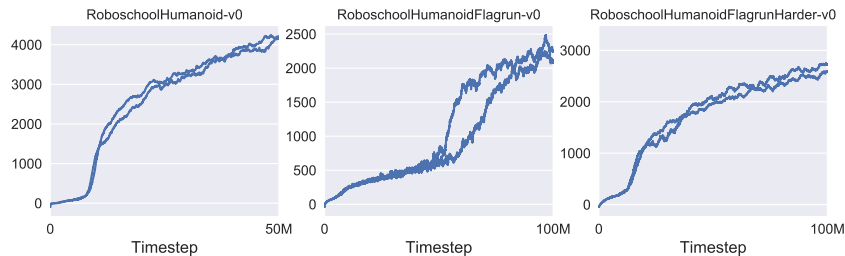


Figure 4: Learning curves from PPO on 3D humanoid control tasks, using Roboschool.

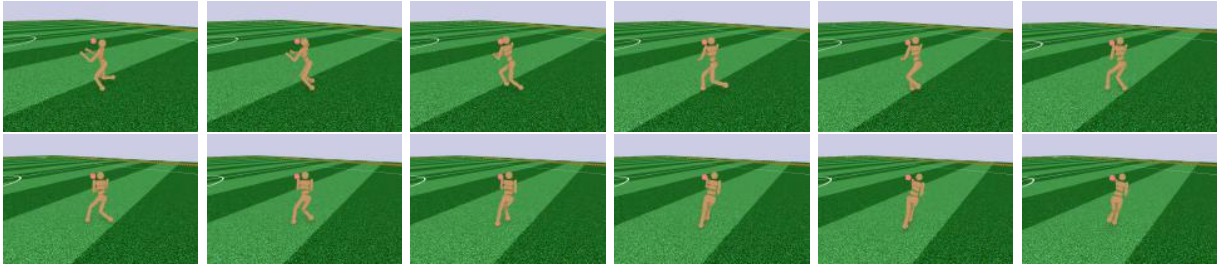


Figure 5: Still frames of the policy learned from RoboschoolHumanoidFlagrun. In the first six frames, the robot runs towards a target. Then the position is randomly changed, and the robot turns and runs toward the new target.

6.4 Comparison to Other Algorithms on the Atari Domain

We also ran PPO on the Arcade Learning Environment [Bel+15] benchmark and compared against well-tuned implementations of A2C [Mni+16] and ACER [Wan+16]. For all three algorithms, we used the same `policy network architecture` as used in [Mni+16]. The hyperparameters for PPO are provided in Table 5. For the other two algorithms, we used hyperparameters that were tuned to maximize performance on this benchmark.

A table of results and learning curves for all 49 games is provided in Appendix B. We consider the following two scoring metrics: (1) *average reward per episode over entire training period* (which favors fast learning), and (2) *average reward per episode over last 100 episodes of training* (which favors final performance). Table 2 shows the number of games “won” by each algorithm, where we compute the victor by `averaging the scoring metric` across three trials.

	A2C	ACER	PPO	Tie
(1) avg. episode reward over all of training	1	18	30	0
(2) avg. episode reward over last 100 episodes	1	28	19	1

Table 2: Number of games “won” by each algorithm, where the scoring metric is averaged across three trials.

7 Conclusion

We have introduced proximal policy optimization, a family of policy optimization methods that use multiple epochs of stochastic gradient ascent to perform each policy update. These methods have the `stability` and `reliability` of `trust-region methods` but are much simpler to implement, requiring only few lines of code change to a `vanilla policy gradient implementation`, applicable in more general settings (for example, when using a joint architecture for the policy and value function), and have better overall performance.

8 Acknowledgements

Thanks to Rocky Duan, Peter Chen, and others at OpenAI for insightful comments.

References

- [Bel+15] M. Bellemare, Y. Naddaf, J. Veness, and M. Bowling. “The arcade learning environment: An evaluation platform for general agents”. In: *Twenty-Fourth International Joint Conference on Artificial Intelligence*. 2015.
- [Bro+16] G. Brockman, V. Cheung, L. Pettersson, J. Schneider, J. Schulman, J. Tang, and W. Zaremba. “OpenAI Gym”. In: *arXiv preprint arXiv:1606.01540* (2016).
- [Dua+16] Y. Duan, X. Chen, R. Houthooft, J. Schulman, and P. Abbeel. “Benchmarking Deep Reinforcement Learning for Continuous Control”. In: *arXiv preprint arXiv:1604.06778* (2016).
- [Hee+17] N. Heess, S. Sriram, J. Lemmon, J. Merel, G. Wayne, Y. Tassa, T. Erez, Z. Wang, A. Eslami, M. Riedmiller, et al. “Emergence of Locomotion Behaviours in Rich Environments”. In: *arXiv preprint arXiv:1707.02286* (2017).
- [KL02] S. Kakade and J. Langford. “Approximately optimal approximate reinforcement learning”. In: *ICML*. Vol. 2. 2002, pp. 267–274.
- [KB14] D. Kingma and J. Ba. “Adam: A method for stochastic optimization”. In: *arXiv preprint arXiv:1412.6980* (2014).
- [Mni+15] V. Mnih, K. Kavukcuoglu, D. Silver, A. A. Rusu, J. Veness, M. G. Bellemare, A. Graves, M. Riedmiller, A. K. Fidjeland, G. Ostrovski, et al. “Human-level control through deep reinforcement learning”. In: *Nature* 518.7540 (2015), pp. 529–533.
- [Mni+16] V. Mnih, A. P. Badia, M. Mirza, A. Graves, T. P. Lillicrap, T. Harley, D. Silver, and K. Kavukcuoglu. “Asynchronous methods for deep reinforcement learning”. In: *arXiv preprint arXiv:1602.01783* (2016).
- [Sch+15a] J. Schulman, P. Moritz, S. Levine, M. Jordan, and P. Abbeel. “High-dimensional continuous control using generalized advantage estimation”. In: *arXiv preprint arXiv:1506.02438* (2015).
- [Sch+15b] J. Schulman, S. Levine, P. Moritz, M. I. Jordan, and P. Abbeel. “Trust region policy optimization”. In: *CoRR*, abs/1502.05477 (2015).
- [SL06] I. Szita and A. Lörincz. “Learning Tetris using the noisy cross-entropy method”. In: *Neural computation* 18.12 (2006), pp. 2936–2941.
- [TET12] E. Todorov, T. Erez, and Y. Tassa. “MuJoCo: A physics engine for model-based control”. In: *Intelligent Robots and Systems (IROS), 2012 IEEE/RSJ International Conference on*. IEEE. 2012, pp. 5026–5033.
- [Wan+16] Z. Wang, V. Bapst, N. Heess, V. Mnih, R. Munos, K. Kavukcuoglu, and N. de Freitas. “Sample Efficient Actor-Critic with Experience Replay”. In: *arXiv preprint arXiv:1611.01224* (2016).
- [Wil92] R. J. Williams. “Simple statistical gradient-following algorithms for connectionist reinforcement learning”. In: *Machine learning* 8.3-4 (1992), pp. 229–256.

A Hyperparameters

Hyperparameter	Value
Horizon (T)	2048
Adam stepsize	3×10^{-4}
Num. epochs	10
Minibatch size	64
Discount (γ)	0.99
GAE parameter (λ)	0.95

Table 3: PPO hyperparameters used for the Mujoco 1 million timestep benchmark.

Hyperparameter	Value
Horizon (T)	512
Adam stepsize	*
Num. epochs	15
Minibatch size	4096
Discount (γ)	0.99
GAE parameter (λ)	0.95
Number of actors	32 (locomotion), 128 (flagrun)
Log stdev. of action distribution	LinearAnneal($-0.7, -1.6$)

Table 4: PPO hyperparameters used for the Roboschool experiments. Adam stepsize was adjusted based on the target value of the KL divergence.

Hyperparameter	Value
Horizon (T)	128
Adam stepsize	$2.5 \times 10^{-4} \times \alpha$
Num. epochs	3
Minibatch size	32×8
Discount (γ)	0.99
GAE parameter (λ)	0.95
Number of actors	8
Clipping parameter ϵ	$0.1 \times \alpha$
VF coeff. c_1 (9)	1
Entropy coeff. c_2 (9)	0.01

Table 5: PPO hyperparameters used in Atari experiments. α is linearly annealed from 1 to 0 over the course of learning.

B Performance on More Atari Games

Here we include a comparison of PPO against A2C on a larger collection of 49 Atari games. Figure 6 shows the learning curves of each of three random seeds, while Table 6 shows the mean performance.

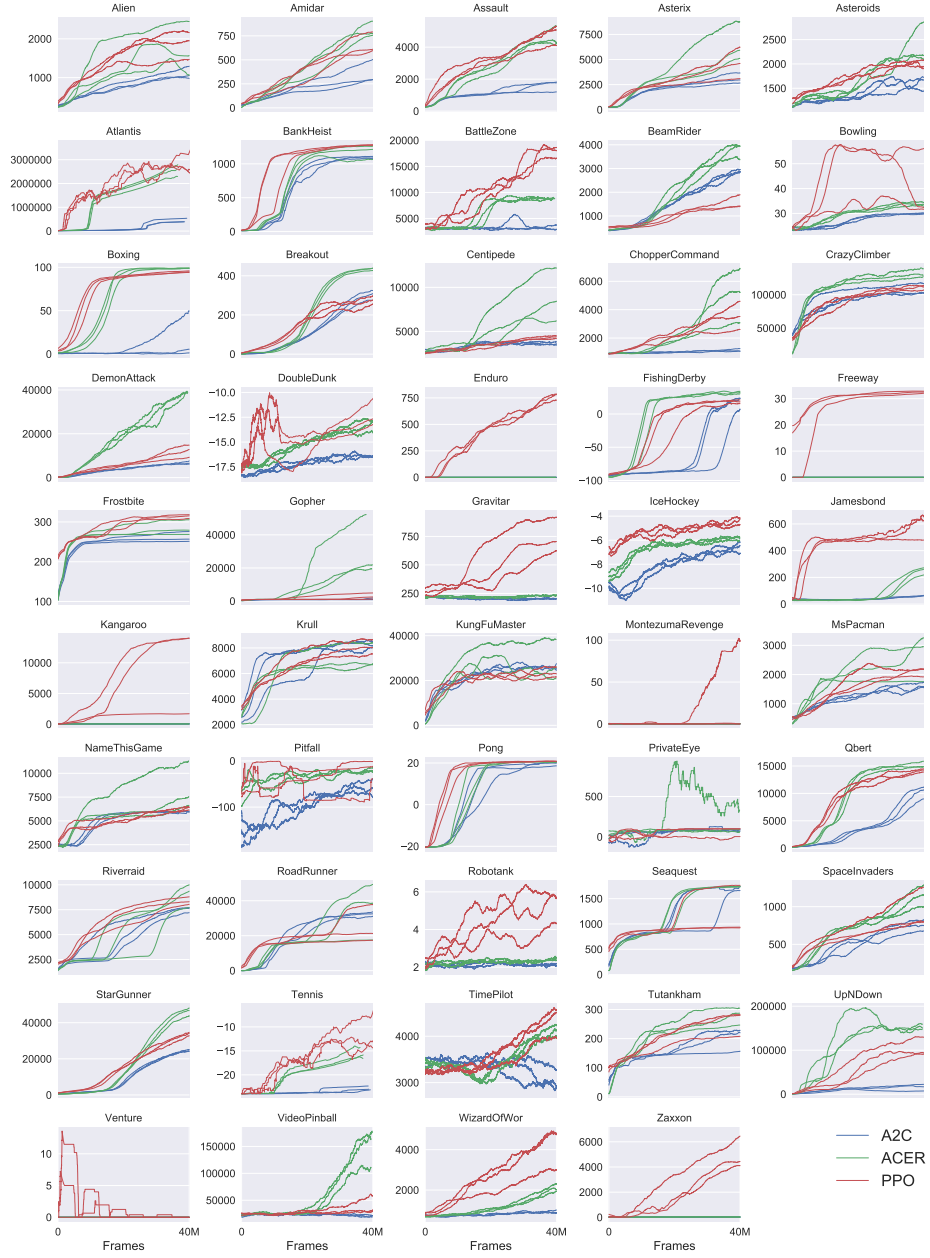


Figure 6: Comparison of PPO and A2C on all 49 ATARI games included in OpenAI Gym at the time of publication.

	A2C	ACER	PPO
Alien	1141.7	1655.4	1850.3
Amidar	380.8	827.6	674.6
Assault	1562.9	4653.8	4971.9
Asterix	3176.3	6801.2	4532.5
Asteroids	1653.3	2389.3	2097.5
Atlantis	729265.3	1841376.0	2311815.0
BankHeist	1095.3	1177.5	1280.6
BattleZone	3080.0	8983.3	17366.7
BeamRider	3031.7	3863.3	1590.0
Bowling	30.1	33.3	40.1
Boxing	17.7	98.9	94.6
Breakout	303.0	456.4	274.8
Centipede	3496.5	8904.8	4386.4
ChopperCommand	1171.7	5287.7	3516.3
CrazyClimber	107770.0	132461.0	110202.0
DemonAttack	6639.1	38808.3	11378.4
DoubleDunk	-16.2	-13.2	-14.9
Enduro	0.0	0.0	758.3
FishingDerby	20.6	34.7	17.8
Freeway	0.0	0.0	32.5
Frostbite	261.8	285.6	314.2
Gopher	1500.9	37802.3	2932.9
Gravitar	194.0	225.3	737.2
IceHockey	-6.4	-5.9	-4.2
Jamesbond	52.3	261.8	560.7
Kangaroo	45.3	50.0	9928.7
Krull	8367.4	7268.4	7942.3
KungFuMaster	24900.3	27599.3	23310.3
MontezumaRevenge	0.0	0.3	42.0
MsPacman	1626.9	2718.5	2096.5
NameThisGame	5961.2	8488.0	6254.9
Pitfall	-55.0	-16.9	-32.9
Pong	19.7	20.7	20.7
PrivateEye	91.3	182.0	69.5
Qbert	10065.7	15316.6	14293.3
Riverraid	7653.5	9125.1	8393.6
RoadRunner	32810.0	35466.0	25076.0
Robotank	2.2	2.5	5.5
Seaquest	1714.3	1739.5	1204.5
SpaceInvaders	744.5	1213.9	942.5
StarGunner	26204.0	49817.7	32689.0
Tennis	-22.2	-17.6	-14.8
TimePilot	2898.0	4175.7	4342.0
Tutankham	206.8	280.8	254.4
UpNDown	17369.8	145051.4	95445.0
Venture	0.0	0.0	0.0
VideoPinball	19735.9	156225.6	37389.0
WizardOfWor	859.0	2308.3	4185.3
Zaxxon	16.3	29.0	5008.7

Table 6: Mean final scores (last 100 episodes) of PPO and A2C on Atari games after 40M game frames (10M timesteps).